

EMBEDDED SYSTEMS & LINUX TRAINING PACKAGE

A Rigorous, First-Principles Curriculum for ECE/CSE Graduates

Duration: 36 Weeks | **Level:** Beginner to Job-Ready

Author: Senior Embedded Systems Mentor & Curriculum Builder

Workspace Root: /home/siva/sivaramireddy/Project1/

Master Curriculum: Beginner-to-Job-Ready Embedded Systems & Linux (36-Week Program)

Welcome to the ultimate, highly rigorous training curriculum designed to transition a fresh Electronics and Communication Engineering (ECE) or Computer Science Engineering (CSE) graduate into a production-capable, highly competitive Embedded Systems and Linux Engineer.

Target Goal

By the end of this 36-week program, you will not just know concepts; you will have written bare-metal peripheral drivers for ARM Cortex-M microcontrollers, built real-time scheduling applications using FreeRTOS, written multi-threaded Unix systems programming solutions, compiled custom Linux kernels and device trees, created custom distributions using Buildroot and Yocto, and developed functional Linux character device drivers.

Workspace Directory Layout

This repository is structured to mirror professional software development environments. Every week, module, and configuration is modularized:

```

/home/siva/sivaramireddy/Project1/
├── README.md # Curriculum Overview, Hardware checklist &
Onboarding (This file)
├── trainer-operating-rules.md # The pedagogical "constitution" and review
rules for mentors
├── 36-week-roadmap.md # Timeline breakdown across 6 major milestones
├── weekly-trainer-template.md # Template used to maintain weekly plan
consistency
├── daily-study-tracker.md # Standardized daily execution and reflection
sheet
├── habit-scorecard.md # Accountability matrix tracking code-style &
professional discipline
├── milestone-scorecard.md # Pass/fail rubric gatekeepers for each phase
transition
├── resources.md # Comprehensive guide to datasheets, textbooks,
and software
├── plans/ # The core execution engine (36 weekly master
plans)
│   ├── week-01-execution-plan.md # Phase 1: Git & Dev Setup
│   ├── week-02-execution-plan.md # Digital Logic & Number Systems
│   ├── ...
│   └── week-36-execution-plan.md # Phase 6: Final Capstone & Job Readiness

```

Hardware Requirements (Target Development Kit)

To complete this curriculum successfully, you will need physical hardware. Simulation is discouraged for peripheral interfacing.

- Host Computer:** A workstation running a native Linux distribution (preferably Ubuntu 22.04 LTS or Debian 12), or a macOS/Windows machine running Ubuntu 22.04 LTS via a dedicated virtual machine or Dual Boot. Windows WSL2 is acceptable but requires advanced USB-IP mapping for hardware debugging.
- Microcontroller Board:** **STM32F407G-DISC1** (Discovery board with STM32F407VG MCU, ARM Cortex-M4 with FPU) or **NUCLEO-F429ZI**. These are highly documented industry-standard boards.
- Embedded Linux Board:** **Raspberry Pi 4 Model B (4GB or 8GB)** or **BeagleBone Black (BBB)**. Both are excellent for device tree, custom kernel, and driver development.
- Debugging and Prototyping Gear:**
 - FTDI USB-to-UART TTL Serial Adapter (e.g., FT232RL or CH340G).
 - 8-Channel USB Logic Analyzer (24MHz sampling rate, compatible with Sigrok/PulseView).
 - High-quality solderless breadboard, premium jumper wires (M-M, M-F, F-F).
 - I2C peripheral (e.g., MPU6050 6-Axis Accelerometer/Gyro or BMP280 Barometric Pressure Sensor).
 - SPI peripheral (e.g., RC522 RFID module or a SPI OLED Display SSD1306).
 - Standard electronics toolkit: Multimeter, resistors (220Ω, 1kΩ, 10kΩ), LEDs, tactile pushbuttons.

Software Prerequisites (Host Setup)

We will install and configure all of these tools during the first two weeks of the curriculum: - **Build Utilities:** `build-essential`, `gcc`, `g++`, `make`, `cmake`. - **Cross-Compilation Toolchains:** `gcc-arm-none-eabi` (for bare-metal), `gcc-aarch64-linux-gnu` / `gcc-arm-linux-gnueabi` (for Embedded Linux target). - **Embedded Debugging Tools:** `openocd`, `gdb-multiarch`, `minicom`, `picocom`, `screen`. - **System Analysis & Diagnostics:** `strace`, `ltrace`, `ldd`, `nm`, `objdump`, `readelf`, `valgrind`, `htop`, `wireshark`. - **Version Control & Repository Discipline:** `git`, `github-cli` (optional). - **Embedded Linux Environment:** `buildroot`, `yocto` (poky repository), `u-boot-tools`.

How to Utilize This Curriculum

1. **Consult the Roadmap:** Open [36-week-roadmap.md](https://github.com/Embedded-Systems-Linux/36-week-roadmap.md) to understand the long-term scope and milestones.
 2. **Fill in Trackables Daily:** Every morning, copy the template in [daily-study-tracker.md](https://github.com/Embedded-Systems-Linux/daily-study-tracker.md) to log your work, and update your [habit-scorecard.md](https://github.com/Embedded-Systems-Linux/habit-scorecard.md).
 3. **Execute Weekly Plans:** Enter the `plans/` folder and open the corresponding execution plan for the week. Follow the day-by-day instructions precisely.
 4. **Grading Gatekeepers:** At the end of each milestone phase (6 weeks), you must complete the practical assignment defined in [milestone-scorecard.md](https://github.com/Embedded-Systems-Linux/milestone-scorecard.md) and have it reviewed by a mentor or cross-checked against the Trainer Operating Rules.
-

The Educational Philosophy

This program operates under a **no-black-box** policy: - **No HAL Libraries:** You will not use CubeMX HAL or Arduino-style libraries during the micro-controller phase. You will configure clocks, timers, and serial channels by direct register writes using pointers and memory-mapped register offsets based strictly on target datasheets and reference manuals. - **No Pre-Packaged Kernels:** You will not use standard pre-built Linux distributions. You will fetch the mainline Linux kernel and U-boot source code, write a custom device tree, cross-compile the architecture, and structure a custom Root Filesystem from scratch. - **Strict Coding Standards:** Every C file must conform to MISRA-C guidelines (where appropriate) and be checked with static analysis tools. Commits must be atomic and follow conventional commit styling.

Let the journey to becoming a world-class systems engineer begin!

Trainer Operating Rules & Pedagogical Guidelines

This document serves as the "constitution" for the trainer (or self-studying student acting as an evaluator) of this curriculum. It outlines the pedagogical philosophy, the code review process, strict grading criteria, and instructions for managing the student's learning progression.

1. Core Pedagogical Philosophy

1.1 The "No-Black-Box" Rule

The absolute core of this curriculum is deep, first-principles technical comprehension. The following tools and practices are strictly prohibited until specifically introduced in Phase 6: * **NO vendor HALs:** ST HAL, LL, TI-Drivers, Arduino core, etc. are banned. * **NO graphical configuration tools:** CubeMX, MCC, SysConfig, etc. are banned. Clocks, pins, and peripherals must be calculated and written directly into control registers using raw C pointers and memory-mapped offsets. * **NO pre-packaged OS distributions:** Running standard Raspberry Pi OS, Debian, or Armbian is forbidden during the Embedded Linux phase. The system must be bootstrapped via custom SPL, U-Boot, DTB, Kernel, and a BusyBox/Buildroot-generated root file system.

1.2 The "Tee-junction" Theory of Learning

For every single abstraction (e.g., calling `printf()`), the student must know exactly what happens one level down: * `printf()` -> Calls POSIX `write()` -> Enters kernel VFS -> Triggers tty line discipline -> Triggers UART device driver -> Writes byte to physical UART Transmit Data Register (TDR). If a student cannot explain the lower-level mechanics of an abstraction they are using, they have failed that topic.

2. Code Review and Grading Rules

Every assignment must undergo a rigorous code review process. A submission is graded as **PASS** or **FAIL**. There are no partial grades.

2.1 Pass/Fail Assessment Criteria

A submission fails automatically if it violates any of the following parameters:

Criteria	Issue	Description
Architectural	Banned Abstractions	Use of ST HAL, Arduino library, or any pre-built helper.
Architectural	Floating States	Configuring input GPIOs without setting explicit pull-up/down resistors unless external pull-ups exist.
Safety / C	Undefined Behavior	Use of uninitialized variables, array index out of bounds, or signed overflow.
Safety / C	Dynamic Memory	Using <code>malloc()</code> / <code>free()</code> inside ISRs or bare-metal loops (except managed custom memory pools).
Clarity	Magic Numbers	Using raw hex/decimal values in register configuration instead of clear bitmask macros (e.g., <code>(1 << 5)</code> or named register macros).
Repo Hygiene	Bloated Commits	Committing compiled binaries (<code>.elf</code> , <code>.bin</code> , <code>.hex</code> , <code>.o</code>), editor backup files, or giant monolithic single commits.

2.2 The Code Quality Rubric

- 1. Strict Types:** The code must use explicit fixed-width integers from `<stdint.h>` (`uint8_t` , `int32_t` , etc.) instead of standard `int` , `short` , or `char` (unless dealing with ASCII text).
- 2. Volatile Correctness:** The `volatile` qualifier must be applied correctly to every variable accessed across execution contexts (e.g., modified inside an ISR and read in the main loop, or representing hardware registers).
- 3. Register Manipulation Cleanliness:** Register modifications must use clear mask operations:
- 4. Set bits:** `REG |= MASK;`
- 5. Clear bits:** `REG &= ~MASK;`
- 6. Toggle bits:** `REG ^= MASK;`
- 7. Read state:** `if ((REG & MASK) == EXPECTED) { ... }`
- 8. Error Handling:** Every single system call or peripheral API must check return values. Failure to handle a failed `malloc()` , `open()` , or failed I2C transaction is an automatic fail.

3. Weekly Grading & Progression Protocol

flowchart TD

```
Start[Student Submits Weekly Deliverables] → ReviewCommits{Review Git Commits}
ReviewCommits -- Non-Conventional / Monolithic → Fail[Fail: Refactor Repo & Re-submit]
ReviewCommits -- Clean / Conventional → BuildCheck{Build Code & Static Analysis}
BuildCheck -- Compiler Warnings / Static Failures → Fail
BuildCheck -- Warnings Clean → ReviewCode{Manual Code Review}
ReviewCode -- Violates Rules or Quality Rubric → Fail
ReviewCode -- Code Approved → RunTest{Functional / Hardware Verification}
RunTest -- Fails Functional / Timings Check → Fail
RunTest -- Passes → Pass[Pass: Advance to Next Week]
```

3.1 Weekly Review Steps

1. **Audit Git History:** The trainer runs `git log --graph --oneline` on the student's repository. Commits must be structured (e.g., `feat(uart): implement basic polling transmitter`). Monolithic, single-commit weeks are rejected.
2. **Compiler Warning Check:** Compile the project with `-Wall -Wextra -Werror -pedantic`. If there is even one warning, the week is rejected.
3. **Static Analysis:** Run `cppcheck` or `clang-tidy` on the source folder. Any memory leaks, dead code, or styling violations must be resolved.
4. **Functional Testing on Hardware:** Flash the binary to the microcontroller/host and hook up the USB Logic Analyzer.
5. **Example UART Check:** The trainer must capture the Tx line on the Logic Analyzer and verify that the Baud Rate timing matches expectations within $\pm 2\%$.
6. **Example Driver Check:** The driver must not lock up in infinite polling loops. Timeout guards must be implemented for all loop checking flag-registers (e.g., wait for TXE flag with a timeout decrement counter).

4. Remediation and Mentor Rules

- **No Code Spoon-Feeding:** When a student fails a code review, the trainer must not write the correct code for them. The trainer should provide the file name, line range, and a brief conceptual hint (e.g., "Look up the reset state of GPIOx_MODER in the reference manual, and think about what happens to the default value if you do not clear the register before setting your new mode").

- **Weekly Retrospective:** Every Friday, hold a 15-minute review where the student must physically explain their code and draw the architectural memory map or hardware timing diagram on a whiteboard.

Master Roadmap: 36-Week Embedded Systems & Linux curriculum

This document provides a birds-eye timeline of the entire 36-week program, broken down into six major phases. Each phase represents a thematic block ending in a gatekeeper milestone challenge.

High-Level Phase Overview

Phase	Duration	Focus Area	Capstone Gateway
Phase 1	Weeks 01–06	C Programming & Digital Logic	Custom Allocator & Software Logic Gate Simulator
Phase 2	Weeks 07–12	Cortex-M MCU Bare-Metal Drivers	Full Bare-Metal I2C Peripheral Driver (MPU6050)
Phase 3	Weeks 13–18	Systems Programming & Linux Toolchain	Multi-Threaded IPC Network Packet Processor
Phase 4	Weeks 19–24	Real-Time Operating Systems (RTOS)	FreeRTOS-Based Data Logger with Interrupt-to-Task DMA
Phase 5	Weeks 25–30	Embedded Linux & Kernel Modules	Linux Kernel Driver for HW Peripheral with Sysfs & Interrupts
Phase 6	Weeks 31–36	Buildroot, Yocto & Production Capstone	Yocto-built Real-time Sensor Gateway with Remote Web/API Interface

Weekly Curriculum Timeline

Phase 1: Foundations of Embedded C & Digital Logic (Weeks 01–06)

- **Goal:** Establish ironclad C programming skills, repo hygiene, development environments, and a foundational understanding of hardware logic and memory mapping.
- **Week 01:** Dev Environment Setup, Git Workflows, and Conventional Commits.
- **Week 02:** Digital Logic Refresh, Number Systems (Bin, Hex, Oct), Logical Operators & Gates.
- **Week 03:** Embedded C Fundamentals: Types, Constants, Volatile, Compilation Pipeline.
- **Week 04:** Advanced Pointers, Memory Layout of C, and Dynamic Allocations.

- **Week 05:** Bitwise Operations, Bitfields, and Register Manipulation Techniques.
- **Week 06:** Structs, Unions, Typedefs, Memory Padding/Alignment, and the C Preprocessor.
- **Milestone 1 Gatekeeper Check:** Complete and defend Phase 1 Milestone.

Phase 2: MCU Architecture & Bare-Metal Driver Development (Weeks 07–12)

- **Goal:** Master ARM Cortex-M architecture, read technical datasheets, register-map calculations, and write custom peripheral drivers without HALs.
- **Week 07:** ARM Cortex-M4 Architecture, Vector Table, Memory Map, Registers.
- **Week 08:** GPIO Driver Development (Clocks, Pins, Electrical configurations).
- **Week 09:** Nested Vectored Interrupt Controller (NVIC), Exceptions, Race Conditions.
- **Week 10:** Timers & Counters (SysTick, General Purpose, Input Capture, Output Compare).
- **Week 11:** Communication Protocols I: UART (Baud Rate, Polling, Interrupts, Ring Buffers).
- **Week 12:** Communication Protocols II: I2C & SPI (Timings, Master/Slave, Register maps).
- **Milestone 2 Gatekeeper Check:** Complete and defend Phase 2 Milestone.

Phase 3: Systems Programming & Linux Essentials (Weeks 13–18)

- **Goal:** Transition to Linux environments, command line mastery, compiling mechanics, low-level I/O, IPCs, and concurrency.
- **Week 13:** Linux OS Command Line Mastery, FHS, and System Configuration.
- **Week 14:** Compiler Toolchains: GCC, Makefiles, Linker Scripts, ELF File Anatomy.
- **Week 15:** Low-Level System Call File I/O (`open` , `read` , `write` , `ioctl` , `mmap`).
- **Week 16:** Linux Process Control (`fork` , `exec` , `wait` , IPC: Pipes, FIFOs, Shared Memory).
- **Week 17:** POSIX Threads (`pthread` , Mutexes, Semaphores, Thread Safety, Race Conditions).
- **Week 18:** Linux Shell Scripting & Automation (Bash, `sed` , `awk` , Performance tools).
- **Milestone 3 Gatekeeper Check:** Complete and defend Phase 3 Milestone.

Phase 4: Real-Time Operating Systems (RTOS) (Weeks 19–24)

- **Goal:** Shift from polling loops to real-time scheduling, concurrency, resource allocation, and synchronization under FreeRTOS.
- **Week 19:** Introduction to RTOS, FreeRTOS Port, Task Creation, Idle Task.
- **Week 20:** Task Scheduler, Co-operative vs Preemptive, Context Switch Mechanics.
- **Week 21:** Task Synchronization: Binary/Counting Semaphores, Mutexes, Priority Inversion.
- **Week 22:** Inter-Task Communication: Message Queues, Event Groups, Task Notifications.
- **Week 23:** Memory Management (heap_1 to heap_5), Memory Profiling, Stack Overflow Protection.
- **Week 24:** Software Timers, ISR-to-Task Communication (`FromISR` APIs), RTOS Debugging.
- **Milestone 4 Gatekeeper Check:** Complete and defend Phase 4 Milestone.

Phase 5: Embedded Linux & Kernel Drivers (Weeks 25–30)

- **Goal:** Build custom Embedded Linux architectures, compile kernels, construct device trees, and write physical kernel-space drivers.
- **Week 25:** Embedded Linux Architecture & Boot Flow (ROM, SPL, U-Boot, Kernel, Rootfs).
- **Week 26:** Cross-Compilation, Custom Kernel Compilation, Serial Consoles, Boot Log Analysis.
- **Week 27:** Device Trees (DTS/DTSI/DTC), Nodes, Properties, Custom Device Bindings.
- **Week 28:** Linux Kernel Modules (Build rules, LKM structure, Parameters, `printk`).
- **Week 29:** Character Drivers (FOPs, Major/Minor, Class, Dev Nodes, Kernel I/O buffers).
- **Week 30:** Kernel Hardware Access (GPIO, Memory Map, Virtual Memory, Kernel Interrupts).
- **Milestone 5 Gatekeeper Check:** Complete and defend Phase 5 Milestone.

Phase 6: Buildroot, Yocto, & Advanced Capstone (Weeks 31–36)

- **Goal:** Master system integration engines, debug production systems, write network socket layers, and deliver a production capstone product.
- **Week 31:** Rootfs Automation with Buildroot (Configs, overlays, package generation).
- **Week 32:** Production-grade OS Creation with Yocto (BitBake, Recipes, Layers, Classes).
- **Week 33:** Real-World Peripheral Integration: Analog-to-Digital (ADC) & PWM Drivers.
- **Week 34:** Network Programming (Socket API, TCP/UDP client-server, embedded HTTP/REST).
- **Week 35:** Advanced Debugging & Profiling (Valgrind, GDB Server, Strace, LDD, NM, Perf).
- **Week 36:** Capstone Project Delivery, Portfolio Defense, System Design Mock Interviews.
- **Milestone 6 Gatekeeper Check:** Capstone evaluation and graduation.

Graduation and Career Readiness

Upon passing all 6 Milestone Gatekeeper evaluations, you will undergo target resume customization, comprehensive LinkedIn profiling, and mock technical grilling sessions centered strictly around the real hardware and kernel driver modules built over the course of this training program.

Weekly Execution Plan: Week XX (Sample Template)

This document is the standard format for all weekly execution plans.

Weekly Goal

A one-to-two sentence explanation of what specific component, mechanism, or principle the student is mastering this week.

Weekly Outcome

A bulleted list of actual, tangible skills and coding capabilities the student will possess and be able to demonstrate on physical hardware or the Linux host by Friday evening.

Session Breakdown (Monday - Friday)

Monday: [Focus Topic Name]

- **Conceptual Objective:** Core concepts, math, specifications, registers, or kernel structures to learn today.
- **Practical Activity:** Step-by-step programming, command line execution, or hardware testing.
- **Code/Circuit Reference:**

```
// Example code or config structure showing proper style
```

Tuesday: [Focus Topic Name]

- **Conceptual Objective:** Core concepts...
- **Practical Activity:** Practical steps...
- **Code/Circuit Reference:**

```
// Code structure
```

Wednesday: [Focus Topic Name]

- **Conceptual Objective:** Core concepts...
- **Practical Activity:** Practical steps...
- **Code/Circuit Reference:**

```
// Code structure
```

Thursday: [Focus Topic Name]

- **Conceptual Objective:** Core concepts...
- **Practical Activity:** Practical steps...
- **Code/Circuit Reference:**

```
// Code structure
```

Friday: [Focus Topic Name]

- **Conceptual Objective:** Core concepts...
- **Practical Activity:** Practical steps...
- **Code/Circuit Reference:**

```
// Code structure
```



Hands-On Assignment: [Assignment Title]

Functional Requirements

A comprehensive list of strict specifications the project must meet.

Structural Requirements & Code Skeleton

The architectural layout of files, dependencies, and code boundaries.

```
/* Skeleton showing architectural partitioning */
```

Hardware Connection Schematic (If Applicable)

ASCII representation or pinout configuration table mapping MCU to peripheral/analyzer.

Deliverables

A clear checklist of what must exist in the student's repository at the time of review. * [] **Code Directory:** Path to source code. * [] **Compilation Artifacts:** Clean compilation logs. * [] **Repository Commits:** Expected commit count/style (Conventional Commits). * [] **Verification Media:** Screenshots of serial outputs, logs, or logic analyzer captures.

? Self-Check Questions

Five hard, deep-dive conceptual or troubleshooting questions that the student must be able to answer during Friday's review. 1. Question 1? 2. Question 2? 3. Question 3? 4. Question 4? 5. Question 5?

Stretch Task (Optional)

An extremely demanding extension of the weekly assignment designed for top 5% performers.

Motivation Checkpoint

Industry context, explaining how this week's topic directly maps to real-world embedded or automotive systems, or consumer product engineering.

Trainer Review Rule

The exact validation procedure the trainer will execute, including logic analyzer settings, terminal inputs, compile-time flags, or test scripts. * **Pass Criteria:** * **Fail Criteria:**

Daily Study & Reflection Tracker

Students should copy this template at the beginning of each day to maintain an active log of their learning journey, debug logs, and architectural realizations.



Date: [YYYY-MM-DD] | **Week:** [01-36] | **Day:** [Mon/Tue/Wed/Thu/Fri]



Time Allocation & Focus Blocks

- **Block 1 (AM - Core Study):** [Start Time] to [End Time] (Focus topic: ____)
 - **Block 2 (PM - Hands-on Lab):** [Start Time] to [End Time] (Focus topic: ____)
 - **Total Focus Hours:** ____ Hours
-



Daily Deep-Dive & Realizations

Summarize today's core technical takeaways in your own words. Do not copy paste. Describe the underlying hardware mechanics, system call interactions, or register logic you unlocked. * **Key Takeaway 1:** * **Key Takeaway 2:** * **Key Takeaway 3:**



Debugging Log & Technical Blockages

Embedded systems engineers spend 80% of their time debugging. Document any bugs, compiler errors, linker failures, or hardware lock-ups you faced today, how you diagnosed them, and how you resolved them.

Bug 1: [Short Title, e.g., UART Baud Rate Off-by-Half]

- **Symptom:** (e.g., Serial terminal outputs garbage characters at 115200 baud).
- **Diagnosis Steps Taken:** (e.g., Connected logic analyzer, measured Tx bit width. Found bit width was 17.3µs instead of expected 8.68µs. Baud rate was running at ~57600).
- **Root Cause:** (e.g., MCU core clock configuration register was left at default, meaning APB1 clock was 8MHz instead of 16MHz. Baud rate calculation divisor was wrong).
- **Resolution:** (e.g., Modified `system_stm32f4xx.c` to enable PLL and configure APB1 prescaler correctly, matching UART divisor parameters).

Bug 2: [Short Title]

- Symptom:
 - Diagnosis Steps:
 - Root Cause:
 - Resolution:
-

Git Checkpoint & Commit Summary

Ensure you are pushing atomic, clean commits every day. Check your local Git log. * **Latest Commit Hash:** [Insert Commit SHA-1] * **Commit Message:** [e.g., feat(uart): add interrupt-driven Tx support with ring buffer] * **Changed Files:** * [] File 1 * [] File 2

Tomorrow's Game Plan

State the specific goals, register targets, or software layers you will write tomorrow to stay on pace. 1. 2. 3.

Professional Engineering Habit Scorecard

This accountability scorecard is designed to track non-functional engineering habits that separate novice coders from elite, industry-ready Systems Engineers. Track these parameters weekly.



Habit Categories & Scoring Grid

Rate yourself or your student weekly from **1 (Poor)** to **5 (Outstanding)** in each of the following areas:

Habit Category	Core Engineering Principle	Week 1-6	Week 7-12	Week 13-18	Week 19-24	Week 25-30	Week 31-36
Git Hygiene	Conventional commits, atomic sizing, clean branching.						
Warnings Clean	Zero compilation warnings (compiled with <code>-Werror</code>).						
Static Compliance	Static analysis (Cppcheck/Tidy) runs with 0 errors.						
No Magic Numbers	Bit masks and pin macros utilized instead of raw numbers.						
Hardware Care	Safe wiring, ESD grounding, power rails double-checked.						
Defensive I/O	All system calls and peripheral API return values checked.						
Memory Rigor	No unmanaged pointers, no stack overflows, no heap leaks.						
Documentation	Code comments describe "Why", not "What". logs kept.						

Scoring Definitions & Expectations

1. Git Hygiene & Repo Discipline

- **5 (Outstanding):** Each feature, driver, or peripheral addition resides on a separate Git branch. Pull requests contain clear descriptions of changes. Commits are atomic (no two unrelated changes in a single commit) and strictly follow Conventional Commits formatting (e.g., `feat(gpio): implement debounce logic via software timer`).
- **3 (Acceptable):** Commits occur daily on the `main` branch. Commit messages are clear but do not follow the strict Conventional Commits syntax.
- **1 (Failure):** Multi-week monolithic commits. Git history consists of generic statements like "fixed code", "debugging", or "final push". Compiled files (`.elf` , `.bin` , `.o`) are committed.

2. Compilation and Code Safety Compliance

- **5 (Outstanding):** Code compiles flawlessly with `-Wall -Wextra -pedantic -Werror` on the host, or equivalent strict flags on the cross-compiler. Zero compiler warnings exist. Cppcheck static analyzer is integrated into the build pipeline and yields zero errors or style warnings.
- **3 (Acceptable):** Code compiles with warnings. Warnings are ignored as long as the binary functions correctly. No static analysis is performed.
- **1 (Failure):** Project compiles only after modifying compiler configurations to ignore warning errors, or fails to build completely.

3. Hardware Care & Handling Safety

- **5 (Outstanding):** Before powering any custom circuit, the student uses a multimeter to verify zero short-circuits between the positive power rail (3.3V or 5V) and Ground (GND). The board is kept on an ESD-safe mat. The microcontroller is disconnected from USB power before changing breadboard wire connections.
- **3 (Acceptable):** Jumper wires are hot-swapped while the micro-controller is powered. The board sits on top of standard conductive surfaces (metal/static bags). No short-circuit checking is done, relying on the USB hub's overcurrent protection.
- **1 (Failure):** Microcontroller board is permanently damaged due to shorts, reverse-polarity power connection, or ESD discharge due to gross negligence.

4. Code Standards & Architecture (Anti-Magic Numbers)

- **5 (Outstanding):** Every register manipulation is explicitly detailed using shift operators and pre-defined register structures.
- **Correct:** `GPIOA→MODER &= ~(3U << (5 * 2)); GPIOA→MODER |= (1U << (5 * 2)); // Configure PA5 as output`
- **Incorrect:** `GPIOA→MODER = 0x00000400; // PA5 output (magic value resets other pins!)`

- **3 (Acceptable):** Masking is done but magic hex constants are used without clear comments explaining the calculation.
- **1 (Failure):** Code is packed with direct decimal/hex writes to registers, making it impossible to read without having the processor's 1000-page Reference Manual open on a matching page.

Milestone Phase Gatekeeper Scorecard

To transition from one phase of the curriculum to the next, the student must pass the corresponding Milestone Gatekeeper Challenge. A mentor must evaluate these challenges, or the student must rigorously self-verify using logic analyzers and system analysis tools.



Milestone 1: Foundations (End of Week 06)

- **Goal:** Prove advanced C programming mastery and low-level memory structure comprehension.
 - **Challenge:** Create a custom **Dynamic Memory Allocator (custom malloc/free)** in pure C, and write a **Software Digital Logic Gate Simulator** that builds logic gates (AND, OR, XOR, NOT, NAND, NOR) out of software structures and uses the allocator to dynamically allocate gate nodes and link signals.
 - **Rubric & Constraints:**
 - **Custom Allocator:** Must manage a static `uint8_t memory_pool[4096]` array using a Linked List of free/allocated blocks.
 - **No Standard Lib:** Banned use of standard `<stdlib.h>` allocation tools (no `malloc`, `realloc`, `free`).
 - **Logic Gate Simulator:** Connect multiple virtual gates to form a 4-bit Binary Full Adder in software. Must track propagation delays using simulated clock ticks.
 - **Safety:** Clean compilation with zero memory leaks (verified using a memory-tracing hook that counts allocations and frees, printing out residual blocks at exit).
-



Milestone 2: MCU Bare-Metal Drivers (End of Week 12)

- **Goal:** Prove absolute bare-metal registers control and peripheral synchronization.
- **Challenge:** Write a complete **Bare-Metal Device Driver for the MPU6050 6-Axis Accelerometer/Gyroscope Sensor** using custom I2C master peripheral routines written strictly by register manipulation on the STM32F4.
- **Rubric & Constraints:**
 - **Hardware I2C Register Writes:** Configure clocks (RCC), pins (GPIO alternative functions, open-drain mode, external/internal pull-ups), and I2C registers (I2C1->CR1, CR2, OAR1, CCR, TRISE, SR1, SR2) from scratch.
 - **Zero HAL:** Use of `HAL_I2C_Init()`, `HAL_I2C_Master_Transmit()`, or similar is an automatic failure.

- **Timeout Security:** Every register polling loop must contain a timeout decrementor guard (e.g.,

```
uint32_t timeout = 100000; while (!(I2C1→SR1 & I2C_SR1_TXE)) { if (--timeout == 0)
return ERR_TIMEOUT; } ) to prevent software lock-ups.
```
 - **Logic Analyzer Proof:** Record I2C frames at 100kHz standard speed. Show proper start bits, address frames, register selects, acknowledgements (ACK/NACK), data reads, and stop bits. Baud timing must be within $\pm 2\%$ of spec.
-



Milestone 3: Systems Programming (End of Week 18)

- **Goal:** Prove proficiency in multi-processing, multi-threading, concurrency, and Linux system calls.
 - **Challenge:** Build a **Multi-Threaded IPC Network Packet Processor** in Linux.
 - **Rubric & Constraints:**
 - **Process Division:** A parent process spawns a dedicated high-priority child process (representing hardware interfaces) and a team of worker threads.
 - **IPC Mechanics:** The child process generates dummy TCP/IP packets and feeds them to the parent process using a Shared Memory segment (`shmget` / `shmat`) synchronized via POSIX Named Semaphores (`sem_open`).
 - **Multi-threaded Thread Pool:** The parent process receives packets and schedules them across a thread pool of 4 worker threads. Worker threads are coordinated via a Thread-Safe FIFO Queue, synchronized using Mutexes (`pthread_mutex_t`) and Condition Variables (`pthread_cond_t`).
 - **No Data Corruption:** Workers parse packets, extract payload strings, and write statistics to a local log file using system-level I/O (`open` , `write` , `flock` for write locking). Zero race conditions allowed.
-



Milestone 4: Real-Time Operating Systems (End of Week 24)

- **Goal:** Transition from bare-metal loops to deterministic, real-time operating parameters.
- **Challenge:** Build an **RTOS-based Smart Battery Monitor & Data Logger** using FreeRTOS on the STM32F4.
- **Rubric & Constraints:**
 - **Task Partitioning:**
 - **Telemetry Task (Priority High):** Polls ADC peripheral for voltage/current values every 10ms. Uses Direct DMA from ADC to Memory.
 - **Logging Task (Priority Medium):** Receives sensor telemetry packets via FreeRTOS Message Queues and formats data.
 - **Display/Console Task (Priority Low):** Outputs statistics to the UART console.

- **Synchronization:** Uses a FreeRTOS Mutex with Priority Inheritance enabled to share memory parameters between Telemetry and Console tasks, preventing Priority Inversion.
 - **Real-Time Guarding:** Timer interrupts must trigger Task Notifications (`vTaskNotifyGiveFromISR`) to unblock the Telemetry task dynamically, avoiding latency.
 - **Stack Profiling:** Enable stack overflow checking hook (`vApplicationStackOverflowHook`). Demonstrate that all tasks run within bounds and record heap stats.
-

Milestone 5: Embedded Linux & Drivers (End of Week 30)

- **Goal:** Prove mastery of custom OS compilation, device tree mapping, and kernel-space drivers.
 - **Challenge:** Write a **Linux Out-of-Tree Kernel Character Device Driver** for a custom physical peripheral (e.g., controlling a hardware external LED array and reading a physical button input using hardware interrupts) on the Raspberry Pi 4.
 - **Rubric & Constraints:**
 - **Device Tree Overlay:** Write a Device Tree Overlay (`.dts`) defining custom GPIO pins, setting pin directions, and setting compatibility strings. Compile using `dtc` and load into the system boot configuration.
 - **Character Driver Registration:** Dynamic major/minor allocation using `alloc_chrdev_region()`, file operations (`read` , `write` , `open` , `release` , `ioctl`).
 - **Hardware Interaction:** Access physical memory space by mapping base addresses with `ioremap()` , modifying target registers cleanly, and safely handling user-space buffers with `copy_to_user()` and `copy_from_user()` .
 - **Interrupt Service Routine:** Register button GPIO as an interrupt line using `request_threaded_irq()` . Implement a bottom-half deferred mechanism using Tasklets or Workqueues to handle debouncing and log events to standard kernel buffer (`printk`).
-

Milestone 6: Capstone Project (End of Week 36)

- **Goal:** Absolute job-readiness. Complete system integration, custom toolchains, real-time communications, and production styling.
- **Challenge:** Build an **Industrial IoT Gateway & Sensor Hub (Yocto-based)**.
- **Rubric & Constraints:**
 - **Distribution OS:** Create a fully custom, lightweight Linux distribution utilizing **Yocto Project (Poky)**. Create a custom hardware layer (`meta-industrial-gateway`) containing target system configs, kernel configurations, and custom application recipes.
 - **Communication Infrastructure:** The Gateway collects real-time sensor data from downstream STM32F4 client nodes (running FreeRTOS) communicating over SPI or UART.

- **Secure Edge Processing:** The Gateway parses data in user space via a multi-threaded C/C++ application running as a systemd service (`systemctl`), publishing telemetry via secure network sockets (TCP/IP) or MQTT to a local server dashboard.
- **Diagnostics:** Must present detailed execution reports confirming zero memory leaks (Valgrind clean), system performance logs (collected via `perf`), and systemd boot time optimization (total boot time to user-space main application under 8 seconds).

Reference Manuals, Textbooks & Engineering Resources

This document is the master index of all critical reference manuals, textbooks, online resources, and documentation required during the 36-week curriculum. You should refer to these resources constantly.



Essential Textbooks & Reference Literature

Embedded C & Systems Foundations

1. **"The C Programming Language" (2nd Edition)** – Brian W. Kernighan & Dennis M. Ritchie (The absolute classic for C basics and layout).
2. **"Embedded C Coding Standard"** – Michael Barr (Barr Group) (A must-read for developing clean, bug-free, safe C code).
3. **"Making Embedded Systems: Design Patterns for Great Software"** – Elecia White (Excellent architectural design patterns).

MCU Architecture & Hardware Development

1. **"The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors"** – Joseph Yiu (The absolute bible for understanding vector tables, registers, exceptions, and NVIC).
2. **"STM32F407 Reference Manual (RM0090)"** – STMicroelectronics (1700+ pages of registers, registers, and registers. You will read this weekly in Phase 2).
3. **"STM32F407G-DISC1 User Manual (UM1472)"** – STMicroelectronics (Pinouts, board layout, and schematics).

Linux & Systems Programming

1. **"The Linux Programming Interface (TLPI)"** – Michael Kerrisk (1500+ pages. The absolute authority on Linux system calls, processes, threads, memory, and sockets).
2. **"Linux System Programming"** – Robert Love (Practical systems programming, filesystems, and process model).
3. **"Advanced Programming in the UNIX Environment (APUE)"** – W. Richard Stevens (Crucial for deep POSIX threading and sockets).

RTOS (Real-Time Operating Systems)

1. **"Mastering the FreeRTOS Real Time Kernel"** – Real Time Engineers Ltd. (The official and excellent user guide for FreeRTOS architectures).

Embedded Linux & Kernel Drivers

1. **"Mastering Embedded Linux Programming" (3rd Edition)** – Chris Simmonds (Excellent overview of U-Boot, Bootloaders, Buildroot, and Yocto).
 2. **"Linux Device Drivers" (3rd Edition)** – Jonathan Corbet, Alessandro Rubini, & Greg Kroah-Hartman (A bit older but still the classic conceptual reference for character drivers, locking, and memory maps).
 3. **"Linux Kernel Programming"** – Kaiwan N Billimoria (Modern kernel internals, memory management, and writing robust modules).
-

Software Toolchains & Utilities

Microcontroller Toolchain

- **Compiler:** GCC Cross-Compiler for ARM Cortex-M: `gcc-arm-none-eabi`.
- **Debugger Engine:** OpenOCD (Open On-Chip Debugger) – acts as a GDB server bridge to the physical ST-LINK hardware interface.
- **GDB Multiarch:** `gdb-multiarch` or `arm-none-eabi-gdb` – for physical source-level debugging inside terminal or IDE.
- **Logic Analyzer Suite:** Sigrok & PulseView – open-source software for analyzing physical bus logic captured on a logic analyzer.

Linux Systems Tools

- **Debugging & Memory Tracing:** `gdb` (GNU Debugger), `valgrind` (memory leak and access checker), `ltrace` (library calls), `strace` (system calls).
 - **System Metrics & Hardware Checks:** `lsof` (list open files), `ldd` (list dynamic library dependencies), `nm` (dump symbols from object file), `objdump` (disassemble and analyze binary structures).
-

Essential Hardware Datasheets

To write registers driver code, you must keep the following datasheets pinned in your browser tabs: *

STM32F407xx Datasheet (DS8626): Pin definitions, electrical characteristics, memory sizes. *

STM32F407xx Reference Manual (RM0090): Exhaustive details on register maps, bits, clock controls

(RCC), GPIO, UART, SPI, I2C, interrupts. * **MPU6050 Register Map and Description**: Required for Phase 2 Milestone, defining the I2C registers to configure registers, scales, and fetch raw accelerations/gyroscopes data. * **ARMv7-M Architecture Reference Manual**: Specific details on core registers, NVIC logic, assembly instructions, sleep modes.

Elite Online Communities & Reference Blogs

- **Interrupt by Memfault** (interrupt.memfault.com): The finest technical blog on embedded debugging, CI/CD, and advanced firmware patterns.
- **Embedded FM Podcast** (embedded.fm): Great conceptual discussions and developer interviews.
- **Kernel.org Documentation** (kernel.org/doc/html/latest/): Direct documentation for the Linux Kernel, driver interfaces, Device Tree syntax.

Weekly Execution Plan: Week 01

Weekly Goal

Establish a high-discipline, industry-standard development environment on the Linux host, master professional version control workflows using Git, and enforce strict commit structures utilizing Conventional Commits and Git hooks.

Weekly Outcome

By Friday, the student will have a fully bootstrapped Linux development environment (native or VM), configured custom bash alias controls, established global Git conventions, and written an automated shell-script Git hook (`commit-msg`) that programmatically blocks any commit violating Conventional Commit guidelines.



Session Breakdown (Monday - Friday)

Monday: The Linux Host & FHS Basics

- **Conceptual Objective:** Understand the Linux Directory Structure (Filesystem Hierarchy Standard - FHS). Comprehend the role of `/usr/bin` , `/usr/local/bin` , `/opt` , `/etc` , and user home paths in cross-compilation and toolchain management.
- **Practical Activity:**
 - Boot Linux host (Ubuntu 22.04 LTS native or VM).
 - Perform system package update.
 - Install core build tools: `sudo apt update && sudo apt install -y build-essential git openocd cppcheck clang-tidy valgrind minicom picocom`.
 - Create a clean workspace structure under `/home/$USER/workspace`.
- **Code/Circuit Reference:**

```
# Verify host compiler capability
gcc --version
make --version

# FHS Checklist
ls /
file /usr/bin/gcc
```

Tuesday: Advanced Bash Environment Configuration

- **Conceptual Objective:** Learn how the shell initializes (`.bashrc` vs `.bash_profile`), path variable manipulation (`PATH`), export dynamics, and configuring environment variables for embedded cross-compilers.
- **Practical Activity:**
 - Open `~/.bashrc` and construct custom aliases for system information and micro-controller connection monitoring.
 - Map a dedicated variable `EMBEDDED_TOOLS_DIR` to `/opt/toolchains`.
 - Source `.bashrc` and verify modification using print commands.
- **Code/Circuit Reference:**

```
# Append to ~/.bashrc
export TOOLS_PATH="$HOME/embedded_tools"
export PATH="$PATH:$TOOLS_PATH"

# Custom utility alias
alias check-usb="lsusb | grep -iE 'stlink|ftdi|ch340|cp210x'"
alias print-env="echo 'PATH=$PATH && arm-none-eabi-gcc --version 2>/dev/null || echo 'No cross-compiler yet'"
```

Wednesday: Git Foundations & Advanced Log Mechanics

- **Conceptual Objective:** Understand Git internals: the three-stage architecture (working directory, staging area, local repository), commit graph DAGs (Directed Acyclic Graphs), and the differences between merges, rebases, and fast-forwards.
- **Practical Activity:**
 - Initialize a new Git repository cleanly.
 - Configure global user identity parameters.
 - Master directory and file exclusion by drafting a comprehensive `.gitignore` targeting system binary objects, build outputs, and editor swap files.
- **Code/Circuit Reference:**

```
# Core contents of a standard embedded C .gitignore
*.o
*.elf
*.bin
*.hex
*.map
*.su
.vscode/
.idea/
*~
*.swp
build/
```

Thursday: Conventional Commits Syntax & Atomic Commits

- **Conceptual Objective:** Master the Conventional Commits v1.0.0 specification. Understand the structure: `type(scope): description`, where type is `feat`, `fix`, `docs`, `style`, `refactor`, `test`, or `chore`. Comprehend the importance of atomic commits (one logical change per commit).
- **Practical Activity:**
 - Practice staging and committing incremental changes.
 - Analyze repository history using custom graph formatting commands.
- **Code/Circuit Reference:**

```
# Good Commit Messages
git commit -m "feat(env): add custom stlink detection aliases to bashrc"
git commit -m "docs(readme): document physical hardware requirements"

# Beautiful Git Log Graph Alias
git config --global alias.lg "log --color --graph --pretty=format:'%Cred%h%Creset -
%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --abbrev-commit"
```

Friday: Git Hooks & Scripted Repo Discipline

- **Conceptual Objective:** Learn about Git Hooks—shell scripts executed automatically before or after key Git lifecycle events. Focus specifically on the `commit-msg` hook to programmatically audit commit messages.
- **Practical Activity:**
 - Create a custom shell script inside `.git/hooks/commit-msg`.
 - Grant the script executable permissions.

- Test the hook by trying to commit non-conforming messages.
- **Code/Circuit Reference:**

```
# Setup the commit-msg hook
cat << 'EOF' > .git/hooks/commit-msg
#!/usr/bin/env bash
commit_msg_file=$1
commit_msg=$(cat "$commit_msg_file")

# Regular expression to match conventional commits
pattern="^(feat|fix|docs|style|refactor|test|chore)(\([a-zA-Z0-9_-]+\))?: .+"

if [[ ! $commit_msg =~ $pattern ]]; then
    echo "ERROR: Invalid commit message format: '$commit_msg'"
    echo "Commit message must follow Conventional Commits specification."
    echo "Example: feat(gpio): configure pin PA5 as output"
    exit 1
fi
EOF
chmod +x .git/hooks/commit-msg
```

Hands-On Assignment: Custom Validation Hook

Infrastructure

Functional Requirements

Create a fully operational C project directory structure with a customized pre-commit validation shell script. 1. The project must have a standard directory structure: `src/`, `include/`, `build/`. 2. A bash script named `validate.sh` must be written in the root directory. It must automatically scan all `.c` and `.h` files under `src/` and `include/` and fail if any file contains: - Trailing whitespaces. - Banned standard library allocations (`malloc`, `free`, `realloc`). 3. You must install a `pre-commit` Git hook that automatically runs `./validate.sh`. If `validate.sh` returns a failure exit code, the commit must be blocked.

Structural Requirements & Code Skeleton

```
project-repo/
├── .git/
├── .gitignore
├── validate.sh
├── src/
│   └── main.c
├── include/
│   └── main.h
└── .git/hooks/
    └── pre-commit → (linked to or copying execution of validate.sh)
```

`validate.sh` **Shell Skeleton:**


```
#!/usr/bin/env bash
# Script to validate code rules before commit

errors=0

echo "Running pre-commit validation... "

# Check for trailing whitespaces
trailing_space_files=$(grep -rl '[:space:]$' src/ include/ 2>/dev/null)
if [ ! -z "$trailing_space_files" ]; then
    echo "FAIL: Trailing whitespaces found in:"
    echo "$trailing_space_files"
    errors=$((errors+1))
fi

# Check for forbidden dynamic allocations
forbidden_calls=$(grep -rE 'malloc\(|free\(|realloc\(' src/ include/ 2>/dev/null)
if [ ! -z "$forbidden_calls" ]; then
    echo "FAIL: Forbidden dynamic allocation functions used:"
    echo "$forbidden_calls"
    errors=$((errors+1))
fi

if [ $errors -ne 0 ]; then
    echo "Validation FAILED. Blocked commit."
    exit 1
fi

echo "Validation PASSED. Committing... "
exit 0
```

Deliverables

- [] Git repository initialized with `.gitignore` and conventional log alias configured.
 - [] Physical `.git/hooks/commit-msg` active and blocking non-conforming messages.
 - [] Project directory setup containing `validate.sh` and linked `.git/hooks/pre-commit` blocking trailing spaces and standard library allocations.
 - [] Active system logs or output recordings demonstrating the hooks in action (blocking bad commits, accepting conforming ones).
-

? Self-Check Questions

1. Why is the Filesystem Hierarchy Standard (FHS) critical to understand when dealing with cross-compilation toolchains? Where should a custom compiler toolchain normally be placed?
 2. What is the difference between a Fast-Forward Merge and a Three-Way Merge in Git? How does rebasing change history?
 3. How does a shell environment execute script commands (explain `PATH` resolution dynamics)?
 4. Under what circumstances will Git refuse to execute a hook script?
 5. Why are functions like `malloc()` and `free()` considered forbidden abstractions in high-reliability embedded system firmwares?
-

Stretch Task (Optional)

Extend `validate.sh` to automatically run `cppcheck --enable=all --error-exitcode=1 src/` inside the pre-commit hook. The commit must be cleanly blocked if the static analyzer catches any uninitialized variables, array bounds violations, or styling warnings.

Motivation Checkpoint

In production-level software organizations (such as SpaceX, Tesla, or Apple), thousands of commits are pushed daily. If developer code style, repository discipline, and commit hygiene are unmonitored, history quickly becomes incomprehensible, and bugs slip through compilation boundaries. Mastering automated scripting validation and Git hooks makes you an asset to team architectures on Day 1.

Trainer Review Rule

• Pass Criteria:

- `git log` shows conforming Conventional Commits.
- Attempting to commit a message like `"fixed bug"` is rejected with the custom regex error.
- Attempting to commit code containing `malloc()` is successfully blocked.
- Project directory is warning-free under compiler checks.

• Fail Criteria:

- Any compilation warning or bypass of validation routines.
- Absence of conventional commits in history.

Weekly Execution Plan: Week 02

Weekly Goal

Master low-level data representation Formats (Binary, Hexadecimal, Octal), understand logic gate electrical behaviors, Boolean algebra theorems, and build low-level software simulations of digital hardware logic blocks.

Weekly Outcome

By Friday, the student will be able to perform flawless hex/bin conversions mentally, apply Boolean theorems to simplify hardware circuitry, and write a complete, modular C library simulating logical gates (AND, OR, XOR, NOT, NAND, NOR) that outputs truth tables and simulates 4-bit adders using raw bitwise manipulations.



Session Breakdown (Monday - Friday)

Monday: Radix Formats, Hex, Binary, and Octal conversions

- **Conceptual Objective:** Understand base systems (binary - base 2, octal - base 8, decimal - base 10, hexadecimal - base 16). Comprehend binary scaling, signed formats (One's complement, Two's complement), and how fixed-width integers sit in core registers.
- **Practical Activity:**
 - Complete a mental math matrix converting numbers between binary, decimal, and hexadecimal.
 - Map out how negative integers are represented in two's complement and verify overflow limits on a 8-bit, 16-bit, and 32-bit boundary.
- **Code/Circuit Reference:**

```
Value: 187 (Decimal)
Hexadecimal: 0xBB (11*16^1 + 11*16^0 = 176 + 11 = 187)
Binary: 0b10111011 (128 + 32 + 16 + 8 + 2 + 1 = 187)
Octal: 0273 (2*64 + 7*8 + 3 = 128 + 56 + 3 = 187)
```

Tuesday: Boolean Algebra, Laws, and Logic Simplification

- **Conceptual Objective:** Learn logic gate logic rules (AND, OR, NOT, NAND, NOR, XOR, XNOR). Master Boolean theorems: Commutative, Associative, Distributive, Identity, and De Morgan's Laws

$(\overline{A \cdot B} = \overline{A} + \overline{B})$ and $\overline{A + B} = \overline{A} \cdot \overline{B}$.

• **Practical Activity:**

- Draw truth tables for all 7 fundamental logic gates.
- Prove De Morgan's Law algebraically and simplify a complex multi-gate circuit down to minimal gates.

• **Code/Circuit Reference:**

Boolean Circuit Simplification:

$Y = A \cdot B \cdot C + A \cdot B \cdot C' + A' \cdot B$

Step 1: Factor out $A \cdot B \rightarrow Y = A \cdot B \cdot (C + C') + A' \cdot B$

Step 2: $C + C' = 1 \rightarrow Y = A \cdot B \cdot (1) + A' \cdot B$

Step 3: Factor out $B \rightarrow Y = B \cdot (A + A')$

Step 4: $A + A' = 1 \rightarrow Y = B \cdot (1)$

Simplified Result: $Y = B$

Wednesday: Logical Gates Implementations in C

- **Conceptual Objective:** Understand the mapping between logical gates and C bitwise programming operators (`&` for AND, `|` for OR, `^` for XOR, `~` for NOT). Avoid confusing these with logical evaluation operators (`&&`, `||`, `!`).

• **Practical Activity:**

- Write individual, clean functions in C that simulate AND, OR, XOR, NOT, NAND, NOR, and XNOR.
- The function parameters and return types must be strict booleans or single bits.

• **Code/Circuit Reference:**

```
#include <stdbool.h>

// Core NAND gate simulation
bool gate_nand(bool input_a, bool input_b) {
    return !(input_a && input_b);
}

// Core XOR gate using basic NAND gates only
bool gate_xor(bool a, bool b) {
    bool n1 = gate_nand(a, b);
    bool n2 = gate_nand(a, n1);
    bool n3 = gate_nand(b, n1);
    return gate_nand(n2, n3);
}
```

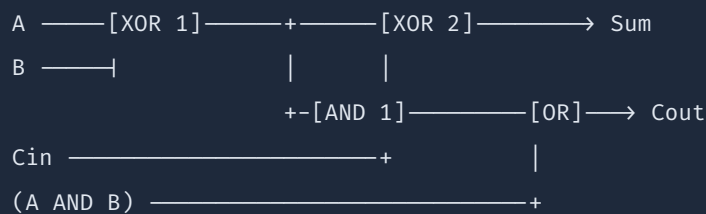
Thursday: Combinational Logic (Half-Adders & Full-Adders)

- **Conceptual Objective:** Understand combinational logic architectures. Study the construction of a half-adder (computes sum and carry-out of two bits) and a full-adder (computes sum and carry-out of two bits plus a carry-in).
- **Practical Activity:**
 - Draw the block diagram for a Full-Adder using XOR, AND, and OR gates.
 - Implement a full-adder function in C by nesting your gate functions from Wednesday.
- **Code/Circuit Reference:**

Full-Adder Logical Diagram:

Inputs: A, B, Cin

Outputs: Sum, Cout



Friday: Sequential Circuits Basics (Latches & Flip-Flops)

- **Conceptual Objective:** Learn the core transition from combinational to sequential logic (state-holding). Study the SR Latch, gated D Latch, and edge-triggered D Flip-Flop. Understand how clocks control propagation.
- **Practical Activity:**
 - Trace an active-low SR NAND latch truth table and identify the "forbidden" state.
 - Write a state-based loop in C simulating clock cycles and latch state updates.
- **Code/Circuit Reference:**

```
typedef struct {
    bool Q;
    bool Q_bar;
} sr_latch_t;

// Update latch outputs based on inputs
void update_sr_latch(bool S, bool R, sr_latch_t *latch) {
    if (!S && R) {
        latch->Q = true;
        latch->Q_bar = false;
    } else if (S && !R) {
        latch->Q = false;
        latch->Q_bar = true;
    } else if (!S && !R) {
        // Forbidden state for Active-Low Latch
        latch->Q = true;
        latch->Q_bar = true;
    }
    // S=1, R=1: Hold state (no change)
}
```

Hands-On Assignment: 4-Bit Binary Full-Adder Emulator

Functional Requirements

Write a production-grade C library that models hardware logic gates and assembles a complete 4-bit Binary Full-Adder. 1. The library must define distinct structures for `gate` behaviors. No standard logical operators `&&`, `||`, or `!` may be used inside the adder implementation—it must construct logic exclusively by calling your individual gate functions. 2. The user must be able to input two 4-bit integers (e.g., \$10\$ and \$7\$) represented in arrays of bits. 3. The adder must output the binary array showing sum bits and carry-out, as well as print the truth table demonstrating correct calculation. 4. Verify edge cases: maximum addition (\$15 + 15 = 30\$) showing correct carry-over.

Structural Requirements & Code Skeleton

`logic_gates.h`:

```

#ifndef LOGIC_GATES_H
#define LOGIC_GATES_H

#include <stdbool.h>

// Fundamental gates
bool gate_and(bool a, bool b);
bool gate_or(bool a, bool b);
bool gate_not(bool a);

// Compound gates built on fundamentals
bool gate_nand(bool a, bool b);
bool gate_nor(bool a, bool b);
bool gate_xor(bool a, bool b);

// Adders
typedef struct {
    bool sum;
    bool carry_out;
} adder_result_t;

adder_result_t half_adder(bool a, bool b);
adder_result_t full_adder(bool a, bool b, bool carry_in);

#endif // LOGIC_GATES_H

```

adder_4bit.c (C-Skeleton):

```
#include "logic_gates.h"
#include <stdio.h>

void adder_4bit(const bool a[4], const bool b[4], bool sum[4], bool *carry_out) {
    adder_result_t bit0 = full_adder(a[0], b[0], false);
    sum[0] = bit0.sum;

    adder_result_t bit1 = full_adder(a[1], b[1], bit0.carry_out);
    sum[1] = bit1.sum;

    adder_result_t bit2 = full_adder(a[2], b[2], bit1.carry_out);
    sum[2] = bit2.sum;

    adder_result_t bit3 = full_adder(a[3], b[3], bit2.carry_out);
    sum[3] = bit3.sum;

    *carry_out = bit3.carry_out;
}
```



Deliverables

- [] Complete `logic_gates.c` and `logic_gates.h` source files.
- [] Test program `main.c` that validates 4-bit addition over all inputs $0 \dots 15$.
- [] Clean Makefile to compile with `-Wall -Wextra -Werror` flags.
- [] Truth table output logs in file `adder_validation.log`.

? Self-Check Questions

1. Why is the number format **Two's Complement** universally preferred inside CPU arithmetic logic units (ALUs) over **One's Complement** or **Signed-Magnitude** formats?
2. What are the logical gate formulations for De Morgan's theorems? State them mathematically and draw the gate mappings.
3. Show how a standard 2-to-1 Multiplexer (MUX) can be constructed strictly using NAND gates.
4. What is the cause of **propagation delay** in physical hardware logic gates? Why must a software logic simulator account for simulated time blocks if doing cycle-accurate modeling?
5. Write the hexadecimal representation for the 32-bit signed integer `-42` in Two's Complement.

Stretch Task (Optional)

Extend the 4-bit emulator to model propagation delay. Assign a default delay (e.g., AND/OR = 2ns, NOT = 1ns, XOR = 3ns). Modify gate functions to compute and track cumulative signal propagation times. Output the exact maximum timing delay observed for a full 4-bit calculation transition (e.g., from `0111 + 0001` to `1000`).

Motivation Checkpoint

Inside any modern processor core, millions of transistors form logic gates, which aggregate to construct ALUs, vector pipelines, and bus routers. When you write a simple C instruction `x = a + b;`, it compiled directly to CPU assembly commands that physically route electron streams through this exact sequence of adders and carry lines. Understanding logic gating removes the mystery of computer math.

Trainer Review Rule

• Pass Criteria:

- Code compiles without any warnings.
- Zero standard logical operators (`&`, `||`, `!`) appear inside `adder_4bit.c` (must call gate functions).
- Adder produces correct sums and carry-overs for all 256 test cases of 4-bit inputs.

• Fail Criteria:

- Magic values used for binary manipulation.
- Direct logical operators inside adder algorithms instead of functional gate abstraction layers.

Weekly Execution Plan: Week 03

Weekly Goal

Master the C compilation pipeline, understand variable properties (scope, lifetime, linkage), utilize storage class specifiers (`static` , `extern` , `volatile` , `const`), and grasp the low-level behavior of compiled symbols in standard object files.

Weekly Outcome

By Friday, the student will be able to dissect a complex multi-file C project, map out exactly how code translates from preprocessing through linking, identify symbol scopes (`static` vs global symbols), and write program blocks that control variable lifetimes across files without leaking scopes.



Session Breakdown (Monday – Friday)

Monday: The C Preprocessor (Translation Units & Macros)

- **Conceptual Objective:** Understand what happens inside the preprocessor. Study header file inclusions (`#include`), conditional compilation (`#ifdef` , `#ifndef` , `#else` , `#endif`), header guards to prevent multiple definition errors, and macro definitions (object-like and function-like macros).
- **Practical Activity:**
 - Write a C file with multiple nested headers and inspect the preprocessed output file (`.i`) generated by compiling with `-E` .
 - Implement robust macro guards and note the elimination of duplicate declarations.
- **Code/Circuit Reference:**

```
# Compile to preprocess stage only
gcc -E src/main.c -o build/main.i
```

```
# Check line directives and header expanded code inside the .i file
cat build/main.i | head -n 50
```

Tuesday: Compilation & Assembly Phases

- **Conceptual Objective:** Study the compilation pipeline: translation from C to assembly representation (`.s`), optimization parameters (`-O0` , `-O1` , `-O2` , `-O3` , `-Os`), and translation of assembly to machine instructions inside binary object files (`.o`).
- **Practical Activity:**
 - Compile a simple loop function under different optimization levels (`-O0` vs `-O2`).
 - Inspect the resulting assembly code (`.s`) and compare how register usage and loop structures are optimized by the compiler.
- **Code/Circuit Reference:**

```
# Generate assembly file from C
gcc -S -O2 src/math.c -o build/math.s

# Trace assembly instructions
cat build/math.s
```

Wednesday: Variables Scopes, Lifetimes, and Linkage

- **Conceptual Objective:** Master variable scopes (block/local, file/global) and lifetimes (automatic/stack, static/data, dynamic/heap). Understand Linkage: internal linkage (accessible only within one translation unit) vs external linkage (accessible across files).
- **Practical Activity:**
 - Write a program containing global variables, local variables, and static block variables.
 - Trace variable addresses to observe how they map to different locations in virtual memory.
- **Code/Circuit Reference:**

```
int global_var = 10;           // External linkage, static lifetime
static int file_var = 20;      // Internal linkage, static lifetime

void function(void) {
    int local_var = 30;        // No linkage, automatic stack lifetime
    static int stat_var = 40;  // No linkage, static lifetime
}
```

Thursday: The Power of keywords - Volatile and Const

- **Conceptual Objective:** Understand the `volatile` qualifier (tells compiler never to optimize away reads/writes because value can change outside program flow, i.e., memory-mapped hardware registers). Study `const` (defines read-only values). Learn combination behaviors: `volatile const` pointers.

- **Practical Activity:**

- Write a polling delay loop. Compile with `-O3` without `volatile`. Observe the compiler stripping away the loop in assembly.
- Declare the loop counter variable as `volatile` and observe the loop preserved.

- **Code/Circuit Reference:**

```
// Polling a hardware status register safely
#define UART_STATUS_REG ((volatile uint32_t*)0x40011000)

void wait_for_data(void) {
    // Without volatile, the compiler loads the register value once
    // and loops infinitely on that cached CPU register!
    while ((*UART_STATUS_REG & 0x01) == 0) {
        // Wait ...
    }
}
```

Friday: Static and Extern in Multi-File Architectures

- **Conceptual Objective:** Grasp the dual role of `static` (restricts symbol visibility to the current file when applied globally; preserves variable value across function calls when applied locally). Understand `extern` (declares existence of symbol defined in another file).

- **Practical Activity:**

- Build a C modular layout where internal functions are protected using `static` declarations.
- Expose specific API control structures to `main.c` via headers using `extern` correctly.

- **Code/Circuit Reference:**

```
// sensor.c
static int raw_sensor_reading; // Private global variable (internal linkage)

int get_sensor_value(void) { // Public function (external linkage)
    return raw_sensor_reading;
}
```

Hands-On Assignment: Modular State Machine Engine

Functional Requirements

Create a multi-file C project that implements a state machine utilizing modular files, `static` encapsulation, `extern` interfaces, and strict build optimizations. 1. The project must have three files: `main.c`, `state_machine.c`, and `state_machine.h`. 2. `state_machine.c` must encapsulate the current state using a private file-scope static variable. The variable must not be accessible from `main.c`. 3. Provide a public API in `state_machine.h` to update and read the state. 4. Integrate a simulated hardware interrupt tick. The interrupt tick must modify a `volatile` global state change flag. 5. Provide a Makefile that compiles all files with `-Wall -Wextra -Werror -O2`, generates the intermediate preprocessor (`.i`), assembly (`.s`), and object files (`.o`) in a `build/` directory, and links them into a final executable.

Structural Requirements & Code Skeleton

`state_machine.h`:

```
#ifndef STATE_MACHINE_H
#define STATE_MACHINE_H

typedef enum {
    STATE_IDLE,
    STATE_RUNNING,
    STATE_ERROR
} system_state_t;

// Expose access APIs
void state_init(void);
void state_update(void);
system_state_t state_get(void);

// Expose hardware flag via extern (safe usage declaration)
extern volatile int g_hw_interrupt_flag;

#endif // STATE_MACHINE_H
```

`state_machine.c` (C-Skeleton):

```

#include "state_machine.h"

// Private file-scoped variables
static system_state_t current_state = STATE_IDLE;
static int transition_counter = 0; // Tracks transitions privately

// Global volatile flag defined here
volatile int g_hw_interrupt_flag = 0;

void state_init(void) {
    current_state = STATE_IDLE;
    transition_counter = 0;
}

void state_update(void) {
    if (g_hw_interrupt_flag) {
        g_hw_interrupt_flag = 0; // Clear flag

        switch (current_state) {
            case STATE_IDLE:
                current_state = STATE_RUNNING;
                transition_counter++;
                break;
            case STATE_RUNNING:
                current_state = STATE_IDLE;
                transition_counter++;
                break;
            case STATE_ERROR:
                // Require external system reset
                break;
        }
    }
}

system_state_t state_get(void) {
    return current_state;
}

```

Deliverables

- [] Multi-file workspace (`main.c` , `state_machine.c` , `state_machine.h`).
- [] Makefile producing distinct intermediate outputs: `.i` , `.s` , `.o` , and `.elf` .
- [] Output assembly analysis report `assembly_analysis.txt` comparing the compiled delay loops with and without `volatile` .

? Self-Check Questions

1. Why does using `static` on a global variable help prevent namespace collision in large projects?
2. If you declare a variable as `const volatile int timer_reg;` , what does that mean to the compiler? Can a variable be both `const` and `volatile` at the same time?
3. Explain step-by-step how the linker matches external declarations (`extern`) with their matching definitions. What happens if the definition is missing?
4. What is the fundamental difference in register execution behavior when compiling with optimization level `-O0` versus `-O3` ?
5. What does the `#error` preprocessor directive do, and how can it be used to check configuration headers?

Stretch Task (Optional)

Use `nm` and `objdump` utilities on your compiled object files (`state_machine.o`). Locate the symbols for `current_state` , `transition_counter` , `g_hw_interrupt_flag` , and `state_update` . Map each symbol to its corresponding object file section (`.text` , `.data` , `.bss`). Document your findings in the analysis report.

Motivation Checkpoint

A major difference between standard application C coding and high-performance embedded systems programming is the precise control over where data is stored in memory and how the compiler behaves. In microcontrollers, RAM size can be as small as 8KB. Knowing how to restrict symbol visibilities, limit global clutter, and secure register actions via `volatile` is the baseline expectation of systems developers.

Trainer Review Rule

- **Pass Criteria:**

- All compilation artifacts generated successfully in a separate `build/` folder.
- Zero compilation warnings are generated with `-O2`.
- Symbol visibility is strictly enforced: compiling a direct read of `current_state` inside `main.c` must trigger a compilation error (internal linkage checked).
- **Fail Criteria:**
 - Use of `extern` references to read static file-scoped variables.
 - Absence of intermediate compilation deliverables (`.i` , `.s` , `.o`).

Weekly Execution Plan: Week 04

Weekly Goal

Master pointers in C, understand the memory layout of running programs, and write custom dynamic memory allocation strategies that control static arrays without standard library functions.

Weekly Outcome

By Friday, the student will have designed and implemented a custom **Dynamic Block Memory Manager** in C. They will understand pointer arithmetic, double-pointers, function pointers, stack frame dynamics, heap configurations, and how to detect/avoid memory leaks and alignment corruptions.

Session Breakdown (Monday - Friday)

Monday: Deep-Dive Pointer Arithmetic & Pointer to Pointer

- **Conceptual Objective:** Master the core physics of pointers. Understand pointer scales (adding \$1\$ to an integer pointer advances address by 4 bytes). Learn double-pointers (`char **argv`), dereferencing pipelines, and void pointers (`void*`) for generic data routing.
- **Practical Activity:**
 - Write a program using pointer arithmetic to manually parse through multidimensional arrays and structures.
 - Implement dereferences to print data layouts byte-by-byte.
- **Code/Circuit Reference:**

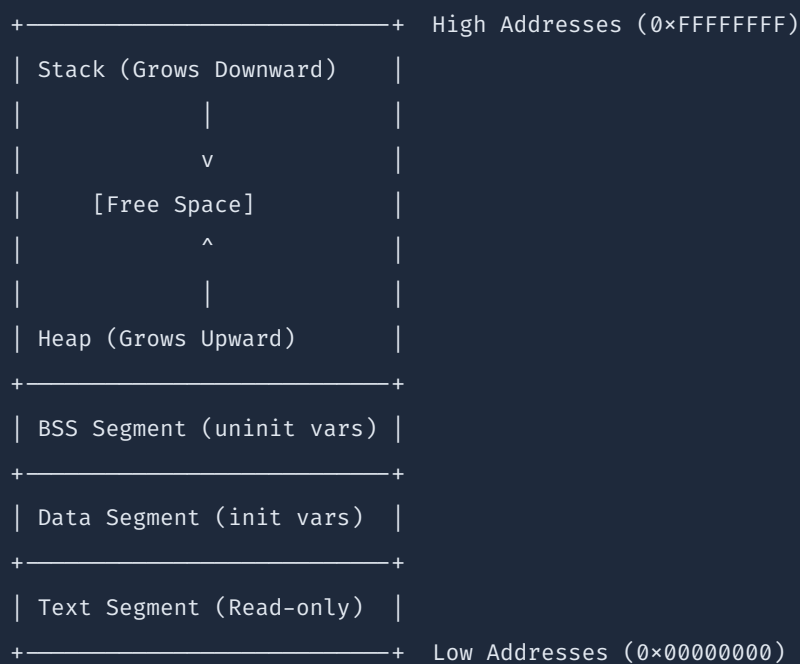
```
int array[5] = {10, 20, 30, 40, 50};
int *ptr = array;

// Pointer arithmetic advancements
printf("Address of index 0: %p\n", (void*)ptr);
printf("Address of index 1: %p\n", (void*)(ptr + 1)); // Advance by sizeof(int) = 4
bytes
printf("Value at index 2: %d\n", *(ptr + 2));          // Equivalent to array[2]
```

Tuesday: The C Application Memory Map

- **Conceptual Objective:** Study the memory model of an executing binary:
- **Text Segment:** Machine code (read-only).
- **Data Segment:** Initialized global and static variables.
- **BSS Segment:** Uninitialized global and static variables (cleared to 0 at boot).
- **Stack:** Local variables, return addresses, and stack frames.
- **Heap:** Dynamic memory.
- **Practical Activity:**
 - Write a program with variables located in every segment.
 - Print the memory address of each variable and draw a corresponding memory map diagram showing stack and heap directions.
- **Code/Circuit Reference:**

C Virtual Memory Layout:



Wednesday: Function Pointers & Callback Architectures

- **Conceptual Objective:** Understand that functions reside in memory inside the Text Segment and possess corresponding start addresses. Master function pointer declaration syntax, argument assignments, and callback functions for event-driven systems.
- **Practical Activity:**
 - Declare and initialize function pointers matching specific math parameters.
 - Build a callback-driven sensor polling loop that triggers different callbacks based on value thresholds.

• **Code/Circuit Reference:**

```
// Function pointer declaration: returns void, takes int
void (*event_handler)(int event_id);

void on_temp_alert(int temp) {
    printf("ALERT: Temperature exceeded limit! Current: %d\n", temp);
}

void check_sensor(int value, void (*callback)(int)) {
    if (value > 100) {
        callback(value); // Execute callback
    }
}
```

Thursday: Dynamic Memory Allocation Mechanics

- **Conceptual Objective:** Understand how the standard allocator manages memory. Learn about allocation requests (`malloc` , `calloc`), block resizing (`realloc`), and memory recycling (`free`). Comprehend fragmentation, memory leaks, and double-free exceptions.
- **Practical Activity:**
 - Write a program that intentionally leaks memory and track it with Valgrind.
 - Observe stack trace errors generated by double-frees or invalid memory accesses.
- **Code/Circuit Reference:**

```
# Check memory leak using Valgrind
gcc -g -O0 src/leak_demo.c -o build/leak_demo
valgrind --leak-check=full --show-leak-kinds=all ./build/leak_demo
```

Friday: Custom Block Allocators Mechanics (Singly Linked Heap)

- **Conceptual Objective:** Conceptualize how an allocator operates without operating system system-calls (like `sbrk`). Learn how to partition a static array of raw bytes into a linked list of managed heap blocks.
- **Practical Activity:**
 - Design a block metadata header structure containing the size of the block and a boolean indicating whether it is free or allocated.
 - Plan the link mechanism for traversing adjacent blocks.
- **Code/Circuit Reference:**

```
typedef struct block_header {
    size_t size;
    bool is_free;
    struct block_header *next;
} block_header_t;
```

Hands-On Assignment: Raw Static Array Memory Allocator

Functional Requirements

Create a custom, high-reliability memory allocation library (`my_alloc.c`, `my_alloc.h`) that partitions and manages a static memory pool array of 4KB (4096 bytes) using a raw block linked list. 1. The library must implement two core functions: `void* my_malloc(size_t size)` and `void my_free(void *ptr)`. 2. Do not use standard library `<stdlib.h>` functions (`malloc`, `free`, `realloc`). 3. Every allocated block must be preceded by a `block_header_t` block metadata node. 4. `my_malloc` must use a **First-Fit** algorithm: search the linked block list for the first free block of sufficient size. If the block is significantly larger than requested, split the block into an allocated chunk and a remaining free chunk. 5. `my_free` must mark the block as free. Additionally, it must scan adjacent blocks and **coalesce (merge)** consecutive free blocks to prevent fragmentation. 6. The allocator must enforce memory alignment (align all allocated addresses to 4-byte boundaries).

Structural Requirements & Code Skeleton

`my_alloc.h`:

```
#ifndef MY_ALLOC_H
#define MY_ALLOC_H

#include <stddef.h>
#include <stdbool.h>

void my_alloc_init(void);
void* my_malloc(size_t size);
void my_free(void *ptr);
void my_alloc_dump(void); // Utility to print layout of heap memory

#endif // MY_ALLOC_H
```

`my_alloc.c` (C-Skeleton):

```

#include "my_alloc.h"
#include <stdio.h>

#define POOL_SIZE 4096
#define ALIGNMENT 4
#define ALIGN(size) (((size) + (ALIGNMENT - 1)) & ~(ALIGNMENT - 1))

typedef struct block_header {
    size_t size;
    bool is_free;
    struct block_header *next;
} block_header_t;

#define HEADER_SIZE sizeof(block_header_t)

// Memory pool array
static uint8_t memory_pool[POOL_SIZE];
static block_header_t *free_list_head = NULL;

void my_alloc_init(void) {
    free_list_head = (block_header_t*)memory_pool;
    free_list_head->size = POOL_SIZE - HEADER_SIZE;
    free_list_head->is_free = true;
    free_list_head->next = NULL;
}

void* my_malloc(size_t size) {
    size_t aligned_size = ALIGN(size);
    block_header_t *current = free_list_head;

    while (current != NULL) {
        if (current->is_free && current->size ≥ aligned_size) {
            // Can we split the block?
            if (current->size ≥ aligned_size + HEADER_SIZE + ALIGNMENT) {
                block_header_t *new_block = (block_header_t*)((uint8_t*)current +
HEADER_SIZE + aligned_size);
                new_block->size = current->size - aligned_size - HEADER_SIZE;
                new_block->is_free = true;
                new_block->next = current->next;

                current->size = aligned_size;
                current->next = new_block;
            }
        }
    }
}

```

```

        }
        current->is_free = false;
        return (void*)((uint8_t*)current + HEADER_SIZE);
    }
    current = current->next;
}
return NULL; // Out of memory
}

void my_free(void *ptr) {
    if (ptr == NULL) return;

    block_header_t *header = (block_header_t*)((uint8_t*)ptr - HEADER_SIZE);
    header->is_free = true;

    // Coalesce free blocks
    block_header_t *current = free_list_head;
    while (current != NULL && current->next != NULL) {
        if (current->is_free && current->next->is_free) {
            current->size += HEADER_SIZE + current->next->size;
            current->next = current->next->next;
        } else {
            current = current->next;
        }
    }
}
}

```

Deliverables

- [] Source files `my_alloc.c` and `my_alloc.h`.
 - [] Application test script `main.c` that validates allocating, splitting, freeing, and coalescing allocations under edge-case conditions.
 - [] Makefile compiling the execution binary.
-

Self-Check Questions

1. Why is memory alignment critical for high-performance processors? What happens physically at the memory bus if a 32-bit access is not aligned?

2. What are the key performance tradeoffs between **First-Fit**, **Best-Fit**, and **Worst-Fit** memory allocation algorithms?
 3. How does the coalesce mechanism resolve the **external fragmentation** issue inside heap structures?
 4. Explain the difference between a pointer declaration `const int *ptr` versus `int *const ptr`.
 5. What is a **memory leak**, and why is it particularly catastrophic in long-running embedded systems (such as a satellite flight controller)?
-

Stretch Task (Optional)

Extend the allocator library to be **thread-safe** by integrating dummy mutex hook functions, and implement a **Worst-Fit** configuration options selectable via a compiler macro definition (`-DALLOC_ALGO_WORST`).

Motivation Checkpoint

In high-reliability embedded system architectures (like automotive ISO-26262 systems), calling the standard C library `malloc()` or `free()` is banned. This is due to the non-deterministic execution times of typical allocators and the risk of memory fragmentation locking up the system. Knowing how to construct dynamic, deterministic block allocators or pool managers is a core requirement for mission-critical firmware developers.

Trainer Review Rule

• Pass Criteria:

- Allocator splits blocks correctly (dump shows residual free blocks created on allocation).
- Freeing two consecutive blocks successfully merges them into a single larger block.
- Aligned data offsets are strictly divisible by 4.
- Zero compiler warnings or warnings of memory leak inside test application.

• Fail Criteria:

- Direct call to standard `<stdlib.h>` `malloc` inside implementation.
- Misaligned memory offsets.

Weekly Execution Plan: Week 05

Weekly Goal

Master bitwise operations, understand how to utilize bitfields and unions for memory-efficient data structures, and learn the exact register manipulation methods required to configure microcontroller peripheral registers.

Weekly Outcome

By Friday, the student will have built a complete, register-accurate **Software Emulation of a GPIO Controller Peripheral**. They will possess the capability to perform complex bit masking, toggle specific register pins, unpack binary streams using unions and bitfields, and read/write raw memory locations safely.

Session Breakdown (Monday - Friday)

Monday: The Bitwise Toolkit (AND, OR, XOR, NOT, Shifts)

- **Conceptual Objective:** Master low-level bit operations: setting bits (`|`), clearing bits (`& ~`), toggling bits (`^`), checking states (`&`), and logical shifts (`<<` , `>>`). Understand the difference between logical shifts (pads with 0) and arithmetic shifts (preserves the sign bit).
- **Practical Activity:**
 - Complete a bit manipulation worksheet implementing quick operations.
 - Implement a set of macro definitions to perform these actions cleanly on generic integers.
- **Code/Circuit Reference:**

```
#define SET_BIT(reg, bit)    ((reg) |= (1U << (bit)))
#define CLEAR_BIT(reg, bit) ((reg) &= ~(1U << (bit)))
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1U << (bit)))
#define CHECK_BIT(reg, bit) (((reg) & (1U << (bit))) != 0)
```

Tuesday: Bit Masking and Multi-Bit Fields Manipulation

- **Conceptual Objective:** Learn how to modify fields consisting of multiple bits without affecting surrounding registers (e.g., setting a 2-bit pin mode field at bits 10-11). Comprehend the clear-then-set design pattern.

- **Practical Activity:**

- Write a program to isolate and modify a 4-bit subfield located in the middle of a 32-bit variable.
- Print binary states dynamically to verify surrounding bits remain unchanged.

- **Code/Circuit Reference:**

```
// Target: Set bits 4-5 of REG to 0b10 (value 2) without changing other bits
uint32_t reg = 0xABCD1234;
uint32_t mask = (3U << 4); // Mask covering bits 4 & 5

reg &= ~mask;           // Step 1: Clear the bits (REG = REG & ~(0x30))
reg |= (2U << 4);       // Step 2: Set the target value (REG = REG | 0x20)
```

Wednesday: Struct Bitfields (Syntax & Memory Layouts)

- **Conceptual Objective:** Master C Struct Bitfields—a mechanism to declare structures with explicitly sized members (in bits). Understand the compilation rules, compiler padding, byte boundaries, and endianness issues.

- **Practical Activity:**

- Declare struct bitfields representing network frames or hardware control packets.
- Measure struct sizes using `sizeof()` and observe how compilers pack bits into bytes.

- **Code/Circuit Reference:**

```
typedef struct {
    uint32_t pin_0 : 2; // Mode: 2 bits
    uint32_t pin_1 : 2; // Mode: 2 bits
    uint32_t pin_2 : 2; // Mode: 2 bits
    uint32_t reserved : 26; // Reserved padding bits
} gpio_mode_register_t;
```

Thursday: Unions and Bitfields (The Dual Representation Pattern)

- **Conceptual Objective:** Study Unions—structures where all members share the same memory location. Master the Dual Representation Pattern: combining struct bitfields with a raw `uint32_t` in a union to allow access to individual bit fields AND the entire register raw value.

- **Practical Activity:**

- Create a union mapping a register's bit fields to a raw data value.
- Modify individual bits in the struct, and print the raw hex representation of the union.

- **Code/Circuit Reference:**

```
typedef union {
    struct {
        uint32_t speed : 2; // Bits 0-1
        uint32_t parity : 1; // Bit 2
        uint32_t stop : 1; // Bit 3
        uint32_t tx_en : 1; // Bit 4
        uint32_t rx_en : 1; // Bit 5
        uint32_t rsvd : 26; // Padding
    } bits;
    uint32_t raw; // Direct register-wide access
} uart_control_reg_t;
```

Friday: Accessing Memory-Mapped Hardware Registers

- **Conceptual Objective:** Learn how memory-mapped I/O (MMIO) functions. Understand that microcontrollers map peripheral control registers to specific hardware addresses. Learn how to cast raw address locations to pointers of volatile unions/structs.
- **Practical Activity:**
 - Define fake register addresses matching dummy buffers.
 - Construct a C module containing pointer macros to interact with those fake addresses.
- **Code/Circuit Reference:**

```
// Simulating an address layout in memory
static uint32_t simulated_io_space[10];

#define SIM_GPIO_BASE_ADDR ((uintptr_t)simulated_io_space)
#define SIM_GPIO_MODER ((volatile uint32_t*)(SIM_GPIO_BASE_ADDR + 0x00))
#define SIM_GPIO_ODR ((volatile uint32_t*)(SIM_GPIO_BASE_ADDR + 0x04))

void toggle_pin5(void) {
    *SIM_GPIO_ODR ^= (1U << 5); // Read-Modify-Write directly on the simulated port
}
```

Hands-On Assignment: GPIO Register Controller Emulator

Functional Requirements

Create a register-accurate Software Emulator of an ARM Cortex-M GPIO peripheral block. 1. Define a `gpio_port_t` structure that mirrors the physical register layout of STM32F4 GPIO ports (MODER,

OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR). 2. The registers must be defined using a clean combination of **unions and struct bitfields** to allow both raw register-wide access and bit-field member access. 3. MODER must allocate 2 bits per pin for 16 pins: - `00` = Input, `01` = General Purpose Output, `10` = Alternate Function, `11` = Analog. 4. Write modular, register-level API functions to: - `void gpio_set_mode(gpio_port_t *port, uint8_t pin, uint8_t mode)` - `void gpio_write_pin(gpio_port_t *port, uint8_t pin, bool state)` - `void gpio_toggle_pin(gpio_port_t *port, uint8_t pin)` - `bool gpio_read_pin(const gpio_port_t *port, uint8_t pin)` 5. Implement the Bit Set/Reset Register (BSRR) functionality. BSRR is a 32-bit write-only register: writing `1` to bits 0-15 sets the corresponding ODR bit; writing `1` to bits 16-31 resets the corresponding ODR bit. Writing `0` does nothing. Your emulation loop must monitor BSRR writes and update ODR accordingly.

Structural Requirements & Code Skeleton

`gpio_emulator.h`:

```

#ifndef GPIO_EMULATOR_H
#define GPIO_EMULATOR_H

#include <stdint.h>
#include <stdbool.h>

// Register structure representing GPIO port registers
typedef struct {
    union {
        struct {
            uint32_t pin0 : 2;
            uint32_t pin1 : 2;
            uint32_t pin2 : 2;
            uint32_t pin3 : 2;
            uint32_t pin4 : 2;
            uint32_t pin5 : 2;
            uint32_t pin6 : 2;
            uint32_t pin7 : 2;
            uint32_t pin8 : 2;
            uint32_t pin9 : 2;
            uint32_t pin10 : 2;
            uint32_t pin11 : 2;
            uint32_t pin12 : 2;
            uint32_t pin13 : 2;
            uint32_t pin14 : 2;
            uint32_t pin15 : 2;
        } pin;
        uint32_t raw;
    } MODER;

    uint32_t OTYPER;
    uint32_t OSPEEDR;
    uint32_t PUPDR;
    uint32_t IDR;
    uint32_t ODR;
    uint32_t BSRR;
} gpio_port_t;

// APIs
void gpio_init(gpio_port_t *port);
void gpio_update_bsrr_logic(gpio_port_t *port); // Processes BSRR writes to update ODR
void gpio_set_mode(gpio_port_t *port, uint8_t pin, uint8_t mode);

```

```
void gpio_write_pin(gpio_port_t *port, uint8_t pin, bool state);
void gpio_toggle_pin(gpio_port_t *port, uint8_t pin);
bool gpio_read_pin(const gpio_port_t *port, uint8_t pin);

#endif // GPIO_EMULATOR_H
```

Deliverables

- [] Hardware emulation source files `gpio_emulator.c` and `gpio_emulator.h`.
- [] Application test suite `main.c` validating register states, BSRR writes, MODER masks, and toggles.
- [] Makefile producing a warning-free compilation binary.

? Self-Check Questions

1. Why is a simple assignment like `REG = (1 << 5);` highly dangerous inside dynamic register setups? What is the correct way to modify single bits without disturbing others?
2. Explain how the BSRR register achieves atomic bit setting and resetting in microcontrollers without requiring Read-Modify-Write instructions. Why is this critical to prevent race conditions?
3. How does compiler optimization affect struct bitfields? Can you rely on the physical layout order of bitfields when porting code across different compilers or architectures?
4. What is the difference between Big-Endian and Little-Endian byte ordering? Show how the 32-bit hex value `0x12345678` is laid out in memory under both environments.
5. Why must variables representing hardware registers always be declared with the `volatile` keyword?

Stretch Task (Optional)

Extend the emulation to model physical input pins. Create a separate background execution thread that dynamically updates the Input Data Register (IDR) to simulate a physical button press bouncing (simulate states bouncing between 1 and 0 every few milliseconds before settling).

Motivation Checkpoint

At the physical silicon layer, microcontrollers have no concepts of "interfaces", "screens", or "sensors". The processor core sees the universe purely as arrays of memory addresses. Peripheral drivers are

written entirely by applying bit masks to target registers. Mastering register manipulations is the foundational key to unlocking microcontroller peripheral control.



Trainer Review Rule

- **Pass Criteria:**

- GPIO emulator accurately maps bitfields (setting `port→MODER.pin.pin5 = 1` changes raw value to correct index bitmask).
- Writing to BSRR correctly modifies ODR register bits (checked inside the BSRR logic monitor function).
- Code compiles without warnings with `-Wall -Wextra -Werror`.

- **Fail Criteria:**

- Using standard logic operators where bitwise is required.
- ODR or MODER configuration corrupted on adjacent pins during single pin writes.

Weekly Execution Plan: Week 06

Weekly Goal

Master structural layout controls, unions, memory padding, data structure alignment, and advanced preprocessor metaprogramming inside production C applications.

Weekly Outcome

By Friday, the student will have designed and implemented a **Serialized Frame Packetizer** in C. They will understand compiler structure packing directives, how to analyze alignment padding bytes manually, and how to write safe, macro-based code templates to automate repetitive logic.



Session Breakdown (Monday - Friday)

Monday: Structures, Typedefs, and Object models

- **Conceptual Objective:** Master C struct representations. Understand how data is laid out sequentially in memory. Comprehend structure initialization methods, typedef abstractions, and passing structs to functions (by value vs by reference).
- **Practical Activity:**
 - Write a program declaring complex nested structs.
 - Implement functional drivers passing pointers of these structures, demonstrating efficiency improvements over passing by value.
- **Code/Circuit Reference:**

```
typedef struct {
    uint8_t sensor_id;
    uint32_t timestamp;
    float reading;
} sensor_packet_t;

// Pass by reference (highly efficient, only transfers a 32/64-bit pointer address)
void process_packet(const sensor_packet_t *packet) {
    printf("Sensor ID: %u, Value: %.2f\n", packet->sensor_id, packet->reading);
}
```

Tuesday: Structure Padding, Alignment, and Packing

- **Conceptual Objective:** Understand that compilers automatically insert padding bytes into structures to align multi-byte data on native boundaries (e.g., a 32-bit float must align to an address divisible by 4). Study `pragma pack` and attribute packing directives.
- **Practical Activity:**
 - Declare the same structural components in different orders. Measure structure sizes using `sizeof()` and observe how ordering changes padding.
 - Apply `__attribute__((packed))` to strip padding and inspect the resulting layout byte-by-byte.
- **Code/Circuit Reference:**

```
// Size of this struct is 12 bytes due to padding:  
// [char (1 byte)] + [3 padding bytes] + [int (4 bytes)] + [short (2 bytes)] + [2  
padding bytes]  
typedef struct {  
    char a;  
    int b;  
    short c;  
} padded_struct_t;  
  
// Size of this struct is 7 bytes (zero padding):  
typedef struct __attribute__((packed)) {  
    char a;  
    int b;  
    short c;  
} packed_struct_t;
```

Wednesday: Advanced Preprocessor Metaprogramming (Macro Utilities)

- **Conceptual Objective:** Master preprocessor expansion mechanics. Study token concatenation (`##`), stringification (`#`), variadic macros (`__VA_ARGS__`), and how to implement compile-time assertion utilities.
- **Practical Activity:**
 - Write macro helpers to automatically generate structural access APIs.
 - Implement a `STATIC_ASSERT` preprocessor tool that fails compilation if a target structure size exceeds limits.
- **Code/Circuit Reference:**


```
// Compile-time assertion macro (fails compilation if condition is false)
#define STATIC_ASSERT(cond, msg) typedef char static_assertion_##msg[(cond) ? 1 : -1]

// Verify packed struct is exactly 7 bytes at compile time!
STATIC_ASSERT(sizeof(packed_struct_t) == 7, PackedStructSizeIncorrect);
```

Thursday: Unions for Multi-Type Message Parsing

- **Conceptual Objective:** Master union mechanics for multi-type communication messaging. Learn how to overlay a raw byte command payload with different command structures without copying memory.
- **Practical Activity:**
 - Declare command unions defining different packet layouts.
 - Receive a raw buffer, overlay the union, read command ID fields, and parse data members dynamically.
- **Code/Circuit Reference:**

```
typedef enum {
    CMD_LED_CTRL,
    CMD_MOTOR_SPEED
} cmd_id_t;

typedef struct {
    uint8_t pin;
    bool state;
} cmd_led_t;

typedef struct {
    uint16_t rpm;
} cmd_motor_t;

typedef union {
    cmd_led_t led;
    cmd_motor_t motor;
} cmd_payload_t;

typedef struct {
    cmd_id_t cmd_id;
    cmd_payload_t payload;
} command_packet_t;
```

Friday: Endianness and Structure Serializations

- **Conceptual Objective:** Master data serialization across hardware environments. Understand how big-endian and little-endian networks communicate. Learn how to safely serialize a struct to a raw byte array for transport, and unpack it on a receiving target.
- **Practical Activity:**
 - Write serialization and deserialization functions that convert structural fields into standardized, network-endian (big-endian) byte streams.
 - Verify byte order transitions on the host.
- **Code/Circuit Reference:**

```
// Safe serialization helper for a 16-bit integer (converts to Big-Endian network
format)
void serialize_uint16(uint8_t *buffer, uint16_t value) {
    buffer[0] = (uint8_t)((value >> 8) & 0xFF);
    buffer[1] = (uint8_t)(value & 0xFF);
}

uint16_t deserialize_uint16(const uint8_t *buffer) {
    return (uint16_t)((buffer[0] << 8) | buffer[1]);
}
```

Hands-On Assignment: Serialized Command Packetizer

Functional Requirements

Create a robust Serialized Command Packetizer that serializes and deserializes structured command frames for a simulated UART physical serial interface. 1. The project must define a packet frame structure containing: - Start Byte (`0xAA`) - 1 byte. - Command Type - 1 byte. - Data Length - 1 byte. - Data Payload - up to 8 bytes. - CRC-8 Checksum - 1 byte. 2. The packet structure must be explicitly **packed** to prevent any compiler padding bytes from corrupting the layout. 3. Write serialization APIs: `size_t serialize_packet(const packet_t *pkt, uint8_t *buffer)` and `bool deserialize_packet(const uint8_t *buffer, size_t len, packet_t *pkt)`. 4. The serialization must enforce Big-Endian byte ordering for any multi-byte fields (such as multi-byte payloads). 5. Implement a simple CRC-8 checksum calculation routine and verify packet integrity upon deserialization. 6. Fail deserialization if the start byte is incorrect, the payload length is out of range, or the checksum fails.

Structural Requirements & Code Skeleton

`packetizer.h`:

```

#ifndef PACKETIZER_H
#define PACKETIZER_H

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#define START_BYTE 0xAA
#define MAX_PAYLOAD_SIZE 8

typedef enum {
    CMD_PING = 0x01,
    CMD_SET_SPEED = 0x02,
    CMD_GET_TEMP = 0x03
} cmd_type_t;

// Packed structure definition
typedef struct __attribute__((packed)) {
    uint8_t start_byte;
    uint8_t cmd_type;
    uint8_t length;
    uint8_t payload[MAX_PAYLOAD_SIZE];
    uint8_t checksum;
} packet_t;

size_t serialize_packet(const packet_t *pkt, uint8_t *buffer);
bool deserialize_packet(const uint8_t *buffer, size_t len, packet_t *pkt);
uint8_t calculate_crc8(const uint8_t *data, size_t len);

#endif // PACKETIZER_H

```

Deliverables

- [] Serializer source files `packetizer.c` and `packetizer.h`.
 - [] Application test script `main.c` showing multiple serialization transactions, intentional payload corruptions (corrupted checksum), and verifying correct handling.
 - [] Makefile compiling with strict warnings and optimizations.
-

? Self-Check Questions

1. Why does the compiler add padding bytes to structures? What is the hardware penalty of unaligned memory access on modern processors?
 2. What is the fundamental difference between `#define` macros and `typedef` in C? Why is `typedef` safer for declaring pointer types?
 3. How does `#pragma pack(1)` differ from `__attribute__((packed))`? Which is safer for cross-compiler compatibility?
 4. Write a preprocessor macro `GET_ARRAY_SIZE(arr)` that automatically calculates the number of elements in an array at compile time.
 5. Why are float values particularly risky to serialize directly via `memcpy` between an ARM microcontroller and an Intel x86 server host?
-

Stretch Task (Optional)

Extend the packetizer to support variable-sized payloads using a custom preprocessor metaprogramming tool that automatically generates custom serialization code blocks based on command schemas.

Motivation Checkpoint

When serializing configurations, sensor frames, or actuator commands across networks (like CAN bus in cars, Modbus in factories, or UART serial connections in consumer electronics), data alignment must be exact down to the individual bit. If one node pads data by a single byte, the receiver parses corrupted values. Mastering serialization and packing is a key milestone for interface development.

Trainer Review Rule

• Pass Criteria:

- `sizeof(packet_t)` matches expected size exactly (checked via a custom preprocessor assert).
- Attempting to deserialize a packet with a mutated data checksum fails and returns `false`.
- Binary outputs show correct Big-Endian conversion for multi-byte payloads.

• Fail Criteria:

- Direct array element assignment without checking length limits.
- Zero static assertion checks for packed sizes.

Weekly Execution Plan: Week 07

Weekly Goal

Master the internal architecture of the ARM Cortex-M4 microcontroller core, understand the physical register set, the memory map, the CPU boot sequence, and the anatomy of the Vector Table.

Weekly Outcome

By Friday, the student will have written a custom, bare-metal Startup C/Assembly file and a bare-metal Linker Script from scratch. They will be able to explain how the hardware reads the Vector Table at boot, initializes the Stack Pointer (MSP), and branches to the `main()` entry point without relying on any IDE-generated wizard files.



Session Breakdown (Monday - Friday)

Monday: The ARM Cortex-M Core Architecture

- **Conceptual Objective:** Understand the Harvard Architecture (separate instruction and data buses), CPU operational modes (Thread Mode vs Handler Mode), privilege levels (Privileged vs Unprivileged), and the physical registers set (R0-R12 general purpose, R13 Stack Pointer MSP/PSP, R14 Link Register, R15 Program Counter).
- **Practical Activity:**
 - Read the ARMv7-M Architecture Reference Manual core registers chapter.
 - Draw the register layout block diagram highlighting the Special Registers (xPSR, PRIMASK, FAULTMASK, BASEPRI, CONTROL).
- **Code/Circuit Reference:**

Cortex-M Core Register File:

+-----+		
R0 - R12	: General Purpose Registers	
+-----+		
R13 (SP)	: Stack Pointer (MSP / PSP)	
+-----+		
R14 (LR)	: Link Register (Holds Return Address)	
+-----+		
R15 (PC)	: Program Counter (Holds Exec Address)	
+-----+		
xPSR	: Program Status Register (flags)	
+-----+		

Tuesday: Memory-Mapped I/O & The STM32 Memory Map

- **Conceptual Objective:** Study the 4GB addressable memory space of ARM 32-bit cores. Understand the physical partitioning: Flash memory (starts at `0x08000000` on STM32), SRAM (starts at `0x20000000`), Peripherals (starts at `0x40000000`), and the System Control Block (SCB) region.
- **Practical Activity:**
 - Open the STM32F407xx Datasheet and locate the boundary addresses of flash memory, SRAM1, SRAM2, and GPIOA peripheral.
 - Construct a C program defining pointer macros mapped directly to these base addresses.
- **Code/Circuit Reference:**

```
#define FLASH_BASE_ADDR    0x08000000U
#define SRAM_BASE_ADDR     0x20000000U
#define PERIPH_BASE_ADDR   0x40000000U

#define GPIOA_BASE_ADDR    (PERIPH_BASE_ADDR + 0x00200000U) // GPIOA mapped to
0x40020000
```

Wednesday: The Boot Sequence & Vector Table Mechanics

- **Conceptual Objective:** Comprehend the exact, hardware-level boot sequence of the processor. At power-on or reset, the core:
 - Reads memory address `0x00000000` (or booted alias like `0x08000000`) to fetch the Initial Stack Pointer value and loads it into MSP.
 - Reads address `0x00000004` to fetch the address of the Reset Handler and loads it into PC.
 - Begins executing the Reset Handler instruction.

- **Practical Activity:**

- Design a C structure representing the Vector Table containing Exception vectors (Reset, NMI, HardFault, SVC, PendSV, SysTick) and Peripheral interrupts.
- Mark the Vector Table to be placed at the very beginning of Flash memory.

- **Code/Circuit Reference:**

```
// Vector Table declaration using C function pointers
typedef void (*element_t)(void);

// Force placement in a specific linker section
__attribute__((section(".isr_vector")))
const element_t vector_table[] = {
    (element_t)0x20020000, // 1. Initial Stack Pointer (Top of SRAM, 128KB on F4)
    Reset_Handler,        // 2. Reset Handler (Address PC starts execution)
    NMI_Handler,           // 3. Non-Maskable Interrupt Handler
    HardFault_Handler,     // 4. Hard Fault Handler
};
```

Thursday: Designing a Bare-Metal Linker Script

- **Conceptual Objective:** Understand what a Linker Script (`.ld`) does: it defines the physical memory regions of the hardware target and specifies where the linker should place compiled code sections (`.text` , `.data` , `.bss` , `.rodata`).

- **Practical Activity:**

- Create a custom `linker_script.ld` specifying FLASH and SRAM sizes for the STM32F407VG (\$1\text{MB}\$ Flash, \$128\text{KB}\$ SRAM).
- Define section mappings to place `.isr_vector` strictly at the start of Flash.

- **Code/Circuit Reference:**

```

/* Minimal Linker Script: linker_script.ld */
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vector))
        *(.text)
        *(.text.*)
        *(.rodata)
        *(.rodata.*)
        _etext = .; /* Define end of text symbol */
    } > FLASH

    .data : AT(_etext)
    {
        _sdata = .;
        *(.data)
        *(.data.*)
        _edata = .;
    } > SRAM

    .bss :
    {
        _sbss = .;
        *(.bss)
        *(.bss.*)
        _ebss = .;
    } > SRAM
}

```

Friday: Writing the Reset Handler Startup File

- **Conceptual Objective:** Study what a Startup File must do before jumping to `main()`:

- Copy the initialized data section (`.data`) from Flash memory (where it is stored) to SRAM (where it runs).
- Clear the uninitialized data section (`.bss`) in SRAM to zero.
- Call the `main()` function.

- **Practical Activity:**

- Write a custom `startup.c` file containing the Reset Handler and all core exception vectors as weak aliases to a default dummy loop.
- Compile and link using `arm-none-eabi-gcc` and your linker script.

- **Code/Circuit Reference:**

```
// startup.c
#define SRAM_START 0x20000000U
#define SRAM_SIZE (128U * 1024U)
#define STACK_TOP (SRAM_START + SRAM_SIZE)

extern unsigned int _etext; // Linker symbols
extern unsigned int _sdata;
extern unsigned int _edata;
extern unsigned int _sbss;
extern unsigned int _ebss;

void main(void);

void Reset_Handler(void) {
    // 1. Copy .data section from Flash to SRAM
    unsigned int *src = &_etext;
    unsigned int *dst = &_sdata;
    while (dst < &_edata) {
        *dst++ = *src++;
    }

    // 2. Clear .bss section to 0
    dst = &_sbss;
    while (dst < &_ebss) {
        *dst++ = 0;
    }

    // 3. Call application main
    main();

    // 4. Infinite trap loop if main returns
    while(1);
}
```

Hands-On Assignment: Custom Startup & Linker Pipeline

Functional Requirements

Create a fully custom, bare-metal build environment that compiles a simple C application without any pre-packaged IDE files. 1. The project must consist of four files: `main.c`, `startup.c`,

`linker_script.ld` , and `Makefile` . 2. Do not link standard library startup routines (compile with `-nostdlib` flag). 3. The Vector Table in `startup.c` must define at least `Reset_Handler`, `NMI_Handler`, and `HardFault_Handler`. 4. The Reset Handler must manually perform `.data` copying and `.bss` clearing before jumping to `main()` . 5. `main.c` should implement a simple loop. 6. The Makefile must use the `arm-none-eabi-gcc` cross-compiler toolchain, generate a final `.elf` binary, compile with `-g` to embed debugging symbols, and generate a memory map file (`-Wl,-Map=build/output.map`) to verify symbol placement.

Structural Requirements & Code Skeleton

`main.c` :

```
int counter = 42; // Placed in .data (initialized global)
int uninit_var;   // Placed in .bss (cleared to 0)

void main(void) {
    uninit_var = 100;
    while (1) {
        counter++;
        if (counter > 1000) {
            counter = uninit_var;
        }
    }
}
```

`Makefile` :

```
CC = arm-none-eabi-gcc
LD = arm-none-eabi-ld
OBJCOPY = arm-none-eabi-objcopy

CFLAGS = -mcpu=cortex-m4 -mthumb -nostdlib -Wall -Wextra -O0 -g
LDFLAGS = -T linker_script.ld -Wl,-Map=build/output.map

all: build/output.elf build/output.bin

build/output.elf: main.c startup.c
    mkdir -p build
    $(CC) $(CFLAGS) $(LDFLAGS) startup.c main.c -o $@

build/output.bin: build/output.elf
    $(OBJCOPY) -O binary $< $@

clean:
    rm -rf build
```



Deliverables

- [] Custom linker script file `linker_script.ld`.
- [] Startup source file `startup.c` with Vector Table.
- [] Test program `main.c` and compilation Makefile.
- [] Extracted memory map file `build/output.map` verifying that `.isr_vector` is at `0x08000000`, `counter` is in SRAM, and `uninit_var` is in SRAM and initialized to 0.

? Self-Check Questions

1. Why does the processor read the initial Stack Pointer (MSP) before the Reset Vector address at boot? What is the consequence if these two values are corrupted in Flash?
2. What does the `-mthumb` compiler flag tell the ARM processor? What happens if you try to compile ARM Cortex-M code without it?
3. What is the difference between physical load addresses (LMA) and virtual execution addresses (VMA) of the `.data` section? How does the Linker Script coordinate this using `AT()`?
4. How do you declare a custom C function to be placed inside a specific linker-defined section instead of the default `.text` section?

5. What is the role of the Vector Table Offset Register (VTOR) inside the System Control Block (SCB)?
Why is VTOR critical when developing custom bootloaders?
-

Stretch Task (Optional)

Modify the custom Linker Script to define a custom stack limit symbol and write a simple `Stack_Overflow_Detector` check inside the startup file that sets a hardware register flag if the stack pointer grows past memory allocations.

Motivation Checkpoint

Every IDE (such as Keil, IAR, or STM32CubeIDE) hides the startup assembly and linker configuration from you behind templates. But when you move to automotive-grade ECUs, secure boot architectures, or custom bootloaders, you cannot use IDE templates. You must partition flash memories manually, direct code into secure sections, and initialize cores by manual scripts. Mastering startup sequences is the bridge to becoming a senior system architect.

Trainer Review Rule

• Pass Criteria:

- `build/output.map` maps `.isr_vector` exactly at address `0x08000000` (no offset).
- Executable links with zero standard libraries and builds with zero warnings.
- Startup copies initialized globals correctly (can be verified inside `gdb`).

• Fail Criteria:

- Linking standard library startup files (presence of standard libraries).
- Memory map shows unaligned sections or incorrect stack base limits.

Weekly Execution Plan: Week 08

Weekly Goal

Master low-level General Purpose Input/Output (GPIO) peripheral configuration, understand peripheral clock gating (RCC), configure pin electrical parameters, and write custom bare-metal drivers using direct register writes without HAL.

Weekly Outcome

By Friday, the student will have written a custom **Bare-Metal GPIO Driver** in C that configures pins as inputs (with internal pull-up/down resistors) and outputs (push-pull vs open-drain), controls the peripheral clocks via RCC, toggles physical LEDs, and reads physical pushbuttons.



Session Breakdown (Monday – Friday)

Monday: Clock Gating & The Reset and Clock Control (RCC)

- **Conceptual Objective:** Understand that microcontrollers disable all peripheral clocks at reset to conserve energy. Study the Reset and Clock Control (RCC) peripheral and learn how to enable peripheral clock gates (e.g., AHB1/AHB2 buses) before attempting to read/write peripheral registers.
- **Practical Activity:**
 - Open the STM32F407 Reference Manual and locate the RCC peripheral register map.
 - Find the exact bit in `RCC_AHB1ENR` required to enable the clock for GPIO Port A and Port D.
- **Code/Circuit Reference:**

```
#define RCC_BASE_ADDR    0x40023800U
#define RCC_AHB1ENR      ((volatile uint32_t*)(RCC_BASE_ADDR + 0x30U))

void gpio_enable_clock(void) {
    // Enable GPIOA (bit 0) and GPIOD (bit 3) clock gates
    *RCC_AHB1ENR |= (1U << 0) | (1U << 3);
}
```

Tuesday: GPIO Mode & Electrical Configuration Registers

- **Conceptual Objective:** Master the core configurations of a physical pin:

- **MODER:** Mode (Input, Output, Alternate Function, Analog).
- **OTYPER:** Output Type (Push-Pull: drives both high and low; Open-Drain: only drives low, requires pull-up resistor to pull high).
- **OSPEEDR:** Slew Rate / Speed (Low, Medium, Fast, High speed).
- **PUPDR:** Pull-Up/Pull-Down (No pull, Pull-up, Pull-down resistors).
- **Practical Activity:**
 - Study register maps for GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, and GPIOx_PUPDR.
 - Write C mask algorithms to set Pin 5 of Port A to Output Push-Pull, High Speed.
- **Code/Circuit Reference:**

```
#define GPIOA_BASE_ADDR    0x40020000U
#define GPIOA_MODER        ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x00U))
#define GPIOA_OTYPER        ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x04U))

void configure_pa5_output(void) {
    // PA5 → Mode output is '01' at bits 10-11
    *GPIOA_MODER &= ~(3U << (5 * 2)); // Clear bits 10-11
    *GPIOA_MODER |= (1U << (5 * 2)); // Set bits 10-11 to 01

    // PA5 → Output Type Push-Pull (bit 5 clear)
    *GPIOA_OTYPER &= ~(1U << 5);
}
```

Wednesday: GPIO Data Registers (IDR vs ODR)

- **Conceptual Objective:** Understand how to write to pins and read state:
- **Output Data Register (ODR):** Drives output high (`1`) or low (`0`).
- **Input Data Register (IDR):** Read-only register reflecting the physical logic levels on input pins.
- **Practical Activity:**
 - Connect a logic analyzer to physical GPIO output pins.
 - Write a program that toggles the ODR register and read back the logic levels inside IDR to confirm alignment.
- **Code/Circuit Reference:**

```
#define GPIOA_IDR          ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x10U))
#define GPIOA_ODR          ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x14U))

void toggle_output_pin(void) {
    *GPIOA_ODR ^= (1U << 5); // Toggle output state
}

bool read_input_pin(void) {
    return (*GPIOA_IDR & (1U << 0)) != 0; // Check state of PA0
}
```

Thursday: Alternate Function Multiplexing

- **Conceptual Objective:** Understand that MCU pins are shared between multiple peripherals (e.g., UART, SPI, I2C). Study the Alternate Function registers (AFRL/AFRH) which acts as internal multiplexers routing physical pins to target digital hardware controllers.
- **Practical Activity:**
 - Locate STM32 Alternate Function Mapping tables in datasheet.
 - Route PA2 and PA3 to UART2 TX/RX lines by writing Alternate Function Select values to AFR registers.
- **Code/Circuit Reference:**

```
#define GPIOA_AFR_L        ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x20U))

void route_pins_to_uart2(void) {
    // Enable alternate function mode on PA2 and PA3
    *GPIOA_MODER &= ~((3U << (2 * 2)) | (3U << (3 * 2)));
    *GPIOA_MODER |= ((2U << (2 * 2)) | (2U << (3 * 2))); // Set mode '10' (Alt
Function)

    // Set AF7 (UART2) on Pin 2 (AFRL[11:8]) and Pin 3 (AFRL[15:12])
    *GPIOA_AFR_L &= ~((0xFU << (2 * 4)) | (0xFU << (3 * 4)));
    *GPIOA_AFR_L |= ((7U << (2 * 4)) | (7U << (3 * 4)));
}
```

Friday: Driving GPIO atomic writes via BSRR

- **Conceptual Objective:** Master the Bit Set/Reset Register (BSRR) mechanics. Understand why BSRR enables atomic bit writes without Read-Modify-Write instructions, making it safe from preemption race conditions inside ISRs.
- **Practical Activity:**

- Write a speed test loop comparing execution cycles of direct ODR toggles versus BSRR writes.
- Document timing differences.
- **Code/Circuit Reference:**

```
#define GPIOA_BSRR      ((volatile uint32_t*)(GPIOA_BASE_ADDR + 0x18U))

void set_pin_atomic(void) {
    *GPIOA_BSRR = (1U << 5); // Set PA5 High
}

void reset_pin_atomic(void) {
    *GPIOA_BSRR = (1U << (5 + 16)); // Reset PA5 Low (write to high half register)
}
```

Hands-On Assignment: Custom Register-Level GPIO Driver Library

Functional Requirements

Create a professional-grade, register-level GPIO driver library (`gpio.c` , `gpio.h`) that provides safe register manipulation and electrical protection configurations. 1. The driver must define an enumeration representing Pin Numbers (`PIN0` to `PIN15`), Modes (`INPUT` , `OUTPUT` , `ALT_FUNC` , `ANALOG`), Output Types (`PUSH_PULL` , `OPEN_DRAIN`), and Pull Resistors (`NO_PULL` , `PULL_UP` , `PULL_DOWN`). 2. Implement functional configurations APIs that cleanly calculate register bit offsets using shift operators. 3. Your input pins initialization configuration must explicitly clear other pins register space to prevent corruption. 4. Implement a custom debounce algorithm in software to read an input button. When the button is pressed, toggle an LED output. 5. Create a compilation test package targeting the STM32F4 Discovery board (Pin PA0 is the User Button, Pin PD12 is the Green LED). Hook up your logic analyzer tx line to the LED pin to measure debounce time.

Structural Requirements & Code Skeleton

`gpio.h`:

```

#ifndef GPIO_H
#define GPIO_H

#include <stdint.h>
#include <stdbool.h>

// Register structure representing GPIO controller block
typedef struct {
    volatile uint32_t MODER;    // Offset 0x00
    volatile uint32_t OTYPER;   // Offset 0x04
    volatile uint32_t OSPEEDR;  // Offset 0x08
    volatile uint32_t PUPDR;    // Offset 0x0C
    volatile uint32_t IDR;       // Offset 0x10
    volatile uint32_t ODR;       // Offset 0x14
    volatile uint32_t BSRR;      // Offset 0x18
} gpio_registers_t;

// Port addresses mapped directly to memory
#define GPIOA_PORT ((gpio_registers_t*)0x40020000U)
#define GPIOD_PORT ((gpio_registers_t*)0x40020C00U)

typedef enum {
    MODE_IN = 0,
    MODE_OUT = 1,
    MODE_ALT = 2,
    MODE_ANALOG = 3
} gpio_mode_t;

typedef enum {
    PULL_NONE = 0,
    PULL_UP = 1,
    PULL_DOWN = 2
} gpio_pull_t;

void gpio_init_pin(gpio_registers_t *port, uint8_t pin, gpio_mode_t mode, gpio_pull_t pull);
void gpio_write(gpio_registers_t *port, uint8_t pin, bool state);
void gpio_toggle(gpio_registers_t *port, uint8_t pin);
bool gpio_read(const gpio_registers_t *port, uint8_t pin);

#endif // GPIO_H

```

gpio.c (C-Skeleton):

```
#include "gpio.h"

#define RCC_AHB1ENR ((volatile uint32_t*)0x40023830U)

void gpio_init_pin(gpio_registers_t *port, uint8_t pin, gpio_mode_t mode, gpio_pull_t pull) {
    // 1. Enable Clocks depending on port address
    if (port == GPIOA_PORT) {
        *RCC_AHB1ENR |= (1U << 0);
    } else if (port == GPIOD_PORT) {
        *RCC_AHB1ENR |= (1U << 3);
    }

    // 2. Configure Mode
    port->MODER &= ~(3U << (pin * 2));
    port->MODER |= ((uint32_t)mode << (pin * 2));

    // 3. Configure Pull-up/pull-down
    port->PUPDR &= ~(3U << (pin * 2));
    port->PUPDR |= ((uint32_t)pull << (pin * 2));
}

void gpio_write(gpio_registers_t *port, uint8_t pin, bool state) {
    if (state) {
        port->BSRR = (1U << pin); // Set atomic
    } else {
        port->BSRR = (1U << (pin + 16)); // Reset atomic
    }
}

void gpio_toggle(gpio_registers_t *port, uint8_t pin) {
    port->ODR ^= (1U << pin);
}

bool gpio_read(const gpio_registers_t *port, uint8_t pin) {
    return (port->IDR & (1U << pin)) != 0;
}
```

Deliverables

- [] Driver library files `gpio.h` and `gpio.c`.
- [] Application entry point `main.c` polling button PA0 and toggling LED PD12 using custom software debounce metrics.
- [] Makefile building output binaries.

? Self-Check Questions

1. Why must peripheral clocks be enabled before writing config parameters to peripheral registers? What is the physical core response if clock gating is bypassed?
2. What are the electrical differences between **Push-Pull** and **Open-Drain** output types? Give a standard real-world communication peripheral example that requires open-drain pins.
3. How do **Pull-up** and **Pull-down** resistors secure input pin voltages? What is a floating electrical state, and what is its effect on microcontroller power consumption?
4. Explain how a software button-debounce loop works. What is the physical duration range of mechanical button bouncing?
5. Why are writes to BSRR faster and safer than read-modify-write structures like `ODR |= MASK` inside real-time operating systems?

Stretch Task (Optional)

Extend the GPIO driver library to support configuring and enabling Alternate Functions via APIs like `gpio_init_alt_func(gpio_registers_t *port, uint8_t pin, uint8_t af_number)`. Integrate strict bounds-checking to automatically isolate between AFRL (pins 0-7) and AFRH (pins 8-15) boundaries.

Motivation Checkpoint

GPIO config is the absolute entry point to any hardware physical design. Whether you are lighting an indicator LED, reading a proximity sensor, triggering an external reset line, or routing physical signals to high-speed UART buses, you must manually align electrical configurations. Direct register writes ensure your configurations are precise and footprint-minimal.

Trainer Review Rule

- **Pass Criteria:**

- GPIO registers mapped using exact offsets via structure pointer mappings.
- Driver does not reference ST HAL structures.
- Clock gating is handled dynamically.
- Debounce logic is fully functional on target button.
- **Fail Criteria:**
 - Hard-coded magic parameters inside registers write loop.
 - Bouncing outputs captured on logic analyzer Tx testing.

Weekly Execution Plan: Week 09

Weekly Goal

Master Nested Vectored Interrupt Controller (NVIC) architectures, configure physical External Interrupts (EXTI) triggered by physical input pin state transitions, understand ARM interrupt context-saving dynamics, and manage software race conditions.

Weekly Outcome

By Friday, the student will have written a **Bare-Metal Interrupt-Driven System** on the STM32F4. They will have configured the EXTI controller and the NVIC directly by registers, written an Interrupt Service Routine (ISR) to handle external push-button events dynamically without blocking loop controls, and implemented proper register protections to resolve shared-data race conditions.



Session Breakdown (Monday - Friday)

Monday: The NVIC Architecture & Core Interrupt Controller

- **Conceptual Objective:** Study the Nested Vectored Interrupt Controller (NVIC) integrated inside the ARM core. Understand how NVIC manages prioritize exceptions, interrupt nesting (high priority preempts low priority), vector address routing, and core exception models.
- **Practical Activity:**
 - Read the NVIC core registers specifications (ISER, ICER, ISPR, ICPR, IPR).
 - Map out how priorities are prioritized inside ARM's 8-bit priority registers (only upper 4 bits are physically implemented on STM32F4).
- **Code/Circuit Reference:**

```

#define NVIC_BASE_ADDR    0xE000E100U
#define NVIC_ISER0        ((volatile uint32_t*)(NVIC_BASE_ADDR + 0x00U)) // Interrupt
Set-Enable Register
#define NVIC_IPR           ((volatile uint8_t*)(NVIC_BASE_ADDR + 0x300U)) // Interrupt
Priority Register

void nvic_enable_irq(uint8_t irq_number) {
    // Each ISER register covers 32 IRQs
    *NVIC_ISER0 |= (1U << irq_number);
}

```

Tuesday: External Interrupt Controller (EXTI) & System Configuration (SYSCFG)

- **Conceptual Objective:** Understand that ARM core cannot directly monitor GPIO pins for interrupts. The MCU routes GPIO lines to the External Interrupt (EXTI) controller via the System Configuration (SYSCFG) peripheral multiplexer. EXTI monitors lines for falling/rising signal edges.
- **Practical Activity:**
 - Learn how EXTI lines (0 to 15) are mapped to GPIO pin indices.
 - Configure SYSCFG to route Pin 0 of Port A to EXTI line 0.
- **Code/Circuit Reference:**

```

#define SYSCFG_BASE      0x40013800U
#define SYSCFG_EXTICR1   ((volatile uint32_t*)(SYSCFG_BASE + 0x08U))

#define EXTI_BASE        0x40013C00U
#define EXTI_IMR         ((volatile uint32_t*)(EXTI_BASE + 0x00U)) // Interrupt Mask
Register
#define EXTI_RTSCR       ((volatile uint32_t*)(EXTI_BASE + 0x08U)) // Rising Trigger
Select
#define EXTI_FTSR       ((volatile uint32_t*)(EXTI_BASE + 0x0CU)) // Falling Trigger
Select

void configure_exti0_falling_edge(void) {
    // 1. Route PA0 to EXTI0 via SYSCFG_EXTICR1 (clear bits 0-3 for Port A)
    *SYSCFG_EXTICR1 &= ~(0xFU << 0);

    // 2. Unmask EXTI Line 0
    *EXTI_IMR |= (1U << 0);

    // 3. Enable Falling Edge Trigger, disable Rising Edge
    *EXTI_FTSR |= (1U << 0);
    *EXTI_RTSCR &= ~(1U << 0);
}

```

Wednesday: Interrupt Service Routine (ISR) Mechanics & Vector Table Linking

- **Conceptual Objective:** Comprehend the exact processor context-saving sequence during exception entry:
 - Current instruction finishes execution.
 - Core automatically pushes critical registers (R0-R3, R12, LR, PC, xPSR) to the active Stack (MSP or PSP).
 - Core switches to Handler Mode (Privileged).
 - Core fetches matching ISR address from the Vector Table and branches.
 - Upon exit, the special return code (`EXC_RETURN`) written to PC triggers automatic stack restoration.
- **Practical Activity:**
 - Open your `startup.c` file from Week 7.
 - Implement `EXTI0_IRQHandler` in C, declare it inside the physical Vector Table array at index offset 22 (the EXTI0 slot on STM32F4).
- **Code/Circuit Reference:**


```
// Inside startup.c
void EXTI0_IRQHandler(void) __attribute__((weak, alias("Default_Handler")));

// Vector table array updated
const element_t vector_table[] = {
    // Core exceptions ...
    [16 + 6] = EXTI0_IRQHandler // Vector Table Offset mapping for EXTI0 (Interrupt ID
6)
};
```

Thursday: Interrupt Cleansings & Edge Processing

- **Conceptual Objective:** Understand that hardware requires software to explicitly acknowledge and clear the pending state of an interrupt within the ISR (e.g., writing a `1` to the pending bit in `EXTI_PR`). If the pending bit is not cleared, the core immediately re-triggers the exception on exit, locking up the system.
- **Practical Activity:**
 - Write the core implementation of `EXTI0_IRQHandler`.
 - Read the active state of LED and toggle it, then clear the pending flag.
- **Code/Circuit Reference:**

```
#define EXTI_PR          ((volatile uint32_t*)(EXTI_BASE + 0x14U)) // Pending
Register
#define GPIOD_ODR        ((volatile uint32_t*)0x40020C14U)

void EXTI0_IRQHandler(void) {
    // Verify interrupt is active on EXTI0
    if (*EXTI_PR & (1U << 0)) {
        // Toggle PD12 LED
        *GPIOD_ODR ^= (1U << 12);

        // Clear pending bit (write 1 to clear in STM32!)
        *EXTI_PR = (1U << 0);
    }
}
```

Friday: Race Conditions & Shared Resource Protections

- **Conceptual Objective:** Master the concept of a Race Condition—corruptions that occur when an ISR modifies a shared global variable while the background main loop is reading/modifying it.

Learn how to protect critical sections using global interrupt disable controls (`__disable_irq()` / `__enable_irq()`).

- **Practical Activity:**

- Write a program where a global counter is modified inside an ISR and inside a main loop.
- Implement proper atomic boundaries around the main loop access using primitive registers blocks.

- **Code/Circuit Reference:**

```
volatile uint32_t g_event_counter = 0; // volatile correct

void process_data_safely(void) {
    // Enter Critical Section: disable interrupts
    __asm volatile("cpsid i" : : : "memory");

    // Safe access
    uint32_t local_copy = g_event_counter;
    g_event_counter = 0;

    // Exit Critical Section: enable interrupts
    __asm volatile("cpsie i" : : : "memory");
}
```

Hands-On Assignment: Bare-Metal Interrupt Driver with Critical Sections

Functional Requirements

Create a fully functional, bare-metal interrupt-driven application on the STM32F4. 1. Wire an external pushbutton to Pin PA0 (or use the onboard User Button). Configure PA0 as an input with Pull-down resistor. 2. Route PA0 to EXTI0 via SYSCFG. EXTI0 must trigger on a Rising Edge (when button is pressed). 3. Enable EXTI0 (Interrupt position 6) inside the NVIC. Configure priority to level 2. 4. Set up PD12 (LED) as a General Purpose Output. 5. In `startup.c`, ensure the Vector Table maps `EXTI0_IRQHandler` at vector position 22. 6. The ISR `EXTI0_IRQHandler` must: - Verify the EXTI pending bit. - Increment a volatile global counter variable `g_button_press_count`. - Clear the pending EXTI register flag. 7. In the `main.c` background loop, check `g_button_press_count`. If it is non-zero, print the count (or simulate a serial output) and decrement the counter safely. You must protect this check-and-decrement step inside the main loop using a **critical section** to prevent data corruption.

Structural Requirements & Code Skeleton

main.c (C-Skeleton):

```
#include <stdint.h>
#include <stdbool.h>

// Shared resource
volatile uint32_t g_button_press_count = 0;

void configure_peripherals(void);

void main(void) {
    configure_peripherals();

    while (1) {
        uint32_t count = 0;

        // 1. Enter Critical Section to read and reset shared variable
        __asm volatile("cpsid i" : : : "memory");
        if (g_button_press_count > 0) {
            count = g_button_press_count;
            g_button_press_count = 0; // Clear it
        }
        __asm volatile("cpsie i" : : : "memory");

        // 2. Process data outside critical section to keep interrupts unblocked
        if (count > 0) {
            // Toggle LED PD12
            volatile uint32_t *odr = (volatile uint32_t*)0x40020C14U;
            *odr ^= (1U << 12);
        }
    }
}
```

Deliverables

- [] Startup file `startup.c` containing the updated Vector Table array.
- [] Application file `main.c` containing EXTI and NVIC configurations, the critical section protection routines, and the target ISR.
- [] Clean compilation Makefile.

? Self-Check Questions

1. Why must the EXTI pending bit be cleared inside the ISR? What physically happens if you forget to write to the pending register bit?
 2. What registers does the Cortex-M4 core push to the stack automatically during interrupt entry? Why are R4-R11 left off the stacking list, and who is responsible for protecting them?
 3. How does priority grouping work inside the NVIC? Explain the difference between **Preemption Priority** and **Sub-priority**.
 4. What is the role of the C compiler keyword `volatile` when applied to global variables shared between an ISR and `main()`? What happens if `volatile` is omitted?
 5. Why should code inside an ISR run as fast as possible? What is the architectural penalty of placing long delay loops or processing calls inside an ISR?
-

Stretch Task (Optional)

Extend the NVIC driver to configure a second interrupt channel (e.g., EXTI1 mapped to pin PA1). Establish different priorities for EXTI0 (High priority) and EXTI1 (Low priority). Write code demonstrating preemption: trigger EXTI1, execute a delay loop, trigger EXTI0, and verify EXTI0 instantly interrupts EXTI1.

Motivation Checkpoint

Using polling routines (`while(button_state == 0)`) to monitor events is highly inefficient; the CPU is wasted spinning in circles, unable to process other computations. Real-time systems operate strictly on **Interrupt-Driven Architectures**. But introducing interrupts introduces concurrency, which yields race conditions. Mastering NVIC, EXTI registers, and critical section isolation is standard requirement for high-reliability engineers.

Trainer Review Rule

• Pass Criteria:

- EXTI configuration done strictly via direct register addresses.
- Vector Table maps `EXTI0_IRQHandler` at index position 22.
- Critical sections are isolated using assembly interrupt control instructions (`cpsid i` / `cpsie i`).
- System executes flawlessly without lock-ups during rapid button presses.

• Fail Criteria:

- Delay loops or polling loops inside ISR blocks.
- Shared data access inside main loop executed without atomic protection wrappers.

Weekly Execution Plan: Week 10

Weekly Goal

Master microcontroller hardware timer architectures, configure the SysTick core timer for precise millisecond-level tick generation, understand prescalers and Auto-Reload registers (ARR), and generate precise software delays and timed interrupts.

Weekly Outcome

By Friday, the student will have configured both the **SysTick Timer** and a general-purpose hardware timer (**TIM2**) at the register level on the STM32F4. They will be able to calculate timer parameters, write precise non-blocking delay routines, and generate periodic interrupts to drive background tasks.



Session Breakdown (Monday – Friday)

Monday: The ARM SysTick Core Timer

- **Conceptual Objective:** Study the SysTick Timer—a simple, 24-bit down-counter integrated inside the ARM Cortex-M core. Understand how it acts as the primary timekeeping engine for operating systems (RTOS) and basic timing systems.
- **Practical Activity:**
 - Read the SysTick registers specifications (CSR, RVR, CVR, CALIB).
 - Calculate the Reload Value required to generate a precise 1ms interrupt under a core system clock of 16MHz.
- **Code/Circuit Reference:**

```

#define SYST_BASE          0xE000E010U
#define SYST_CSR            ((volatile uint32_t*)(SYST_BASE + 0x00U)) // Control and
Status
#define SYST_RVR            ((volatile uint32_t*)(SYST_BASE + 0x04U)) // Reload Value
#define SYST_CVR            ((volatile uint32_t*)(SYST_BASE + 0x08U)) // Current Value

volatile uint32_t g_system_ticks = 0;

void systick_init_1ms(uint32_t system_clock_hz) {
    // 1. Calculate reload value (Value = Clock / 1000 - 1)
    *SYST_RVR = (system_clock_hz / 1000U) - 1U;

    // 2. Clear current value
    *SYST_CVR = 0;

    // 3. Enable SysTick: Core Clock Source (bit 2), Interrupt Enable (bit 1), Counter
    Enable (bit 0)
    *SYST_CSR |= (1U << 2) | (1U << 1) | (1U << 0);
}

```

Tuesday: Writing SysTick ISR & Delay Functions

- **Conceptual Objective:** Learn how to write the SysTick Interrupt Handler to manage time. Study how to construct non-blocking delays using timed variables compared to basic, CPU-blocking delays.
- **Practical Activity:**
 - Add `SysTick_Handler` to your startup file Vector Table at index 15.
 - Write a highly accurate millisecond delay function `delay_ms(uint32_t ms)`.
- **Code/Circuit Reference:**

```
// Inside startup.c vector table slot 15:
// [15] = SysTick_Handler

void SysTick_Handler(void) {
    g_system_ticks++;
}

// Precise milli-second delay
void delay_ms(uint32_t ms) {
    uint32_t start = g_system_ticks;
    while ((g_system_ticks - start) < ms) {
        // Yield or wait (using volatile checks)
    }
}
```

Wednesday: General-Purpose Timer Architectures (TIM2-TIM5)

- **Conceptual Objective:** Master the architecture of a 16-bit/32-bit general-purpose hardware timer block. Understand:
- **Prescaler (PSC):** Divides the input bus clock to run the timer at a lower, calculated frequency.
- **Auto-Reload Register (ARR):** Holds the maximum count value. When the counter reaches ARR, it overflows (resets to 0) and generates an Update Event.
- **Counter (CNT):** The physical register that increments on every timer tick.
- **Practical Activity:**
 - Open the STM32F4 Reference Manual and locate the TIM2 register map (TIMx_CR1, TIMx_DIER, TIMx_SR, TIMx_PSC, TIMx_ARR, TIMx_CNT).
 - Calculate PSC and ARR values to configure TIM2 to run at 10kHz and trigger an update interrupt every 500ms.
- **Code/Circuit Reference:**

Timer Clock Equation:

Target frequency (f) = Bus Clock / ((PSC + 1) * (ARR + 1))

Example: Bus Clock = 16,000,000Hz (16MHz)

Choose PSC = 1599 (Timer clock = 16,000,000 / (1599 + 1) = 10,000Hz = 10kHz)

Choose ARR = 4999 (Update rate = 10,000 / (4999 + 1) = 2Hz = 0.5s = 500ms)

Thursday: Timer Interrupt Configurations

- **Conceptual Objective:** Learn how to route timer update events to trigger NVIC interrupts. Study the Timer DMA/Interrupt Enable register (DIER) and status register (SR).

• **Practical Activity:**

- Write a C function initializing TIM2 with calculated PSC/ARR values.
- Enable the Update Interrupt bit inside TIM2_DIER and enable TIM2 (IRQ position 28) inside the NVIC ISER register.

• **Code/Circuit Reference:**

```
#define TIM2_BASE          0x40000000U
#define TIM2_CR1           ((volatile uint32_t*)(TIM2_BASE + 0x00U))
#define TIM2_DIER          ((volatile uint32_t*)(TIM2_BASE + 0x0CU)) // DMA/Interrupt
                             Enable
#define TIM2_SR            ((volatile uint32_t*)(TIM2_BASE + 0x10U)) // Status Register
#define TIM2_PSC           ((volatile uint32_t*)(TIM2_BASE + 0x28U))
#define TIM2_ARR           ((volatile uint32_t*)(TIM2_BASE + 0x2CU))

#define RCC_APB1ENR        ((volatile uint32_t*)0x40023840U)

void tim2_init_500ms(void) {
    // 1. Enable TIM2 clock gate on APB1 bus (bit 0)
    *RCC_APB1ENR |= (1U << 0);

    // 2. Set Prescaler and Auto-reload
    *TIM2_PSC = 1599U;
    *TIM2_ARR = 4999U;

    // 3. Enable Update Interrupt
    *TIM2_DIER |= (1U << 0); // UIE bit

    // 4. Enable Counter
    *TIM2_CR1 |= (1U << 0); // CEN bit
}
```

Friday: Input Capture and Output Compare Concepts

- **Conceptual Objective:** Introduce advanced timer peripheral functions:
- **Output Compare:** Automatically toggles/sets a physical pin state when the CNT matches a predefined Capture/Compare Register (CCR) value (basis of PWM).
- **Input Capture:** Records the exact value of CNT when a signal transition (rising/falling edge) occurs on a physical pin (basis of frequency measurement).
- **Practical Activity:**
 - Study CCR and CCMR registers maps.
 - Draw the functional block comparison flowchart of output compare matching.

Hands-On Assignment: Bare-Metal Timer & Heartbeat Interrupt Generator

Functional Requirements

Create a fully functional, bare-metal application that uses SysTick for millisecond timing delays and TIM2 periodic interrupts to toggle a heartbeat LED. 1. Configure **SysTick** to trigger an interrupt every 1ms under a 16MHz CPU frequency. Write a non-blocking `delay_ms(uint32_t ms)` function based on a volatile global system tick. 2. Initialize **TIM2** at the register level to generate a periodic update event every 500ms. Enable the TIM2 update interrupt. 3. Enable TIM2 interrupt (IRQ position 28) inside the NVIC. Set its priority lower than SysTick. 4. Implement `TIM2_IRQHandler` in `startup.c` (slot offset 44 in Vector Table). Inside the handler: - Check the update interrupt pending status flag in `TIM2_SR`. - Clear the status flag (write `0` to bit 0 of `TIM2_SR`). - Toggle a second LED (e.g., PD13). 5. In `main.c`, use your SysTick-based `delay_ms` inside the background loop to toggle PD12 LED every 1 second. The TIM2 interrupt must run concurrently, producing a rapid 500ms blink on PD13 alongside the slower 1-second blink on PD12.

Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):

```

#include <stdint.h>
#include <stdbool.h>

extern volatile uint32_t g_system_ticks;
void delay_ms(uint32_t ms);
void systick_init_1ms(uint32_t clk);
void tim2_init_500ms(void);
void configure_leds(void);

void main(void) {
    systick_init_1ms(16000000U);
    tim2_init_500ms();
    configure_leds();

    // Enable IRQ 28 (TIM2) in NVIC
    volatile uint32_t *isr0 = (volatile uint32_t*)0xE000E100U;
    *isr0 |= (1U << 28);

    volatile uint32_t *bsrr = (volatile uint32_t*)0x40020C18U;

    while (1) {
        // Toggle PD12 Led using SysTick delay
        *bsrr = (1U << 12); // High
        delay_ms(1000);
        *bsrr = (1U << (12 + 16)); // Low
        delay_ms(1000);
    }
}

```

TIM2_IRQHandler (C-Skeleton):

```

#define TIM2_SR    ((volatile uint32_t*)0x40000010U)
#define GPIOD_ODR  ((volatile uint32_t*)0x40020C14U)

void TIM2_IRQHandler(void) {
    if (*TIM2_SR & (1U << 0)) {
        // Toggle PD13 LED
        *GPIOD_ODR ^= (1U << 13);

        // Clear update interrupt flag
        *TIM2_SR &= ~(1U << 0);
    }
}

```

Deliverables

- [] Updated `startup.c` containing Vector Table slots for `SysTick_Handler` and `TIM2_IRQHandler`.
- [] Application source code `main.c` containing initialization equations, delay loops, and ISRs.
- [] Clean compilation Makefile.

? Self-Check Questions

1. How does timer clock prescaling physically differ from clock division? How does the processor use the Prescaler (PSC) and Auto-Reload (ARR) registers together to control timing?
2. Why is it necessary to declare `g_system_ticks` with the `volatile` qualifier? What error would occur if the compiler optimized away reads to this variable inside `delay_ms()`?
3. Show the math calculation required to generate a 1-second periodic update rate on TIM2 if the APB1 clock is running at 42MHz.
4. Why must you clear the update status register flag in `TIM2_SR` inside the ISR? What happens if you clear it at the beginning of the ISR vs at the end of the ISR?
5. Why are hardware timer delays superior to simple assembly-instruction delay loops (e.g., `for(int i=0; i<10000; i++);`)?

Stretch Task (Optional)

Modify the project to configure **TIM2** in **Output Compare Mode**. Connect your logic analyzer to the physical pin associated with TIM2 Channel 1 (e.g., PA5) and demonstrate a hardware-generated PWM clock signal running at 1kHz. Change the Compare Register value dynamically to change duty cycles.

Motivation Checkpoint

In automotive subsystems, avionics networks, and robotics actuators, timing is everything. Whether you are generating precise motor control steps, calculating UART packet timeout bounds, or executing motor PWM, standard delay loops are useless because they waste CPU power. Mastering hardware prescalers, SysTick, and NVIC timed interrupts is a core skill for any embedded engineer.

Trainer Review Rule

• Pass Criteria:

- Timer and SysTick configured purely by register assignments.
- SysTick delay does not lock up or slip ticks during interrupt execution.
- Capturing PD12 and PD13 LED waveforms on logic analyzer proves exact frequencies of 1.0Hz and 2.0Hz.

• Fail Criteria:

- Hard-coded clock timing loop adjustments.
- Missing status register clears.

Weekly Execution Plan: Week 11

Weekly Goal

Master physical-layer UART (Universal Asynchronous Receiver-Transmitter) communications, calculate precise Baud Rate divisor values directly from clock bus frequencies, write register-level transmit/receive drivers, and implement a high-reliability, interrupt-driven UART ring buffer.

Weekly Outcome

By Friday, the student will have written a complete, bare-metal **UART Serial Driver** on the STM32F4. They will be able to transmit and receive data cleanly, handle character reception dynamically using NVIC interrupts, and structure a thread-safe software Ring Buffer to prevent data losses at high transmission speeds (up to 115200 baud).

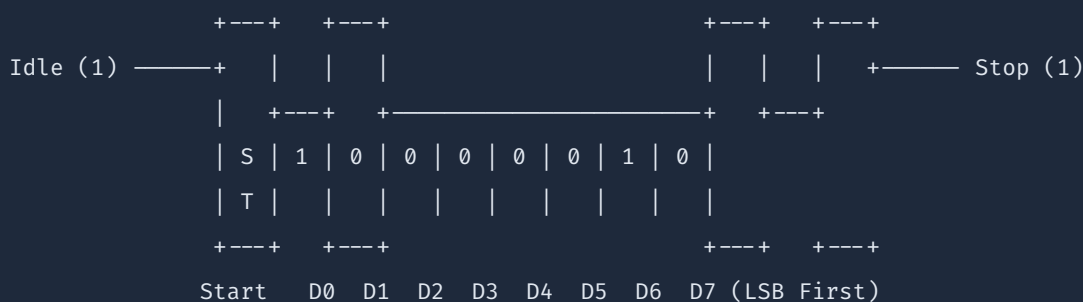
Session Breakdown (Monday - Friday)

Monday: The UART Physical Specification & Frame Layout

- **Conceptual Objective:** Master the timing of asynchronous communications. Understand the composition of a standard UART packet frame: Start Bit (low transition), Data Bits (typically 8 bits, sent LSB first), Parity Bit (optional error checking), and Stop Bits (high transition).
- **Practical Activity:**
 - Sketch a physical timing diagram for the transmission of the ASCII character 'A' ($0 \times 41 = 0b01000001$) with configuration 8N1 (8 data bits, no parity, 1 stop bit).
 - Map out how high and low states represent physical logic levels.
- **Code/Circuit Reference:**

UART 8N1 Timing Diagram (ASCII 'A' = 0×41):

Physical line Idle: HIGH (1)



Tuesday: UART Register Architecture & Baud Rate DIV Calculations

- **Conceptual Objective:** Study UART controller registers (SR, DR, BRR, CR1, CR2, CR3). Master the mathematics required to convert peripheral bus clocks (APB1/APB2) into target baud rates using the fractional divider inside the Baud Rate Register (BRR).
- **Practical Activity:**
 - Write down the clock division equation: $\text{USARTDIV} = \text{Clock} / (16 \times \text{Baud})$.
 - Compute the exact values to load into the 16-bit register USARTx_BRR to generate 115200 baud under a peripheral clock of 16MHz.
- **Code/Circuit Reference:**

Baud Rate Divider Math:

Target Baud = 115200, Clock = 16,000,000Hz

$\text{USARTDIV} = 16,000,000 / (16 \times 115200) = 16,000,000 / 1843200 = 8.6805$

Mantissa (Integer part) = 8 = 0x08

Fraction part = $0.6805 \times 16 = 10.888 \rightarrow$ Round to nearest int = 11 = 0x0B

Result BRR Value = $(\text{Mantissa} \ll 4) \mid \text{Fraction} = (0x08 \ll 4) \mid 0x0B = 0x008B$

Wednesday: Bare-Metal Polling UART Driver

- **Conceptual Objective:** Learn how to configure clocks, GPIO alternate function pins, and enable UART transmitter and receiver paths in software. Write simple polling transmission and reception routines using status register flags (TXE: Transmit Data Register Empty, RXNE: Read Data Register Not Empty).
- **Practical Activity:**
 - Implement register configurations for UART2 (TX on pin PA2, RX on pin PA3).
 - Write functional polling interfaces `uart_write_char` and `uart_read_char`.
- **Code/Circuit Reference:**

```

#define USART2_BASE      0x40004400U
#define USART2_SR        ((volatile uint32_t*)(USART2_BASE + 0x00U))
#define USART2_DR        ((volatile uint32_t*)(USART2_BASE + 0x04U))
#define USART2_BRR       ((volatile uint32_t*)(USART2_BASE + 0x08U))
#define USART2_CR1       ((volatile uint32_t*)(USART2_BASE + 0x0CU))

void uart2_init_polling(void) {
    // Enable GPIOA clock and UART2 clock
    *((volatile uint32_t*)0x40023830U) |= (1U << 0); // GPIOAEN
    *((volatile uint32_t*)0x40023840U) |= (1U << 17); // USART2EN

    // Route PA2/PA3 to Alt Function 7 (UART) ... (done via GPIO registers)

    // Set Baud Rate to 115200 (using calculated DIV = 0x008B)
    *USART2_BRR = 0x008BU;

    // Enable UART (bit 13), Enable Transmitter (bit 3), Enable Receiver (bit 2)
    *USART2_CR1 |= (1U << 13) | (1U << 3) | (1U << 2);
}

void uart_write_char(char c) {
    // Wait until transmit register is empty (TXE bit 7 in SR)
    while (!(*USART2_SR & (1U << 7)));
    *USART2_DR = (uint32_t)c;
}

```

Thursday: Designing a Circular Ring Buffer Architecture

- **Conceptual Objective:** Understand that polling for data is highly vulnerable to data loss: if the CPU is busy processing other tasks when a character arrives, the UART controller's single character buffer is overwritten, causing an Overrun Error (ORE). Study the architecture of a thread-safe software **Ring Buffer** (FIFO queue) to queue data dynamically.
- **Practical Activity:**
 - Define a Ring Buffer structure containing a storage array, head pointer, tail pointer, and write/read tracker metrics.
 - Implement functional, non-blocking queue operations.
- **Code/Circuit Reference:**


```

#define RING_BUF_SIZE 128

typedef struct {
    uint8_t buffer[RING_BUF_SIZE];
    volatile uint32_t head;
    volatile uint32_t tail;
} ring_buffer_t;

bool ring_buffer_push(ring_buffer_t *rb, uint8_t val) {
    uint32_t next = (rb->head + 1) % RING_BUF_SIZE;
    if (next == rb->tail) {
        return false; // Buffer full! Data overflowed.
    }
    rb->buffer[rb->head] = val;
    rb->head = next;
    return true;
}

```

Friday: Interrupt-Driven UART RX Ring Buffer

- **Conceptual Objective:** Learn how to configure UART to generate a hardware interrupt automatically when a character is received (RXNEIE: RXNE Interrupt Enable). Route the interrupt to the NVIC and implement the ISR to read data instantly and queue it inside your ring buffer.
- **Practical Activity:**
 - Connect a USB-to-UART converter between your microcontroller and your PC host.
 - Execute a serial terminal program (e.g., Minicom). Transmit blocks of characters at high speed and verify that zero letters are dropped by the microcontroller using your ring buffer.
- **Code/Circuit Reference:**

```

// Inside USART2 Init:
*USART2_CR1 |= (1U << 5); // Enable RXNE Interrupt

// USART2 IRQ Handler in startup.c
void USART2_IRQHandler(void) {
    if (*USART2_SR & (1U << 5)) { // Check RXNE flag
        uint8_t received_data = (uint8_t)(*USART2_DR & 0xFF);
        ring_buffer_push(&g_rx_buffer, received_data); // Push dynamically
    }
}

```

Hands-On Assignment: Reliable Interrupt-Driven UART Driver Suite

Functional Requirements

Create a professional-grade, bare-metal UART driver suite containing high-reliability interrupt-driven character queuing. 1. The driver must support two files: `uart.c` and `uart.h`. 2. Configure **USART2** to run at **115200 Baud, 8N1** (8 data bits, no parity, 1 stop bit) under 16MHz clock conditions. 3. Configure and initialize an RX ring buffer structure dynamically. 4. Enable the **RXNE Interrupt** on USART2 and map `USART2_IRQHandler` at position 38 in the Vector Table. 5. Provide a non-blocking API: `bool uart_read_byte(uint8_t *data)` that extracts data from the Ring Buffer. If the buffer is empty, it returns `false`. 6. Write a robust string transmitter API: `void uart_write_string(const char *str)`. 7. In `main.c`, build a command-line echo system: whenever a character is received via interrupt and pushed to the ring buffer, the main loop must extract it safely, convert it to uppercase, and transmit it back to the host terminal.

Structural Requirements & Code Skeleton

```
uart.h: ````ifndef UART_H
```

define UART_H

include

include

```
void uart2_init(void); void uart_write_byte(uint8_t byte); void uart_write_string(const char str); bool  
uart_read_byte(uint8_t data);
```

endif // UART_H

```

**`uart.c` (C-Skeleton)**:
```c
#include "uart.h"

#define USART2_BASE 0x40004400U
#define USART2_SR ((volatile uint32_t*)(USART2_BASE + 0x00))
#define USART2_DR ((volatile uint32_t*)(USART2_BASE + 0x04))
#define USART2_BRR ((volatile uint32_t*)(USART2_BASE + 0x08))
#define USART2_CR1 ((volatile uint32_t*)(USART2_BASE + 0x0C))

#define RING_BUF_SIZE 256
typedef struct {
 uint8_t buffer[RING_BUF_SIZE];
 volatile uint32_t head;
 volatile uint32_t tail;
} ring_buffer_t;

static ring_buffer_t rx_buffer = { {0}, 0, 0 };

void uart2_init(void) {
 // 1. Enable Clocks for GPIOA and USART2
 ((volatile uint32_t)0x40023830U) |= (1U << 0); // GPIOA
 ((volatile uint32_t)0x40023840U) |= (1U << 17); // USART2

 // 2. Configure Pin Modes (Alt Function PA2 & PA3)
 // [GPIO pin routes AF7 to PA2 and PA3]

 // 3. Set Baud Rate to 115200 (DIV = 0x008B)
 *USART2_BRR = 0x008BU;

 // 4. Enable RXNE interrupt
 *USART2_CR1 |= (1U << 5);

 // 5. Enable UART, TX, RX
 *USART2_CR1 |= (1U << 13) | (1U << 3) | (1U << 2);
}

void uart_write_byte(uint8_t byte) {
 while (!(*USART2_SR & (1U << 7))); // Wait for TXE
 *USART2_DR = byte;
}

```

```

void uart_write_string(const char *str) {
 while (*str) {
 uart_write_byte((uint8_t)*str++);
 }
}

bool uart_read_byte(uint8_t *data) {
 // Enter critical section to protect index read/write
 __asm volatile("cpsid i" : : : "memory");
 if (rx_buffer.head == rx_buffer.tail) {
 __asm volatile("cpsie i" : : : "memory");
 return false; // Empty
 }
 *data = rx_buffer.buffer[rx_buffer.tail];
 rx_buffer.tail = (rx_buffer.tail + 1) % RING_BUF_SIZE;
 __asm volatile("cpsie i" : : : "memory");
 return true;
}

void USART2_IRQHandler(void) {
 if (*USART2_SR & (1U << 5)) { // RXNE active
 uint8_t data = (uint8_t)(*USART2_DR & 0xFF);
 uint32_t next = (rx_buffer.head + 1) % RING_BUF_SIZE;
 if (next != rx_buffer.tail) { // Not full
 rx_buffer.buffer[rx_buffer.head] = data;
 rx_buffer.head = next;
 }
 }
}

```

---

## Deliverables

---

- [ ] Driver library files `uart.c` and `uart.h`.
  - [ ] Application entry point `main.c` showing the uppercase command echo loop.
  - [ ] Captured USB Logic Analyzer waveform log showing a complete serial transmission packet confirming baud rate and timing calculations.
-



## ? Self-Check Questions

---

1. Show how to calculate the USART\_BRR configuration register value if you are routing communication on USART1 mapped to APB2 clock bus running at 84MHz with a target Baud Rate of 9600.
  2. What are the logical differences between **asynchronous** (e.g., UART) and **synchronous** (e.g., SPI) protocols? Why does UART require strict, predefined clock parameters on both nodes?
  3. What is an **Overflow Error (ORE)**, and what register configurations inside UART cause it? How do you clear a pending ORE?
  4. Why is a Ring Buffer critical for high-speed serial data handling? How does it resolve the latency delta between hardware event triggers and main loop processing?
  5. Under what circumstances will a global index write (e.g., modifying `tail` inside `uart_read_byte`) create a race condition with the ISR pushing to the buffer? How does the critical section protect it?
- 

## Stretch Task (Optional)

---

Extend the serial driver to support **DMA-driven Transmissions**. Configure the DMA peripheral (DMA1 Stream 6 Channel 4) to automatically pull character buffers from memory and stream them to the USART transmitter dynamically without invoking the CPU.

## Motivation Checkpoint

---

UART is the universal debugging port of the electronics industry. From small IoT microcontrollers to giant server racks and automated vehicles, UART serial ports provide the direct diagnostic channel (boot logs, command shells, diagnostic trace data). Mastering reliable, non-blocking serial drivers is the primary step to establishing hardware visibility.

## Trainer Review Rule

---

### • Pass Criteria:

- USART and pins configured purely by register alignments.
- Zero dropped characters during a continuous transmission block of 1000 characters.
- Ring buffer read/write indices safely guarded against race conditions.

### • Fail Criteria:

- Use of polling receiver commands inside ISRs.
- Hard-coded character buffering arrays.

# Weekly Execution Plan: Week 12

---

## Weekly Goal

---

Master synchronous serial protocols, understand the electrical properties of Inter-Integrated Circuit (I2C) and Serial Peripheral Interface (SPI) buses, calculate clock timing registers, and write custom bare-metal master controllers drivers.

## Weekly Outcome

---

By Friday, the student will have written custom **Bare-Metal I2C and SPI Master Drivers** in C on the STM32F4. They will be able to interface physical sensors, execute structural register read/writes, hook up a USB Logic Analyzer to decode active clock/data lines, and complete the Phase 2 Milestone Challenge.

---



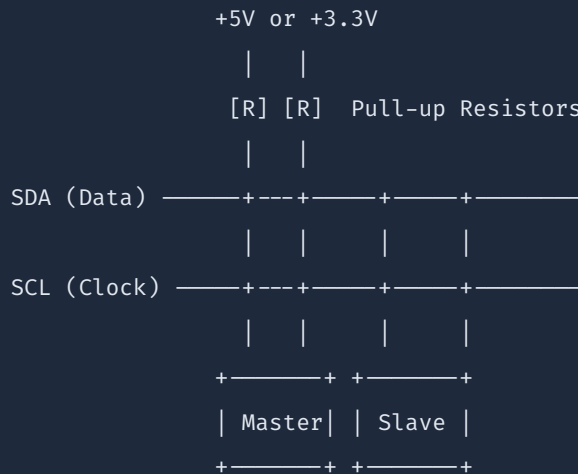
## Session Breakdown (Monday - Friday)

---

### Monday: The I2C Specification & Physical Bus Layer

- **Conceptual Objective:** Master the electrical properties of I2C. Understand:
- **Open-Drain Connections:** Why I2C uses open-drain pins with external pull-up resistors to construct an active-low bus.
- **Start/Stop Conditions:** Start (SDA falling edge while SCL is High), Stop (SDA rising edge while SCL is High).
- **Address Resolution:** 7-bit device addressing with Read/Write bit.
- **Acknowledge (ACK/NACK):** Receiver pulls SDA low to ACK.
- **Practical Activity:**
  - Sketch a physical multi-device I2C bus detailing physical pull-up lines.
  - Draw the timing sequence of a write transaction.
- **Code/Circuit Reference:**

### I2C Multi-Master / Multi-Slave Physical Bus Setup:



## Tuesday: I2C Register Map & Speed Configurations

- **Conceptual Objective:** Study I2C controller registers (CR1, CR2, OAR1, DR, SR1, SR2, CCR, TRISE). Master the calculations required to configure Fast Mode (400kHz) and Standard Mode (100kHz) clock cycles using the Clock Control Register (CCR) and Rise Time Register (TRISE).
- **Practical Activity:**
  - Open the Reference Manual and locate I2C clock configuration sections.
  - Calculate CCR and TRISE parameters for Standard Mode (100kHz) under an APB1 peripheral bus speed of 16MHz.
- **Code/Circuit Reference:**

### I2C Clock Control Register (CCR) Math (Standard Mode):

Target SCL Frequency = 100,000Hz (100kHz) → Period = 10μs

SCL High time = 5μs, SCL Low time = 5μs

Bus Clock = 16,000,000Hz (Period = 0.0625μs = 62.5ns)

CCR Value = SCL High / Bus Period = 5μs / 62.5ns = 5,000ns / 62.5ns = 80

Result CCR Register Value = 80 = 0x50

## Wednesday: Bare-Metal I2C Driver implementation

- **Conceptual Objective:** Learn the exact sequence of register writes to execute I2C commands: Generate START → Write Address → Verify ADDR flag → Write Data → Wait for Byte Transfer Finished (BTF) → Generate STOP.
- **Practical Activity:**
  - Initialize I2C1 (SCL on PB6, SDA on PB7 configured as Alternate Function open-drain).
  - Write robust Master Transmitter and Master Receiver APIs with timeout decrementor loops to prevent lock-ups.

• **Code/Circuit Reference:**

```
#define I2C1_BASE 0x40005400U
#define I2C1_CR1 ((volatile uint32_t*)(I2C1_BASE + 0x00U))
#define I2C1_SR1 ((volatile uint32_t*)(I2C1_BASE + 0x14U))
#define I2C1_DR ((volatile uint32_t*)(I2C1_BASE + 0x10U))

void i2c_start(void) {
 *I2C1_CR1 |= (1U << 8); // Generate START
 // Wait until START condition generated (SB bit 0 in SR1)
 while (!(I2C1_SR1 & (1U << 0)));
}

void i2c_write(uint8_t data) {
 *I2C1_DR = data;
 // Wait until byte transmission complete (TXE bit 7 in SR1)
 while (!(I2C1_SR1 & (1U << 7)));
}
```

## Thursday: The SPI Specification & Register Architecture

- **Conceptual Objective:** Master the Serial Peripheral Interface (SPI) physical layer: full-duplex synchronous communication using four dedicated lines:
- **MOSI:** Master Out Slave In.
- **MISO:** Master In Slave Out.
- **SCK:** Serial Clock.
- **CS/NSS:** Chip Select / Slave Select (active low). Study clock polarity (CPOL) and phase (CPHA) modes.
- **Practical Activity:**
  - Draw timing charts for SPI Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1).
  - Map out the SPI status and control registers.
- **Code/Circuit Reference:**

```
#define SPI1_BASE 0x40013000U
#define SPI1_CR1 ((volatile uint32_t*)(SPI1_BASE + 0x00U))
#define SPI1_SR ((volatile uint32_t*)(SPI1_BASE + 0x08U))
#define SPI1_DR ((volatile uint32_t*)(SPI1_BASE + 0x0CU))

void spi1_init_master(void) {
 // Clock prescaler = 2, Mode 0 (CPOL=0, CPHA=0), Master Configuration
 *SPI1_CR1 = (1U << 2) | (0 << 1) | (0 << 0);
 // Enable SPI
 *SPI1_CR1 |= (1U << 6);
}
```

## Friday: Logical Decoding and Phase 2 Milestone Launch

- **Conceptual Objective:** Understand how to connect a USB Logic Analyzer to active SPI/I2C buses to debug electrical transactions. Start the Phase 2 Milestone Challenge.
- **Practical Activity:**
  - Hook up logic analyzer lines to PB6/PB7. Execute an I2C transaction.
  - Use Sigrok/PulseView to capture, trigger, and decode raw Hex codes on the bus.
- **Code/Circuit Reference:**

```
Capture and decode standard I2C frames using Sigrok CLI
sigrok-cli --driver=fx2lafw --config samplerate=1m --channels D0=SCL,D1=SDA --limit-
samples 100000 -P i2c
```

## Hands-On Assignment: Bare-Metal MPU6050 Accelerometer Interfacing

### Functional Requirements

Create a fully functional, bare-metal application that reads raw data from a physical **MPU6050 6-Axis Accelerometer sensor** via direct register I2C control.

1. Configure PB6 and PB7 as Alternate Function open-drain output pins.
2. Initialize **I2C1** in Standard Master Mode at 100kHz under 16MHz clock conditions. Complete all CCR and TRISE calculations.
3. Write robust register read/write helper functions:
  - `bool i2c_write_register(uint8_t dev_addr, uint8_t reg_addr, uint8_t data)`
  - `bool i2c_read_registers(uint8_t dev_addr, uint8_t reg_addr, uint8_t *buffer, size_t size)`
4. Every polling check on Status Registers (SR1/SR2) must contain a timeout decrementor loop. If a timeout occurs, return an error code instead of locking up the CPU.
5. In `main.c`:
  - Initialize the I2C peripheral.
  - Read the MPU6050 "Who Am I" register (`0x75`). Verify the returned signature

matches `0x68` . - Write to the power management register ( `0x6B` ) to wake the sensor from default sleep mode. - Read raw Accelerometer X/Y/Z data registers sequentially in a loop, converting bytes to integers and streaming them to the UART console.

## Structural Requirements & Code Skeleton

```
i2c.h : ``ifndef I2C_H
```

## define I2C\_H

---

# include

---



# include

---

# include

---

```
void i2c1_init(void); bool i2c_write_register(uint8_t dev_addr, uint8_t reg_addr, uint8_t data); bool
i2c_read_registers(uint8_t dev_addr, uint8_t reg_addr, uint8_t *buffer, size_t size);
```

**endif // I2C\_H**

---

```

`i2c.c` (C-Skeleton):
```c
#include "i2c.h"

#define I2C1_BASE    0x40005400U
typedef struct {
    volatile uint32_t CR1;
    volatile uint32_t CR2;
    volatile uint32_t OAR1;
    volatile uint32_t OAR2;
    volatile uint32_t DR;
    volatile uint32_t SR1;
    volatile uint32_t SR2;
    volatile uint32_t CCR;
    volatile uint32_t TRISE;
} i2c_registers_t;

#define I2C1    ((i2c_registers_t*)I2C1_BASE)

void i2c1_init(void) {
    // Enable GPIO and I2C Clocks
    *((volatile uint32_t*)0x40023830U) |= (1U << 1); // GPIOB
    *((volatile uint32_t*)0x40023840U) |= (1U << 21); // I2C1

    // Configure Pins PB6 (SCL) and PB7 (SDA) Alt Function open-drain pull-ups
    // [GPIO Configuration goes here]

    // Initialize I2C: set input clock speed (16MHz)
    I2C1->CR2 = 16U;

    // Set standard mode clock control (100kHz) → CCR = 80
    I2C1->CCR = 80U;

    // Set maximum rise time (TRISE = Input Clock / 1MHz + 1 = 16 + 1 = 17)
    I2C1->TRISE = 17U;

    // Enable Peripheral
    I2C1->CR1 |= (1U << 0);
}

// Write helper with security check

```

```

bool i2c_write_register(uint8_t dev_addr, uint8_t reg_addr, uint8_t data) {
    uint32_t timeout = 500000;

    // Generate Start
    I2C1→CR1 |= (1U << 8);
    while (!(I2C1→SR1 & (1U << 0))) { if (--timeout == 0) return false; } // SB

    // Write Address (LSB = 0 for write)
    I2C1→DR = (dev_addr << 1);
    while (!(I2C1→SR1 & (1U << 1))) { if (--timeout == 0) return false; } // ADDR
    (void)I2C1→SR2; // Clear ADDR flag by reading SR2

    // Write Register Address
    I2C1→DR = reg_addr;
    while (!(I2C1→SR1 & (1U << 7))) { if (--timeout == 0) return false; } // TXE

    // Write Data
    I2C1→DR = data;
    while (!(I2C1→SR1 & (1U << 7))) { if (--timeout == 0) return false; } // TXE
    while (!(I2C1→SR1 & (1U << 2))) { if (--timeout == 0) return false; } // BTF

    // Generate Stop
    I2C1→CR1 |= (1U << 9);
    return true;
}

```

Deliverables

- [] Master I2C driver files `i2c.c` and `i2c.h`.
 - [] Application entry point `main.c` configuring MPU6050 and streaming accelerometer data to console.
 - [] Capture trace analysis report `i2c_trace.log` or screenshot confirming successful MPU6050 Register Read transactions.
-

Self-Check Questions

1. Why must the I2C physical bus pins be configured as **open-drain** instead of standard push-pull? What electrical issue occurs if two push-pull devices attempt to communicate simultaneously on a shared line?

2. What are the key differences between **Standard Mode (100kHz)** and **Fast Mode (400kHz)** I2C timing configurations inside the Clock Control Register (CCR)?
 3. Why does the I2C master driver clear the ADDR flag inside the Status Register by reading `SR1` followed by reading `SR2` ? What happens if this register access sequence is bypassed?
 4. Explain how **Clock Stretching** enables a slow I2C slave node to hold the master from transmitting data too quickly. Which physical line is held low?
 5. What are the advantages of SPI over I2C in high-speed applications (such as color LCD displays or SD card blocks)? What are the pinout complexity tradeoffs?
-

Stretch Task (Optional)

Extend the I2C master driver to support **Multi-Byte DMA Reads**. Configure the DMA peripheral (DMA1 Stream 0 Channel 1) to automatically read accelerometer raw buffers from the I2C Data Register and transfer them directly into memory, freeing up the processor during data transfers.

Motivation Checkpoint

Synchronous serial protocols (I2C and SPI) are the bedrock of physical sensor integration. Whether you are collecting data from accelerometers, gyroscopes, magnetometers, barometer sensors, or reading high-capacity SD storage blocks, you must implement reliable clock/data bus controls. Direct register-level setups guarantee deep protocol comprehension.

Trainer Review Rule

• Pass Criteria:

- All configurations mapped strictly via raw register offsets.
- Zero reference to ST HAL structures.
- Timing captures on logic analyzer prove stable 100kHz SCL clock timings.
- Data correctly decodes accelerometer movements.

• Fail Criteria:

- Monolithic register configurations that lock up permanently if sensor is disconnected (absence of timeout decrementor loop).

Weekly Execution Plan: Week 13

Weekly Goal

Establish absolute proficiency within the Linux command line environment, master file operations, system permissions, the Filesystem Hierarchy Standard (FHS), process listing, and shell configurations.

Weekly Outcome

By Friday, the student will be comfortable operating entirely without a GUI inside a remote Linux terminal. They will be able to perform structural file surgery, manipulate directories, alter group and owner permissions using octal structures, manage host environment variables, and build core terminal processing operations using pipelined commands.

17 Session Breakdown (Monday - Friday)

Monday: The Terminal Interface & Directory Navigation

- **Conceptual Objective:** Understand virtual terminal interfaces, terminal emulators (Tty), the default shell, absolute vs relative paths, and basic commands. Study directory listing filters and the physical directory mapping layouts.
- **Practical Activity:**
 - Practice advanced terminal navigations (`cd` , `ls` , `pwd` , `mkdir` , `rmdir` , `cp` , `mv` , `rm`).
 - Master recursive operations (`rm -rf`) and custom file details configurations.
- **Code/Circuit Reference:**

```
# Detailed recursive file list sorting by modification time
ls -laRt --color=auto

# Safe copying verifying paths
cp -ivp src_file.c dest_dir/
```

Tuesday: The Filesystem Hierarchy Standard (FHS) & Path Management

- **Conceptual Objective:** Master the Filesystem Hierarchy Standard (FHS) of Linux systems. Comprehend the physical role of every directory: `/bin` , `/sbin` , `/usr/bin` , `/etc`

(configurations), `/var` (variable data/logs), `/dev` (device files), `/sys` & `/proc` (virtual kernel filesystems), and `/opt` (optional third-party software).

- **Practical Activity:**

- Explore virtual file listings inside `/sys/class` and `/proc`.
- Locate where system log files reside (`/var/log/syslog`) and monitor kernel outputs.

- **Code/Circuit Reference:**

```
# Monitor kernel messages in real-time
tail -f /var/log/syslog | grep -i usb

# Check system cpu info virtual file representation
cat /proc/cpuinfo
```

Wednesday: Permissions, Ownership, and Security Controls

- **Conceptual Objective:** Master the Linux file security model: User (u), Group (g), and Others (o). Understand read (r=4), write (w=2), execute (x=1) permissions, directory execute permission (allows entering directory), `chmod` octal structures, and ownership transitions using `chown` and `chgrp`.

- **Practical Activity:**

- Create a script and attempt to execute it. Note permission denied error.
- Apply precise octal permissions (`chmod 755` and `chmod 600`) and verify access boundaries.

- **Code/Circuit Reference:**

```
# Set owner read/write, group read, other none (0640)
chmod 640 secure_config.conf

# Check permissions outputs
ls -l secure_config.conf

# Output matches: -rw-r----- 1 user group ...
```

Thursday: Pipelining, Redirection, and Text Processing Filters

- **Conceptual Objective:** Master the core philosophy of UNIX: combine small, modular utilities using pipes (`|`) and redirection symbols (`>` overwrite, `>>` append, `<` input). Study text processing filters (`grep` , `head` , `tail` , `wc` , `sort` , `uniq`).

- **Practical Activity:**

- Build pipeline scripts to parse large system logs.
- Extract, sort, and isolate specific network log statistics using text processing filters.

- **Code/Circuit Reference:**


```
# Isolate top 10 most frequent warning messages in syslog
cat /var/log/syslog | grep -i warning | awk '{print $5, $6}' | sort | uniq -c | sort -
nr | head -n 10
```

Friday: Environment Variables, Processes, and System Controls

- **Conceptual Objective:** Understand that shells track configurations via Environment Variables (`env` , `printenv`). Study process control signals: how to list active processes (`ps aux` , `htop`), check memory states (`free -h`), and terminate processes safely using signals (`kill` , `killall` , `pkill`).
- **Practical Activity:**
 - Configure custom environment paths dynamically.
 - Start background sleep tasks, locate their Process IDs (PID), and terminate them using different signals (`SIGTERM` 15 vs `SIGKILL` 9).
- **Code/Circuit Reference:**

```
# Start background process
sleep 300 &
[1] 12345 # returns PID

# Check state and terminate
ps -ef | grep "sleep"
kill -9 12345
```



Hands-On Assignment: Automated System Analytics Dashboard

Functional Requirements

Create a fully functional shell analysis pipeline that generates a system diagnostics status dashboard inside a remote workspace directory. 1. The analysis tool must be a single file `sys_dashboard.sh` . 2. Ensure the script has proper execution permissions and handles system errors safely. 3. The dashboard must programmatically calculate and output: - **Host Name & OS Version**. - **Total & Free RAM** in human-readable megabytes. - **CPU Usage Statistics** (extract active load percentage). - **Active USB Serial Devices** (scan for plugged-in FTDI, STLINK, or serial adapters). - **Top 5 Memory-Consuming Processes** active on the host. 4. Output the results cleanly to a temporary log file `/home/$USER/workspace/sys_status.log` and display the dashboard on the terminal.

Structural Requirements & Code Skeleton

sys_dashboard.sh (Shell Skeleton):

```
#!/usr/bin/env bash
# Professional System Dashboard Utility

set -euo pipefail # Fail immediately on error or unset variables

LOG_FILE="/home/$USER/sys_status.log"

echo "===== " | tee "$LOG_FILE"
echo "      SYSTEM DIAGNOSTICS DASHBOARD      " | tee -a "$LOG_FILE"
echo "===== " | tee -a "$LOG_FILE"

# 1. OS & Host Info
echo "Host: $(hostname)" | tee -a "$LOG_FILE"
echo "OS: $(grep 'PRETTY_NAME' /etc/os-release | cut -d'=' -f2 | tr -d '\'"'"') | tee -a
"$LOG_FILE"

# 2. RAM calculations
echo "----- Memory Info -----" | tee -a "$LOG_FILE"
free -h | awk '/Mem:/ {print "Total RAM: " $2 "\nFree RAM:  " $4}' | tee -a "$LOG_FILE"

# 3. CPU Load
echo "----- CPU Load -----" | tee -a "$LOG_FILE"
cpu_load=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 + $4}')
echo "Active CPU Load: ${cpu_load}%" | tee -a "$LOG_FILE"

# 4. USB Serial Adapter Scan
echo "----- USB Serial Devices -----" | tee -a "$LOG_FILE"
usb_adapters=$(lsusb | grep -iE 'ftdi|stlink|prolific|ch340|cp210' || echo "No devices
found")
echo "$usb_adapters" | tee -a "$LOG_FILE"

# 5. Top Processes
echo "----- Top 5 Memory Processes -----" | tee -a "$LOG_FILE"
ps aux --sort=-%mem | awk 'NR<=6 {print $1, $2, $4, $11}' | tee -a "$LOG_FILE"

echo "===== " | tee -a "$LOG_FILE"
echo "Dashboard written to $LOG_FILE"
```

Deliverables

- [] Executable shell script `sys_dashboard.sh`.
 - [] Generated log output file `sys_status.log` detailing system parameters.
 - [] Complete documentation outlining your custom environment variable mappings configured inside your host profile shell.
-

? Self-Check Questions

1. What is the fundamental difference between the `/bin` and `/sbin` directories inside Linux? Why are specific commands restricted to `/sbin`?
 2. What are virtual filesystems? Explain how `/proc` and `/sys` allow users to communicate directly with the Linux kernel using standard text utilities.
 3. If a file has permissions `rwxr-xr-x` (octal 755), what does that mean? How do you modify it so that only the owner can read/write it, and all others are blocked?
 4. What is the difference between sending a `SIGTERM` (15) and a `SIGKILL` (9) signal to an active process? Why is `SIGKILL` dangerous to use as a first resort?
 5. How does command-line piping (`|`) work inside the kernel? What is the role of standard input (stdin), standard output (stdout), and standard error (stderr)?
-

Stretch Task (Optional)

Extend your system dashboard script to scan network configurations: detect active network adapters, print the local IP address, check internet connectivity via ping packets, and record statistics of active network sockets using `netstat` or `ss`.

Motivation Checkpoint

A high-performance embedded systems developer does not operate inside graphic panels or file windows. When debugging a headless Raspberry Pi gateway, configuring automated toolchain pipelines, or compiling custom kernels on remote cloud servers, the terminal command line is your primary weapon. Master the terminal to master the OS.

Trainer Review Rule

• Pass Criteria:

- Script compiles and executes cleanly with `set -euo pipefail` enabled.

- Commands use clean Unix pipelines.
- Output files are formatted and contain correct system values.
- **Fail Criteria:**
- Direct execution errors or unhandled empty commands.
- Hardcoded user directories (must use `$USER` or `$HOME` variables).

Weekly Execution Plan: Week 14

Weekly Goal

Master compiler toolchain construction, write advanced Makefiles containing automatic dependency tracking, understand the anatomy of Executable and Linkable Format (ELF) binary files, and dissect symbols using binary utilities.

Weekly Outcome

By Friday, the student will have built a complex, multi-file C project coordinated entirely by a custom **Incremental Makefile** containing variable mappings and pattern rules. They will be able to dissect compiled ELF outputs, inspect symbol tables, measure section sizes, and decode compiler optimizations.

17 Session Breakdown (Monday - Friday)

Monday: The GCC Compilation Pipeline & Core Flags

- **Conceptual Objective:** Master the flags and stages of the GNU Compiler Collection (GCC). Study optimization levels (`-O0` , `-O1` , `-O2` , `-O3` , `-Os`), debug options (`-g`), warnings (`-Wall` , `-Wextra` , `-pedantic`), and code safety flags (`-Werror`).
- **Practical Activity:**
 - Compile a multi-file project manually using individual GCC calls.
 - Experiment with optimizing code sizes using different flags and document results.
- **Code/Circuit Reference:**

```
# Compile and link with strict diagnostics
gcc -Wall -Wextra -Werror -pedantic -O2 -g -Iinclude/ -c src/helper.c -o build/helper.o
```

Tuesday: Writing Modular Makefiles

- **Conceptual Objective:** Study GNU Make rules: Targets, Dependencies, and Commands. Understand how Make determines incremental rebuilds based on file modification timestamps. Master variables, automatic variables (`$@` target, `$<` first dependency, `$$` all dependencies), and clean commands.
- **Practical Activity:**

- Write a basic Makefile.
- Implement clean commands to wipe objects and final executables safely.
- **Code/Circuit Reference:**

```
# Simple Makefile structure
CC = gcc
CFLAGS = -Wall -Wextra -O2 -Iinclude

build/app: build/main.o build/helper.o
    $(CC) $(CFLAGS) $^ -o $@

build/%.o: src/%.c
    mkdir -p build
    $(CC) $(CFLAGS) -c $< -o $@
```

Wednesday: Makefile Auto-Dependency Generation

- **Conceptual Objective:** Understand a classic Makefile bug: if a header file (`.h`) is modified, the compiler should rebuild any C files that include it, but standard rules do not monitor headers. Study GCC's preprocessor options (`-MMD -MP`) to automatically output dependency files (`.d`) and integrate them into Make.
- **Practical Activity:**
 - Modify your Makefile to include automatic dependency tracking.
 - Test modifying a header and verify Make triggers incremental recompilation of dependant source files.
- **Code/Circuit Reference:**

```
# Makefile with auto-dependency tracking
SRCS = $(wildcard src/*.c)
OBS = $(SRCS:src/%.c=build/%.o)
DEPS = $(OBS:.o=.d)

-include $(DEPS)

build/%.o: src/%.c
    mkdir -p build
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@
```

Thursday: Executable and Linkable Format (ELF) Anatomy

- **Conceptual Objective:** Dissect the Executable and Linkable Format (ELF) specification—the universal standard for executable binaries inside Linux and embedded systems. Understand:
- **ELF Header:** Identifies architecture, entry point address, and format.
- **Program Header Table:** Specifies how segments map to system memory (loadable blocks).
- **Section Header Table:** Defines locations of `.text`, `.data`, `.bss`, `.rodata`, `.symtab` (Symbol Table), and `.strtab` (String Table).
- **Practical Activity:**
 - Compile a C binary.
 - Run system tools (`readelf`, `file`) to extract format details and segment alignments.
- **Code/Circuit Reference:**

```
# Read ELF header of compiled binary
readelf -h build/app

# List all section headers and virtual addresses
readelf -S build/app
```

Friday: Dissecting Symbols with Binary Utilities

- **Conceptual Objective:** Master the binary utilities suite (`nm`, `objdump`, `size`, `strip`) to inspect compiled binary details. Learn how to locate symbol types (Global vs Local), verify section size footprints, and disassemble machine instructions.
- **Practical Activity:**
 - Run `size` to measure memory usage.
 - Run `nm` to list all functions and global variables.
 - Disassemble specific functions using `objdump -d` and trace assembly instructions.
- **Code/Circuit Reference:**

```
# List footprints of sections in human-readable bytes
size build/app

# Disassemble main function specifically
objdump -d build/app | grep -A 20 "<main>:"
```

Hands-On Assignment: Automated Build and Symbol Dissecting System

Functional Requirements

Create a fully modular, multi-file C math library coordinated by a custom, production-grade Makefile, and build an automated shell pipeline to analyze symbol tables. 1. The project must have clean structure: `src/main.c`, `src/math.c`, `include/math.h`, and a root `Makefile`. 2. The Makefile must:

- Use compile flags `-Wall -Wextra -Werror -O2 -g`.
- Maintain intermediate objects (`.o`) and dependency tracking files (`.d`) inside a dedicated `build/` directory.
- Re-compile only modified source and header dependencies.

3. Write a shell script named `dissect_symbols.sh` that compiles the project, then programmatically parses the final ELF executable using binary tools to output a report `symbol_report.txt` containing:

- Total text segment footprint.
- List of all global symbols defined in `src/math.c` and their virtual memory offsets.
- Verification that debugging symbols exist in the ELF.

Structural Requirements & Code Skeleton

`Makefile`:


```

CC = gcc
CFLAGS = -Wall -Wextra -Werror -O2 -g -Iinclude
SRC_DIR = src
BUILD_DIR = build
TARGET = $(BUILD_DIR)/app

SRCS = $(wildcard $(SRC_DIR)/*.c)
OBJS = $(SRCS:$(SRC_DIR)/%.c=$(BUILD_DIR)/%.o)
DEPS = $(OBJS:.o=.d)

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJS)
    @mkdir -p $(BUILD_DIR)
    $(CC) $(CFLAGS) $^ -o $@

-include $(DEPS)

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.c
    @mkdir -p $(BUILD_DIR)
    $(CC) $(CFLAGS) -MMD -MP -c $< -o $@

clean:
    rm -rf $(BUILD_DIR)

```

`dissect_symbols.sh` (Shell Skeleton):

```
#!/usr/bin/env bash
set -euo pipefail

# Compile
make clean
make

REPORT="symbol_report.txt"

echo "=== ELF Symbol Dissection Report ===" > "$REPORT"
echo "Binary: build/app" >> "$REPORT"
echo "Date: $(date)" >> "$REPORT"
echo "_____ " >> "$REPORT"

# 1. Section Footprints
echo "Section Sizes:" >> "$REPORT"
size build/app >> "$REPORT"
echo "_____ " >> "$REPORT"

# 2. Extract math helper functions offsets
echo "Math Library Symbols (Global Functions):" >> "$REPORT"
nm build/app | grep -E ' T | t ' | grep -iE 'add|sub|mul|div|math' >> "$REPORT" || echo
"No matching symbols" >> "$REPORT"
echo "_____ " >> "$REPORT"

# 3. Check for debug symbols presence
if readelf -S build/app | grep -q "debug_info"; then
    echo "Debug Symbols status: PRESENT" >> "$REPORT"
else
    echo "Debug Symbols status: MISSING" >> "$REPORT"
fi

cat "$REPORT"
```



Deliverables

- [] Multi-file C library workspace (`main.c` , `math.c` , `math.h`).
- [] Automated `Makefile` with pattern rules and auto-dependency generation.
- [] Automated Shell Analyzer `dissect_symbols.sh` and output report `symbol_report.txt` .

? Self-Check Questions

1. Why does the compiler compile each C file into an individual object file (`.o`) before linking? What is the function of the linker in this pipeline?
 2. What are Makefile **automatic variables**? State the exact meanings of `$@` , `$<` , and `^` .
 3. How does automatic dependency tracking (`-MMD -MP`) prevent stale builds when header files are modified? Trace how Make utilizes the generated `.d` files.
 4. What is the fundamental difference between the **Program Header Table** and the **Section Header Table** inside an ELF file? Which table is used by the operating system loader?
 5. How does the `strip` command affect executable file sizes? Why should you never strip an ELF binary prior to debugging inside `gdb` ?
-

Stretch Task (Optional)

Extend the Makefile to compile the C library into a **Dynamic Shared Library** (`.so` file) instead of a static binary, and verify dynamic linkages using the `ldd` utility.

Motivation Checkpoint

Modern build systems coordinate thousands of source files. When you write custom code on an embedded board, you must understand exactly how compiled symbols are routed to target Flash/ SRAM, how compiler optimization structures alter timing, and how to verify symbol placements. Understanding toolchains is the baseline differentiator of systems developers.

Trainer Review Rule

• Pass Criteria:

- Makefile executes incremental compilation flawlessly (modifying `math.h` triggers recomp, running double makes does nothing).
- Symbol report accurately isolates compiler function offsets.
- Zero warnings compiled with `-Wall -Wextra -Werror` .

• Fail Criteria:

- Makefile missing automatic variable optimization rules.
- Manual recompiles required after header changes.

Weekly Execution Plan: Week 15

Weekly Goal

Master low-level Linux file system operations, understand File Descriptors (FD), differentiate between standard buffered library I/O and low-level unbuffered system call I/O, configure custom devices via `ioctl`, and implement memory-mapped I/O using `mmap`.

Weekly Outcome

By Friday, the student will have written a **Low-Level System I/O Engine** in C. They will understand file permission flags, read/write error boundary handling, how control registers are queried via `ioctl`, and how to read/write large files highly efficiently using virtual memory page mapping via `mmap`.

Session Breakdown (Monday - Friday)

Monday: Standard Buffered vs Low-Level Unbuffered I/O

- **Conceptual Objective:** Understand how the Linux Virtual File System (VFS) handles I/O. Study the fundamental architectural differences between standard C library buffered calls (`fopen`, `fread`, `fwrite`, `printf` - which utilize user-space buffer caches) and low-level kernel system calls (`open`, `read`, `write`, `close` - which enter the kernel directly).
- **Practical Activity:**
 - Write two applications: one using `fwrite` and one using `write`.
 - Use the system tool `strace` to trace the system calls triggered by both binaries. Notice how many standard library writes are cached before triggering a physical system call.
- **Code/Circuit Reference:**

```
# Trace system calls of standard print program
gcc -O2 src/stdio_demo.c -o build/stdio_demo
strace ./build/stdio_demo
```

Tuesday: File Descriptors & System Calls (`open`, `read`, `write`, `close`)

- **Conceptual Objective:** Master the Linux File Descriptor (FD) concept: non-negative integers representing open system files inside a process's FD table. Study standard file descriptors: `0` (`stdin`), `1` (`stdout`), `2` (`stderr`). Understand call flags (`O_RDONLY`, `O_WRONLY`, `O_CREAT`, `O_APPEND`) and octal permission masks.

- **Practical Activity:**

- Write a C program to copy a file using `open`, `read`, `write`, and `close`.
- Implement robust error checking for every system call (e.g., return value is `-1` and read `errno` from `<errno.h>`).

- **Code/Circuit Reference:**

```
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>

void safe_write_string(const char *path, const char *text) {
    // Open for writing, create if not exist, append, set user rw permissions (0644)
    int fd = open(path, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (fd < 0) {
        perror("Error opening file");
        return;
    }

    size_t len = 0;
    while (text[len]) len++;

    ssize_t bytes_written = write(fd, text, len);
    if (bytes_written < 0) {
        perror("Error writing to file");
    }

    close(fd);
}
```

Wednesday: Device Control Operations via `ioctl`

- **Conceptual Objective:** Learn about the input/output control system call (`ioctl`)—the universal gateway to interface with custom hardware devices and kernel drivers from user space. Understand how `ioctl` acts as a multi-purpose command interface to set hardware parameters (baud rates, frequencies, modes) that do not fit standard read/write formats.

- **Practical Activity:**

- Write a program to query the size of active terminal windows dynamically using standard system TTY `ioctl` commands.
- Log row and column count outputs.

- **Code/Circuit Reference:**

```

#include <sys/ioctl.h>
#include <unistd.h>
#include <stdio.h>

void print_terminal_size(void) {
    struct winsize w;
    // STDOUT_FILENO = 1
    if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &w) == 0) {
        printf("Terminal Rows: %d, Columns: %d\n", w.ws_row, w.ws_col);
    } else {
        perror("ioctl error");
    }
}

```

Thursday: High-Performance Memory Mapping (`mmap`)

- **Conceptual Objective:** Master the memory-mapping system call (`mmap`). Differentiate between standard file reading (which copies data from disk -> kernel page cache -> user buffer) and `mmap` (which maps file segments directly into the process's virtual address space, eliminating user-space copying overhead).
- **Practical Activity:**
 - Write a program that opens a large text file, maps it to a pointer using `mmap` , and modifies the content directly by modifying pointer elements.
 - Verify that modifications are synced back to disk.
- **Code/Circuit Reference:**

```

#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

void map_file_to_pointer(const char *path) {
    int fd = open(path, O_RDWR);
    struct stat sb;
    fstat(fd, &sb); // Get file size

    // Map file to virtual memory
    char *addr = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) {
        perror("mmap error");
        close(fd);
        return;
    }

    // Read and modify direct memory offset
    printf("File contents: %s\n", addr);
    addr[0] = 'H'; // Changes the first character of the physical file!

    // Clean up
    munmap(addr, sb.st_size);
    close(fd);
}

```

Friday: Advanced File Multiplexing Concepts (select, poll)

- **Conceptual Objective:** Understand what happens when you attempt to read from a slow file interface (like a serial port or network pipe). The read blocks, stopping execution. Study the concepts of non-blocking I/O (`O_NONBLOCK`) and file multiplexing (`select` , `poll`) to monitor multiple file descriptors concurrently.
 - **Practical Activity:**
 - Read the `poll()` system call specifications.
 - Draw the multi-stream polling workflow architecture diagram.
-

Hands-On Assignment: Custom Memory-Mapped DB Processor

Functional Requirements

Create a high-performance database processor application that processes records from a custom binary file using low-level unbuffered system calls and memory-mapped virtual regions. 1. Define a database record structure containing a 32-bit ID, an array representing a 32-byte string value, and a 32-bit floating-point numeric value. The structure must be explicitly packed. 2. The application must support command-line arguments: - `-w` : Create the file database and write 10 dummy records using unbuffered `write()` system calls. - `-r` : Read the file database using memory-mapped I/O (`mmap()`), print out all active records, and modify the floating-point values of specific record IDs directly via memory pointer modifications. 3. Every single system call (`open`, `read`, `write`, `mmap`, `munmap`, `fstat`) must contain robust error validation. If a failure occurs, the program must output the exact error description using `perror()` or `strerror(errno)` and exit with a clean failure status code. 4. Provide a Makefile that builds the program cleanly.

Structural Requirements & Code Skeleton

`db_processor.c` (C-Skeleton):


```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <errno.h>

#define DB_FILE "records.db"
#define RECORD_COUNT 10

typedef struct __attribute__((packed)) {
    uint32_t id;
    char name[32];
    float value;
} db_record_t;

void write_db(void) {
    int fd = open(DB_FILE, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Error creating DB file");
        exit(EXIT_FAILURE);
    }

    db_record_t record;
    for (uint32_t i = 0; i < RECORD_COUNT; i++) {
        record.id = i;
        snprintf(record.name, sizeof(record.name), "Record_Node_%u", i);
        record.value = (float)i * 10.5f;

        ssize_t bytes = write(fd, &record, sizeof(db_record_t));
        if (bytes < 0) {
            perror("Error writing record");
            close(fd);
            exit(EXIT_FAILURE);
        }
    }
    close(fd);
    printf("Successfully wrote %d records to %s\n", RECORD_COUNT, DB_FILE);
}

```

```

}

void process_db_mmap(void) {
    int fd = open(DB_FILE, O_RDWR);
    if (fd < 0) {
        perror("Error opening DB file");
        exit(EXIT_FAILURE);
    }

    struct stat sb;
    if (fstat(fd, &sb) < 0) {
        perror("Error statting file");
        close(fd);
        exit(EXIT_FAILURE);
    }

    // Map the file
    db_record_t *records = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
    if (records == MAP_FAILED) {
        perror("Error mmappping DB");
        close(fd);
        exit(EXIT_FAILURE);
    }

    size_t count = sb.st_size / sizeof(db_record_t);
    printf("Reading %zu records from memory map:\n", count);
    for (size_t i = 0; i < count; i++) {
        printf("ID: %u, Name: %s, Value: %.2f\n", records[i].id, records[i].name,
records[i].value);

        // Modify values directly in mapped space (will sync back to disk)
        if (records[i].id == 5) {
            records[i].value = 999.99f;
            printf(" → Updated record 5 value to 999.99\n");
        }
    }

    // Sync modifications
    if (msync(records, sb.st_size, MS_SYNC) < 0) {
        perror("msync failed");
    }
}

```

```
// Clean up
munmap(records, sb.st_size);
close(fd);
}
```

Deliverables

- [] Database processor C source file `db_processor.c` .
 - [] Compilation Makefile.
 - [] Extracted system call log file `strace_output.txt` captured during runs of `-w` and `-r` runs.
-

? Self-Check Questions

1. Why does using standard `fwrite()` result in fewer physical write system calls than direct `write()` loops? Differentiate user-space cache buffering from kernel-space buffering.
 2. What are the file permission octal symbols? Explain what access limitations are applied by permission mask `0644` vs `0600` .
 3. How does `fstat()` retrieve file data? Which structure fields track total file size in bytes?
 4. What is the fundamental difference between memory mapping with `MAP_SHARED` versus `MAP_PRIVATE` flags? Which flag must be used to sync modifications back to physical disk files?
 5. How does `ioctl()` allow developers to build configurations that are not supported by standard read/write APIs? Give a real-world driver context example.
-

Stretch Task (Optional)

Extend the database processor to support **Multi-process Locking**. Use the `flock` or `fcntl` system calls to lock the database file during reads/writes to prevent multiple concurrent processes from corrupting the memory-mapped virtual region.

Motivation Checkpoint

In production systems, processing millions of telemetry packets or managing local database logs requires speed and absolute execution safety. Standard high-level I/O streams are too slow and mask errors behind caching layers. Mastering direct system calls, `ioctl` configurations, and `mmap` page caches is standard architecture requirement for elite systems engineers.



Trainer Review Rule

- **Pass Criteria:**

- Code compiles without warnings with `-Wall -Wextra -Werror`.
- All system call returns are verified for errors (`< 0`).
- Binary processes mmap mappings and outputs correctly.
- `strace_output.txt` shows correct dynamic allocations and system calls.

- **Fail Criteria:**

- Monolithic logic that assumes system calls never fail.
- Missing file descriptor closings.

Weekly Execution Plan: Week 16

Weekly Goal

Master Linux multi-processing architectures, understand process scheduling, learn process cloning (`fork`) and executable replacements (`exec`), synchronize termination (`wait` , `waitpid`), and build secure Inter-Process Communication (IPC) pipelines using Pipes, FIFOs, and Shared Memory.

Weekly Outcome

By Friday, the student will have written a **Multi-Process IPC Pipeline** in C. They will understand child cloning, child process reaping, how to prevent zombie/orphan processes, how to redirect input/output streams dynamically, and how to transfer high-rate data between independent processes using Shared Memory.

17 Session Breakdown (Monday - Friday)

Monday: Process Creation via `fork` & Process Lifecycles

- **Conceptual Objective:** Understand that the kernel tracks every running program as a process containing a unique Process ID (PID). Master `fork()` —the system call that clones the calling process, creating an identical child process. Learn about the dual-return value of `fork()` (returns `0` inside the child, returns the child's PID inside the parent).
- **Practical Activity:**
 - Write a program that forks multiple child processes.
 - Print PIDs and Parent PIDs (`getpid()` , `getppid()`) to map out parent-child family trees.
- **Code/Circuit Reference:**

```

#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>

void fork_demo(void) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return;
    }

    if (pid == 0) {
        // Child Process
        printf("Child Process: PID=%d, Parent PID=%d\n", getpid(), getppid());
    } else {
        // Parent Process
        printf("Parent Process: PID=%d, Spawned Child PID=%d\n", getpid(), pid);
    }
}

```

Tuesday: Executive Replacements (`exec`) & Process Reaping

- **Conceptual Objective:** Master the `exec` family of system calls (`execl` , `execv` , `execp`). Understand how `exec` completely overwrites the calling process's virtual memory, stack, and text segment with a new target binary. Learn about child process status reaping (`wait` , `waitpid`) to prevent the creation of **Zombie Processes** (terminated children whose exit status remains un-read in kernel tables) and **Orphan Processes** (children running after parent terminates).
- **Practical Activity:**
 - Write a parent program that forks a child.
 - The child replaces itself with a system command (like `ls -la`) using `execl` .
 - The parent blocks on `waitpid()` and prints the child's exit status.
- **Code/Circuit Reference:**

```

#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>

void exec_reap_demo(void) {
    pid_t pid = fork();
    if (pid == 0) {
        // Overwrite child with target command
        execl("/bin/ls", "ls", "-la", NULL);
        perror("exec failed"); // Only prints if exec fails!
        _exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0); // Wait for child to exit
        if (WIFEXITED(status)) {
            printf("Child exited with code: %d\n", WEXITSTATUS(status));
        }
    }
}

```

Wednesday: Unnamed Pipes & I/O Redirections (`dup2`)

- **Conceptual Objective:** Study the classic Inter-Process Communication mechanism: the **Pipe** (a unidirectional channel in kernel memory with a read FD and a write FD). Understand how to redirect standard outputs of one process to the standard input of another using `dup2()`.
- **Practical Activity:**
 - Write a program that forks a child and pipes data from the parent to the child.
 - Recreate the command line shell pipe command `ls | wc -l` in C using `pipe()`, `fork()`, and `dup2()`.
- **Code/Circuit Reference:**

```

#include <unistd.h>
#include <stdio.h>

void pipe_demo(void) {
    int pipefd[2];
    pipe(pipefd); // pipefd[0] is read, pipefd[1] is write

    if (fork() == 0) {
        // Child: Read from pipe
        close(pipefd[1]); // Close unused write end
        char buf[32];
        read(pipefd[0], buf, sizeof(buf));
        printf("Child received: %s\n", buf);
        close(pipefd[0]);
    } else {
        // Parent: Write to pipe
        close(pipefd[0]); // Close unused read end
        write(pipefd[1], "Hello Child", 11);
        close(pipefd[1]);
    }
}

```

Thursday: Named Pipes (FIFOs)

- **Conceptual Objective:** Understand that standard unnamed pipes require a common parent-child ancestor process relationship because file descriptors must be shared. Study Named Pipes (FIFOs) —virtual files residing inside the filesystem directory that allow completely independent, unrelated processes to communicate.
- **Practical Activity:**
 - Create a FIFO file on disk using `mkfifo()`.
 - Write two separate independent programs: `writer.c` (opens FIFO in write-only mode) and `reader.c` (opens FIFO in read-only mode).
 - Transfer text blocks between the two independent shell instances.
- **Code/Circuit Reference:**


```
# Create FIFO manually in terminal
mkfifo /tmp/my_fifo

# Terminal 1: write to it
echo "Hello from terminal 1" > /tmp/my_fifo

# Terminal 2: read from it (blocks until terminal 1 writes)
cat /tmp/my_fifo
```

Friday: High-Performance Shared Memory (SHM)

- **Conceptual Objective:** Master POSIX Shared Memory (`shm_open` , `ftruncate` , `mmap`).

Understand that Shared Memory is the fastest possible IPC mechanism because the kernel maps a shared physical memory frame into the virtual space of both processes, allowing direct pointer-to-pointer data writes without context switches or kernel system copies.

- **Practical Activity:**

- Write parent-child programs that initialize a shared memory segment.
- Read and write structures directly via the mapped pointers.

- **Code/Circuit Reference:**

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#define SHM_NAME "/my_shared_memory"
#define SHM_SIZE 4096

void create_shm(void) {
    // Open shared memory block
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, SHM_SIZE);

    // Map to pointer
    void *ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    // ptr can now be read/written by independent processes!
}
```

Hands-On Assignment: Multi-Process Real-Time IPC

Gateway

Functional Requirements

Create a multi-process gateway architecture where a high-priority Sensor Simulator process communicates with a Data Logging process using Shared Memory and Named Semaphores.

1. The project must build two independent binaries: `sensor_sim` and `data_logger`.
2. `sensor_sim` (cloned child or independent process) must:
 - Create or open a POSIX Shared Memory segment named `/sensor_shm`.
 - Initialize and map a packed structure containing: sensor ID, timestamp, and raw data values.
 - Run a loop updating data values every 100ms.
 - Use a **POSIX Named Semaphore** (`sem_open`) to synchronize writes, preventing the logger from reading incomplete structures.
3. `data_logger` must:
 - Open and map `/sensor_shm`.
 - Read the structure safely when the semaphore signals that data is ready.
 - Process the values and log them to a file `telemetry.csv` using unbuffered system calls.
4. If a termination signal (`SIGINT` or `Ctrl-C`) is caught, both processes must shut down cleanly, unmapping memory segments (`munmap`), deleting shared segments (`shm_unlink`), and closing semaphores (`sem_close`, `sem_unlink`) to prevent memory leaks in system tables.

Structural Requirements & Code Skeleton

`shared_def.h`:

```
#ifndef SHARED_DEF_H
#define SHARED_DEF_H

#include <stdint.h>

#define SHM_NAME "/sensor_shm"
#define SEM_NAME "/sensor_sem"

typedef struct __attribute__((packed)) {
    uint32_t sensor_id;
    uint32_t timestamp;
    float reading;
    uint8_t status_flag;
} sensor_data_t;

#endif // SHARED_DEF_H
```

`sensor_sim.c` (C-Skeleton):

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <semaphore.h>
#include <signal.h>
#include "shared_def.h"

int shm_fd = -1;
sensor_data_t *shared_data = NULL;
sem_t *sem = NULL;

void cleanup(int sig) {
    (void)sig;
    if (shared_data != NULL) munmap(shared_data, sizeof(sensor_data_t));
    if (shm_fd != -1) close(shm_fd);
    if (sem != NULL) sem_close(sem);
    shm_unlink(SHM_NAME);
    sem_unlink(SEM_NAME);
    printf("\nCleaned up shared memory and exited.\n");
    exit(0);
}

int main(void) {
    signal(SIGINT, cleanup);

    // 1. Create Shared Memory
    shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(sensor_data_t));

    // 2. Map Shared Memory
    shared_data = mmap(NULL, sizeof(sensor_data_t), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);

    // 3. Create Named Semaphore (initial value = 1)
    sem = sem_open(SEM_NAME, O_CREAT, 0666, 1);

    uint32_t tick = 0;
    while (1) {
        sem_wait(sem); // Lock segment

```

```

    // Update shared structure
    shared_data->sensor_id = 101;
    shared_data->timestamp = tick++;
    shared_data->reading = 22.4f + ((float)(rand() % 100) / 10.0f);
    shared_data->status_flag = 0xAA;

    sem_post(sem); // Unlock segment
    usleep(100000); // Wait 100ms
}
return 0;
}

```

Deliverables

- [] Common header file `shared_def.h`.
 - [] Independent source files `sensor_sim.c` and `data_logger.c`.
 - [] Compilation Makefile building both binaries cleanly.
 - [] Telemetry outputs log `telemetry.csv` demonstrating successful IPC writes.
-

? Self-Check Questions

1. Why does a cloned child process created by `fork()` inherit open file descriptors? What is the impact if a parent closes a file descriptor inside its own thread, and how does it affect the child?
 2. What is a **Zombie Process**? What is its footprint inside the operating system, and how does reaping with `waitpid()` resolve the issue?
 3. How does a named FIFO differ from an unnamed pipe? Why can unrelated shell processes communicate via a FIFO but not via a pipe?
 4. Why is Shared Memory the absolute fastest possible Inter-Process Communication mechanism? What is the role of Semaphores when accessing shared segments?
 5. What are signal handlers? Explain how catching `SIGINT` enables dynamic cleanup of persistent IPC resources on disk.
-



Stretch Task (Optional)

Extend the database logger to handle multiple simulation clients dynamically: manage an array of semaphores to monitor updates from up to 5 concurrent sensor simulators, logging all data structures into a unified telemetry file.



Motivation Checkpoint

In automotive ECUs, flight controllers, or complex Linux gateways, tasks are divided across separate execution units to secure isolation: if a UI renderer crashes, the critical telemetry capture process must continue unaffected. Separating tasks into processes protects the system, but requires mastery of IPC to sync communication cleanly.



Trainer Review Rule

- **Pass Criteria:**

- Independent processes compile cleanly with zero warnings.
- POSIX shared segments are correctly initialized, synced, and unlinked.
- Shared memory structures are protected using Named Semaphores to resolve race conditions.

- **Fail Criteria:**

- Memory mapping bypasses semaphore protection.
- Persistent IPC blocks are orphaned on disk after exit.

Weekly Execution Plan: Week 17

Weekly Goal

Master multi-threaded concurrent programming using POSIX Threads (`pthread`), configure synchronization primitives (Mutexes, Semaphores, Condition Variables), analyze race conditions, and write highly efficient thread-safe data structures.

Weekly Outcome

By Friday, the student will have designed and implemented a **Thread-Safe Concurrent FIFO Queue** (Producer-Consumer Queue) in C. They will understand the core differences between processes and threads, thread scheduling, resource sharing, how to prevent deadlocks, and how to coordinate thread tasks using condition variables.



Session Breakdown (Monday - Friday)

Monday: Processes vs Threads & POSIX Thread Creation

- **Conceptual Objective:** Master the fundamental differences between a process and a thread. A process owns resources (has its own private virtual address space, file descriptor tables, environment). A thread is a lightweight unit of execution running inside a process; all threads of a process share the same text, data, heap, and file descriptors, but possess their own private **stack and register context**.
- **Practical Activity:**
 - Learn how to create threads using `pthread_create()` and synchronize termination using `pthread_join()`.
 - Compile multithreaded binaries by linking the POSIX library (`-lpthread`).
- **Code/Circuit Reference:**

```

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void* thread_worker(void *arg) {
    int id = *(int*)arg;
    printf("Thread %d running ...\n", id);
    return NULL;
}

void spawn_thread(void) {
    pthread_t thread;
    int id = 1;
    pthread_create(&thread, NULL, thread_worker, &id);
    pthread_join(thread, NULL); // Sync wait for completion
}

```

Tuesday: Shared Memory Race Conditions & Mutex Protection

- **Conceptual Objective:** Understand that because threads share the same address space, concurrent updates to global or heap variables will create devastating race conditions. Study the Mutual Exclusion (`pthread_mutex_t`) primitive. Learn the lock-and-unlock pattern to enforce atomic access inside critical sections.
- **Practical Activity:**
 - Write a program where multiple threads increment a global variable simultaneously without protection. Observe the final value is incorrect (data corruption due to interleaving).
 - Implement a `pthread_mutex_t` lock wrapper and verify that final values match calculations.
- **Code/Circuit Reference:**

```
#include <pthread.h>

volatile int g_counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* safe_increment(void *arg) {
    (void)arg;
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock);
        g_counter++; // Critical Section protected!
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Wednesday: POSIX Semaphores (Synchronization & Resource Limits)

- **Conceptual Objective:** Study POSIX Semaphores (`sem_t` from `<semaphore.h>`). Understand that a semaphore is a counting variable that tracks available resources:
- `sem_wait()` : Decrements the counter. If counter is 0, the thread blocks until it is incremented.
- `sem_post()` : Increments the counter and wakes up waiting threads. Comprehend how counting semaphores coordinate queues.
- **Practical Activity:**
 - Write a thread-sync program using unnamed semaphores (`sem_init`).
 - Coordinate execution ordering between two worker threads.
- **Code/Circuit Reference:**


```

#include <semaphore.h>

sem_t sem;

void* first_task(void *arg) {
    // Perform task ...
    sem_post(&sem); // Signal next task
    return NULL;
}

void* second_task(void *arg) {
    sem_wait(&sem); // Block until first_task completes
    // Perform task ...
    return NULL;
}

```

Thursday: Thread Signaling via Condition Variables

- **Conceptual Objective:** Understand that a thread should not spin in a loop checking a variable (busy-waiting) as this wastes CPU cycles. Study **Condition Variables** (`pthread_cond_t`). A condition variable allows a thread to sleep (relinquishing CPU and unlocking a mutex atomic-style) until another thread signals a state change (`pthread_cond_signal` or `pthread_cond_broadcast`).
- **Practical Activity:**
 - Implement a signaling loop where a worker thread waits for a processing flag.
 - Signal the condition from the main thread.
- **Code/Circuit Reference:**

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
bool g_ready = false;

void* wait_for_event(void *arg) {
    pthread_mutex_lock(&mutex);
    while (!g_ready) {
        // Automatically unlocks mutex and blocks thread
        pthread_cond_wait(&cond, &mutex);
    }
    // Wakes up with mutex LOCKED again!
    printf("Event processed!\n");
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

Friday: Deadlocks & Thread-Safe Design Guidelines

- **Conceptual Objective:** Master Thread Safety design principles. Understand **Deadlocks**—situations where two or more threads are permanently blocked, each waiting for a resource held by the other (e.g., Thread 1 holds Mutex A, waits for Mutex B; Thread 2 holds Mutex B, waits for Mutex A). Learn the core rule: always acquire locks in a strict, predefined order.
- **Practical Activity:**
 - Design a deadlock scenario intentionally. Run it and note the application freeze.
 - Apply the strict lock ordering design pattern to resolve the deadlock.
- **Code/Circuit Reference:**

Deadlock Scenario:

Thread 1: lock(A) → lock(B) → unlock(B) → unlock(A)

Thread 2: lock(B) → lock(A) → unlock(A) → unlock(B)

If Thread 1 locks A, and Thread 2 locks B simultaneously → Deadlock!

Resolution (Strict Lock Order):

Thread 1: lock(A) → lock(B)

Thread 2: lock(A) → lock(B) // Thread 2 blocks on A, allowing Thread 1 to finish.

Hands-On Assignment: Thread-Safe Concurrent FIFO Queue

Functional Requirements

Create a highly robust, Thread-Safe Concurrent FIFO Queue (Producer-Consumer Queue) in C, and coordinate active data transfers using POSIX thread synchronization. 1. The queue must manage a structural singly linked list or circular buffer containing integers. 2. Provide thread-safe API endpoints: - `void queue_push(queue_t *q, int val)` : Adds a value to the queue. - `int queue_pop(queue_t *q)` : Removes and returns a value. If the queue is empty, the calling thread must block (sleep) using a **Condition Variable** until a producer pushes data. 3. In `main.c` : - Initialize the queue structure. - Spawn a pool of 2 Producer Threads. Each producer generates 100 integers and pushes them to the queue with random small delays. - Spawn a pool of 3 Consumer Threads. Each consumer pops values from the queue, processes them (prints to stdout), and calculates sums. 4. Ensure the queue handles termination flags safely: when producers complete, signal consumers to exit cleanly without locking up permanently on empty queues.

Structural Requirements & Code Skeleton

```
concurrent_queue.h : ``ifndef CONCURRENT_QUEUE_H
```

define CONCURRENT_QUEUE_H

include

include

define CAPACITY 32

```
typedef struct { int data[CAPACITY]; int head; int tail; int size; pthread_mutex_t mutex;
pthread_cond_t cond_space; // Wait when queue full pthread_cond_t cond_data; // Wait when queue
empty bool is_finished; } queue_t;

void queue_init(queue_t q); void queue_push(queue_t q, int val); int queue_pop(queue_t q, bool
success); void queue_shutdown(queue_t *q);
```

endif // CONCURRENT_QUEUE_H

```

**`concurrent_queue.c` (C-Skeleton)**:
```c
#include "concurrent_queue.h"

void queue_init(queue_t *q) {
 q->head = 0;
 q->tail = 0;
 q->size = 0;
 q->is_finished = false;
 pthread_mutex_init(&q->mutex, NULL);
 pthread_cond_init(&q->cond_space, NULL);
 pthread_cond_init(&q->cond_data, NULL);
}

void queue_push(queue_t *q, int val) {
 pthread_mutex_lock(&q->mutex);

 // Wait if full
 while (q->size == CAPACITY && !q->is_finished) {
 pthread_cond_wait(&q->cond_space, &q->mutex);
 }

 if (q->is_finished) {
 pthread_mutex_unlock(&q->mutex);
 return;
 }

 q->data[q->head] = val;
 q->head = (q->head + 1) % CAPACITY;
 q->size++;

 // Signal waiting consumers that data is ready
 pthread_cond_signal(&q->cond_data);
 pthread_mutex_unlock(&q->mutex);
}

int queue_pop(queue_t *q, bool *success) {
 pthread_mutex_lock(&q->mutex);

 // Wait if empty
 while (q->size == 0 && !q->is_finished) {

```

```

 pthread_cond_wait(&q->cond_data, &q->mutex);
 }

 if (q->size == 0 && q->is_finished) {
 *success = false;
 pthread_mutex_unlock(&q->mutex);
 return -1;
 }

 int val = q->data[q->tail];
 q->tail = (q->tail + 1) % CAPACITY;
 q->size--;
 *success = true;

 // Signal waiting producers that space is available
 pthread_cond_signal(&q->cond_space);
 pthread_mutex_unlock(&q->mutex);

 return val;
}

void queue_shutdown(queue_t *q) {
 pthread_mutex_lock(&q->mutex);
 q->is_finished = true;
 pthread_cond_broadcast(&q->cond_data); // Wake up all waiting
 pthread_cond_broadcast(&q->cond_space);
 pthread_mutex_unlock(&q->mutex);
}

```

---

## Deliverables

- [ ] Queue files `concurrent_queue.c` and `concurrent_queue.h`.
  - [ ] Application entry point `main.c` spawning producer/consumer pools.
  - [ ] Compilation Makefile linking dynamic libraries ( `-lpthread` ).
- 

## ? Self-Check Questions

---

1. Why does a thread possess its own private Stack and Register Context? What memory segments are shared between multiple threads of the same process?

2. What is a **Race Condition**? Explain the mechanical steps of execution (Read, Modify, Write) that lead to memory corruption when two threads update a variable concurrently.
  3. How does a **Mutex** differ from a **Binary Semaphore**? Differentiate ownership rules and how they prevent Priority Inversion inside operating systems.
  4. Why is checking conditions inside a `while` loop ( `while (size == 0)` ) preferred over an `if` statement ( `if (size == 0)` ) when combined with `pthread_cond_wait()` ? Explain the concept of **Spurious Wakeups**.
  5. What is a **Deadlock**? State the four necessary conditions (Coffman Conditions) for a deadlock to occur.
- 

## Stretch Task (Optional)

---

Extend the concurrent queue to be **Lock-Free**. Utilize atomic CPU instructions (such as Compare-And-Swap: `__sync_bool_compare_and_swap` or `<stdatomic.h>`) to implement the push/pop operations without using any mutexes.

## Motivation Checkpoint

---

Whether you are writing a web server handling millions of parallel requests, or designing a high-speed automotive gateway routing sensor streams, concurrent task processing is mandatory. But writing multithreaded systems is highly complex; memory synchronization errors can easily lock up programs. Mastering POSIX threads, mutexes, and cond vars is standard expectation of senior systems developers.

## Trainer Review Rule

---

### • Pass Criteria:

- Thread pool executes cleanly with zero memory leaks or deadlocks.
- Consumers exit cleanly after queue shutdown signals.
- Data counts and sums match producer outputs.

### • Fail Criteria:

- Busy-waiting loops inside pop operations.
- Thread racing errors captured during dynamic testing.

# Weekly Execution Plan: Week 18

---

## Weekly Goal

---

Master advanced Linux Bash shell scripting, harness stream editors ( `sed` ) and pattern processing languages ( `awk` ) for complex text manipulation, and build robust, automated system analysis tools to monitor target statistics.

## Weekly Outcome

---

By Friday, the student will have designed and implemented an **Automated Build, Test, and Analysis Suite** in Bash. They will be able to parse large raw text files, write complex regular expression find-and-replace rules, compile and run C binaries programmatically, and parse outputs to generate clean execution reports.

---

## 17 Session Breakdown (Monday - Friday)

---

### Monday: Bash Scripting Foundations & Advanced Logic

- **Conceptual Objective:** Master professional Bash scripting design patterns. Study exit status variables ( `$?` ), arguments parser parameters ( `$1` , `$@` , `$#` ), conditional checks ( `if` , `case` ), loops ( `for` , `while` ), environment scopes, and error safety declarations ( `set -e` , `set -u` , `set -o pipefail` ).
- **Practical Activity:**
  - Write a script that checks directory paths and parses arguments safely.
  - Implement function libraries with local variable definitions.
- **Code/Circuit Reference:**

```
#!/usr/bin/env bash
set -euo pipefail # Strict Mode

Safe argument check
if ["$#" -ne 2]; then
 echo "Usage: $0 <source_dir> <output_file>"
 exit 1
fi

SOURCE_DIR="$1"
OUTPUT_FILE="$2"
```

## Tuesday: Non-Interactive Stream Editing via `sed`

- **Conceptual Objective:** Study `sed` (Stream Editor)—the classic tool for parsing and transforming text streams. Master fundamental commands: substitution ( `s/pattern/replacement/g` ), line printing ( `/pattern/p` ), deletion ( `d` ), and inline modifications ( `sed -i` ).
- **Practical Activity:**
  - Write a `sed` command to scan a source code file and automatically replace deprecated function symbols with new names.
  - Strip comment lines ( `//` and `/* */` ) from source code programmatically.
- **Code/Circuit Reference:**

```
Inline replace all occurrences of "old_func" with "new_func" in C files
sed -i 's/old_func/new_func/g' src/*.c

Strip blank lines and trailing spaces
sed -i '/^$/d; s/[[:space:]]*$//' src/*.c
```

## Wednesday: Pattern Scanning & Processing via `awk`

- **Conceptual Objective:** Master `awk` —the powerful column-oriented pattern processing language. Understand `awk` record scanning structures ( `BEGIN { ... }` , `/pattern/ { ... }` , `END { ... }` ), built-in variables ( `$0` raw line, `$1` first field, `NF` number of fields, `NR` record number, `FS` field separator ).
- **Practical Activity:**
  - Parse a CSV file containing telemetry statistics.
  - Write an `awk` script to compute column means, locate maximum boundaries, and filter lines based on threshold values.
- **Code/Circuit Reference:**

```
Calculate average CPU load from dynamic CSV logs
Assuming CSV structure: timestamp,cpu_load,mem_used
awk -F',' 'NR > 1 { sum += $2; count++ } END { print "Average CPU Load: " sum/count
"%"}' telemetry.csv
```

## Thursday: Automated Code Auditing & Performance Profiling

- **Conceptual Objective:** Learn how to automate unit testing pipelines. Understand how to run dynamic static checks ( `cppcheck` ), compile applications programmatically via Makefile calls, parse execution outcomes, and measure execution speeds.
- **Practical Activity:**
  - Write a script to compile and run a target test harness.
  - Read and parse console statistics dynamically.
- **Code/Circuit Reference:**

```
Automated Make compilation check
if make; then
 echo "Compilation: PASSED"
else
 echo "Compilation: FAILED"
 exit 1
fi
```

## Friday: Systems Analysis Report Generation

- **Conceptual Objective:** Comprehend the design structure of a complete build automation report. Combine pipelines, `sed`, `awk`, and directory parsers to create standardized test summaries.
- **Practical Activity:**
  - Assemble all features into a unified analysis script.
  - Run the pipeline and output a clean execution dashboard.



## Hands-On Assignment: Automated CI/CD Build & Test Suite

### Functional Requirements

Create a professional Bash automated build and test pipeline named `ci_pipeline.sh`. 1. The script must parse a project workspace directory path as a parameter. 2. Enable strict error handling: `set -euo pipefail`. 3. Programmatically perform the following steps: - **Audit Stage:** Scan all `.c` and `.h` files. Use `grep` and `sed` to flag trailing spaces and count comments. Fail the pipeline if any forbidden

functions (`malloc()`) are found. - **Build Stage:** Run `make clean` followed by `make`. Parse compilation logs. If compilation warnings are detected, output `WARNING: Warning-free builds required` and fail. - **Test Stage:** Run the executable. Redirect standard outputs to a temporary log. - **Analysis Stage:** Use `awk` to parse the output logs. The program outputs lines like `SENSOR_DATA,ID=101,VAL=23.5`. `awk` must extract values, calculate the minimum, maximum, and average values, and fail if any values exceed warning thresholds (e.g., value > 50). 4. Output a final, formatted status dashboard to `build_report.txt` indicating pass/fail status for every stage.

## Structural Requirements & Code Skeleton

`ci_pipeline.sh` (Shell Skeleton):

```
#!/usr/bin/env bash
set -euo pipefail

if ["$#" -ne 1]; then
 echo "Usage: $0 <project_path>"
 exit 1
fi

PROJECT_PATH="$1"
REPORT="build_report.txt"

echo "=== Automated CI/CD Build Report ===" > "$REPORT"
echo "Project: $PROJECT_PATH" >> "$REPORT"
echo "Timestamp: $(date)" >> "$REPORT"
echo "_____ " >> "$REPORT"

cd "$PROJECT_PATH"

1. Audit
echo "Running code audit... "
if grep -rE "malloc\(|free\(" src/ include/ > /dev/null 2>&1; then
 echo "Audit: FAILED (Forbidden heap calls found)" >> "$REPORT"
 exit 1
else
 echo "Audit: PASSED" >> "$REPORT"
fi

2. Build
echo "Compiling project... "
if make > build.log 2>&1; then
 if grep -iq "warning" build.log; then
 echo "Build: FAILED (Compilation warning detected)" >> "$REPORT"
 exit 1
 fi
 echo "Build: PASSED" >> "$REPORT"
else
 echo "Build: FAILED (Compilation error)" >> "$REPORT"
 exit 1
fi

3. Test & Analysis
echo "Running functional tests..."
```



```

./build/app > test.log 2>&1 || true

Extract sensor values using awk
awk '
BEGIN {
 count = 0; sum = 0; max = -999; min = 999; threshold = 50.0; exceeded = 0;
}
/SENSOR_DATA/ {
 split($2, id_part, "=");
 split($3, val_part, "=");
 val = val_part[2];

 count++;
 sum += val;
 if (val > max) max = val;
 if (val < min) min = val;
 if (val > threshold) exceeded++;
}
END {
 print "Processed " count " telemetry samples.";
 if (count > 0) {
 avg = sum / count;
 print "Telemetry Min: " min;
 print "Telemetry Max: " max;
 print "Telemetry Avg: " avg;
 if (exceeded > 0) {
 print "WARNING: " exceeded " samples exceeded threshold!";
 exit 2;
 }
 } else {
 print "ERROR: No telemetry samples found.";
 exit 3;
 }
}
' test.log >> "$REPORT" 2>&1

status=$?
if [$status -eq 0]; then
 echo "Analysis: PASSED" >> "$REPORT"
elif [$status -eq 2]; then
 echo "Analysis: FAILED (Threshold Exceeded)" >> "$REPORT"
else
 echo "Analysis: FAILED (No Data)" >> "$REPORT"

```

```
fi
```

```
cat "$REPORT"
```



## Deliverables

- [ ] Automated script `ci_pipeline.sh`.
- [ ] Target application workspace to compile and test.
- [ ] Executable build report `build_report.txt` indicating completed pipeline runs.

## ? Self-Check Questions

1. Why is the statement `set -euo pipefail` critical when writing professional, production-grade automation scripts? Explain what each flag does.
2. How does `sed` substitution logic work? Write a `sed` command to replace all occurrences of `uint8` with `uint8_t` recursively in C files.
3. What is the execution flow of `awk` scripts? Differentiate the roles of `BEGIN`, pattern scanning, and `END` blocks.
4. How do you parse command line flags (e.g., `-b`, `-t`) dynamically inside a Bash script? Explain `getopts`.
5. Why are automation validation checks crucial inside large-scale firmware organizations?



## Stretch Task (Optional)

Extend the automation pipeline to run under a **cron schedule**: configure your host cron daemon to trigger the `ci_pipeline.sh` automatically every hour, email reports on failures, and log historic trends.



## Motivation Checkpoint

Continuous Integration and Deployment (CI/CD) is the backbone of modern tech companies. When firmware developers commit changes, automated build servers pull the branch, run audits, compile toolchains, flash hardware nodes, and parse execution results instantly. Mastery of Bash, `sed`, and `awk` is the foundational key to unlocking automated pipelines.



## Trainer Review Rule

---

- **Pass Criteria:**

- Script handles exit states safely.
- Pattern rules inside `awk` correctly extract and compute telemetry ranges.
- Zero compilation warnings are tolerated in target compiles.

- **Fail Criteria:**

- Hardcoded paths or environment setups that fail on foreign host environments.
- Missing bounds checks.

# Weekly Execution Plan: Week 19

---

## Weekly Goal

---

Master the core concepts of Real-Time Operating Systems (RTOS), understand the architecture of FreeRTOS, learn how task control blocks (TCBs) are allocated in SRAM, and write modular, multi-tasking software architectures using task creation APIs.

## Weekly Outcome

---

By Friday, the student will have successfully integrated a custom **FreeRTOS Port** on the STM32F4 microcontroller. They will have configured the FreeRTOS config file from scratch, spawned multiple tasks executing concurrently, and structured tasks with distinct priorities and stack parameters.

---



## Session Breakdown (Monday – Friday)

---

### Monday: The RTOS Paradigm & Core Real-Time Parameters

- **Conceptual Objective:** Master the fundamental differences between bare-metal super-loops ( `while(1)` ) and multi-tasking real-time operating systems. Differentiate between Hard Real-Time (missing a deadline results in system failure, e.g., automotive ABS systems) and Soft Real-Time (missed deadlines degrade performance but are not catastrophic). Understand deterministic scheduling.
- **Practical Activity:**
  - Sketch a block diagram comparing a bare-metal interrupt-polling model with an RTOS context-switching architecture.
  - Define the real-time parameter definitions: Latency, Jitter, and Deadlines.
- **Code/Circuit Reference:**

Bare-Metal Super-Loop (Sequential):

```
main() → Task_A() → Task_B() → Task_C() --+
 ^ |
 +-----+
 * If Task_B blocks or delays, Task_C is starved!
```

RTOS Preemptive Model (Concurrent):

```
Task_A (High Priority) ---[Running]-----> [Block]----->[Running]→
Task_B (Low Priority) -----[Ready]→ [Running]→ [Preempted]→
 * The RTOS Scheduler swaps tasks dynamically based on priority.
```

## Tuesday: FreeRTOS Architecture & The Task Control Block (TCB)

- **Conceptual Objective:** Study how FreeRTOS tracks execution states. Understand the **Task Control Block (TCB)**—the primary kernel structure in SRAM representing a task:
- Task Stack Pointer (holds the current top-of-stack of the task).
- Task Priority.
- Task State (Running, Ready, Blocked, Suspended).
- Link pointers to join task-state lists.
- **Practical Activity:**
- Deep-dive into FreeRTOS source file `tasks.c` to locate and analyze the `TCB_t` structure.
- Calculate stack requirements for tasks based on local variables sizes.
- **Code/Circuit Reference:**

```
/* Conceptual TCB structure layout in SRAM */
typedef struct tskTaskControlBlock {
 volatile StackType_t *pxTopOfStack; // Must be first member!
 ListItem_t xStateListItem; // Task state list link
 ListItem_t xEventListItem; // Event list link
 UBaseType_t uxPriority; // Task Priority
 StackType_t *pxStack; // Start address of stack
 char pcTaskName[configMAX_TASK_NAME_LEN];
} TCB_t;
```

## Wednesday: Mastering FreeRTOS Configurations

- **Conceptual Objective:** Master `FreeRTOSConfig.h` —the core configuration file that selectively enables kernel compilation modules. Study critical configuration definitions: `configCPU_CLOCK_HZ`, `configTICK_RATE_HZ` (system tick timer speed), `configMAX_PRIORITIES`, `configMINIMAL_STACK_SIZE`, and scheduling settings.

- **Practical Activity:**

- Construct a clean, minimal `FreeRTOSConfig.h` for the STM32F4.
- Differentiate `configUSE_PREEMPTION` (preemptive vs co-operative scheduling models).

- **Code/Circuit Reference:**

```
/* Minimal FreeRTOSConfig.h for STM32F4 */
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

#define configUSE_PREEMPTION 1
#define configUSE_PORT_OPTIMISED_TASK_SELECTION 1
#define configCPU_CLOCK_HZ (16000000UL) // 16MHz clock
#define configTICK_RATE_HZ ((TickType_t)1000) // 1ms tick
#define configMAX_PRIORITIES (5)
#define configMINIMAL_STACK_SIZE ((unsigned short)130)
#define configTOTAL_HEAP_SIZE ((size_t)(10 * 1024))
#define configMAX_TASK_NAME_LEN (16)

#define vPortSVCHandler SVC_Handler
#define xPortPendSVHandler PendSV_Handler
#define xPortSysTickHandler SysTick_Handler

#endif /* FREERTOS_CONFIG_H */
```

## Thursday: Spawning Tasks via `xTaskCreate`

- **Conceptual Objective:** Master task creation API `xTaskCreate()`. Differentiate parameters: Task Function Pointer, Task Name (string), Stack Depth (number of `StackType_t` variables, typically 32-bit values), Argument Parameter ( `void*` ), Priority, and the Task Handle output. Understand task function signatures ( `void worker(void *pvParameters)` containing an infinite loop that must never return).

- **Practical Activity:**

- Write a program initializing 2 separate task functions with different priorities.
- Implement local print notifications inside each loop.

- **Code/Circuit Reference:**

```

void vTask1(void *pvParameters) {
 (void)pvParameters;
 while (1) {
 // Task operations
 // Task must block using vTaskDelay to yield CPU!
 vTaskDelay(pdMS_TO_TICKS(500));
 }
}

void main(void) {
 // Spawn task
 xTaskCreate(vTask1, "Task_One", 128, NULL, 2, NULL);
 vTaskStartScheduler(); // Starts ticks and jumps to first task
}

```

## Friday: The Idle Task & Dynamic Thread starvations

- **Conceptual Objective:** Study the **Idle Task**—automatically spawned by the kernel when the scheduler starts, running at priority 0. Understand that the Idle Task keeps the CPU from locking when all application tasks are in the Blocked state, and executes memory reclamation of terminated tasks.
- **Practical Activity:**
  - Write a program where a high-priority task enters an infinite loop without calling block or delay APIs ( `vTaskDelay` ).
  - Observe how the high-priority task starves all lower-priority tasks and the Idle task, locking execution.
- **Code/Circuit Reference:**

```

void vStarvationTask(void *pvParameters) {
 (void)pvParameters;
 while (1) {
 // Busy loop (no vTaskDelay).
 // Starves any task of priority < current!
 }
}

```

## Hands-On Assignment: Custom Ported Concurrent LED Blinker

---

### Functional Requirements

Create a fully modular FreeRTOS application compiling natively on the STM32F4 using your custom startup/linker tools, spawning concurrent task threads. 1. The project must compile the core FreeRTOS source files: `tasks.c`, `list.c`, `queue.c`, `port.c` (Cortex-M4 Port), and `heap_4.c` (dynamic memory). 2. Configure `FreeRTOSConfig.h` to set the system clock to 16MHz and ticks to 1ms. 3. Spawn two distinct tasks: - `vLedGreenTask` (**Priority 2**): Toggles LED PD12 every 200ms. - `vLedRedTask` (**Priority 1**): Toggles LED PD14 every 600ms. 4. Utilize `vTaskDelay` exclusively inside the task loops to manage timing. Do not use bare-metal delay loops. 5. In `startup.c`, ensure the Vector Table routes `SVC_Handler`, `PendSV_Handler`, and `SysTick_Handler` to their matching FreeRTOS port entry points (defined as aliases in your config file). 6. Verify concurrent execution: hook up your logic analyzer to PD12 and PD14 to capture the stable, non-interfering blink cycles.

### Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):



```

#include "FreeRTOS.h"
#include "task.h"
#include <stdint.h>

void vLedGreenTask(void *pvParameters) {
 volatile uint32_t *odr = (volatile uint32_t*)0x40020C14U;
 while (1) {
 *odr ^= (1U << 12); // Toggle PD12
 vTaskDelay(pdMS_TO_TICKS(200));
 }
}

void vLedRedTask(void *pvParameters) {
 volatile uint32_t *odr = (volatile uint32_t*)0x40020C14U;
 while (1) {
 *odr ^= (1U << 14); // Toggle PD14
 vTaskDelay(pdMS_TO_TICKS(600));
 }
}

void configure_leds(void) {
 // Enable GPIOD Clock
 ((volatile uint32_t)0x40023830U) |= (1U << 3);

 // Configure PD12 and PD14 as output
 volatile uint32_t *moder = (volatile uint32_t*)0x40020C00U;
 *moder &= ~(((3U << (12 * 2)) | (3U << (14 * 2))));
 *moder |= ((1U << (12 * 2)) | (1U << (14 * 2)));
}

int main(void) {
 configure_leds();

 // Spawn green task (High Priority)
 xTaskCreate(vLedGreenTask, "Green_Blinker", 128, NULL, 2, NULL);

 // Spawn red task (Low Priority)
 xTaskCreate(vLedRedTask, "Red_Blinker", 128, NULL, 1, NULL);

 // Start Scheduler (does not return)
 vTaskStartScheduler();
}

```

```
while (1);
return 0;
}
```

---

## Deliverables

- [ ] Complete project directory containing FreeRTOS source libraries.
  - [ ] Configured `FreeRTOSConfig.h` and updated bare-metal `startup.c`.
  - [ ] Compilation Makefile building dynamic binaries cleanly.
  - [ ] Capture trace waveform `rtos_blink.log` proving concurrent execution frequencies of both tasks.
- 

## ? Self-Check Questions

1. Differentiate **Preemptive Scheduling** from **Co-operative Scheduling**. How does setting `configUSE_PREEMPTION` to `0` change task execution dynamics?
  2. What are the mechanical roles of `SVC_Handler` and `PendSV_Handler` inside the FreeRTOS context-switching pipeline?
  3. Why does the TCB structure place the `pxTopOfStack` pointer as the very first member? How does the assembly port utilize this layout during context swaps?
  4. What is **Task Starvation**? How do you prevent a high-priority CPU-bound task from starving low-priority threads?
  5. What is the role of the Idle Task inside FreeRTOS? Can you put the processor into a sleep mode inside the Idle Task hook function to conserve energy?
- 

## Stretch Task (Optional)

Implement an **Idle Task Hook function** ( `vApplicationIdleHook` ) that configures the microcontroller core to enter low-power sleep mode ( `__WFI` - Wait For Interrupt) whenever the CPU is inactive, wake up dynamically on task timers, and output CPU usage metrics.

## Motivation Checkpoint

Bare-metal firmware super-loops are completely unsuitable for complex modern designs: if a single peripheral locks up while polling, the entire system freezes. Operating systems (RTOS) resolve this by

enforcing time-slice preemption and priority guarantees. Mastering FreeRTOS configurations, TCB architectures, and scheduler ports is a core step to building industrial IoT firmwares.



## Trainer Review Rule

---

- **Pass Criteria:**

- Tasks compile and run successfully using FreeRTOS.
- Context switching functions cleanly (LEDs flash concurrently).
- Standard delay routines are banned (must use `vTaskDelay` ).

- **Fail Criteria:**

- Task execution blocks due to incorrect Vector Table routing of handlers.
- Direct stack manipulations inside task functions.

# Weekly Execution Plan: Week 20

---

## Weekly Goal

---

Master the FreeRTOS Task State Machine, understand the mathematical mechanics of Preemptive vs Co-operative scheduling, and analyze the low-level assembly sequence of a CPU Context Switch.

## Weekly Outcome

---

By Friday, the student will have built a **Context Switch Visualizer Utility** in C/Assembly. They will understand how registers are pushed and popped during task transitions, how the processor manipulates the Stack Pointer (SP) inside PendSV handlers, and how task state lists (Ready, Blocked, Suspended) are sorted dynamically by the kernel.

---



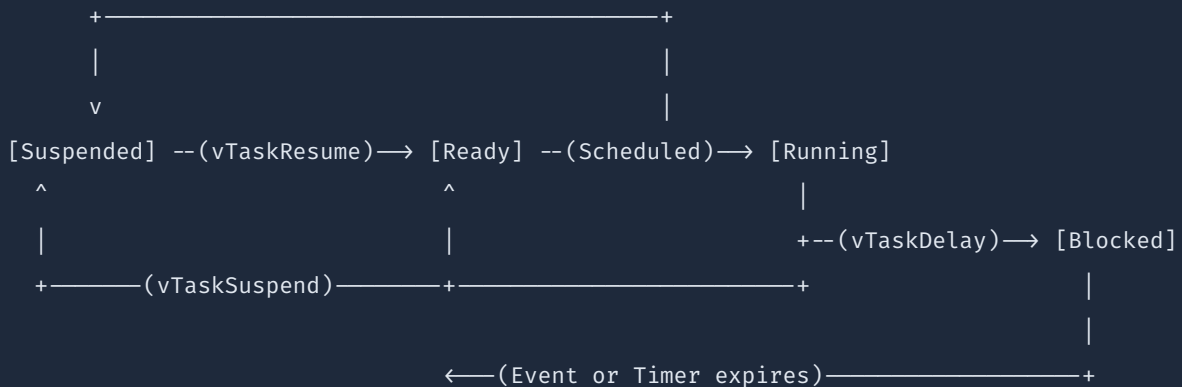
## Session Breakdown (Monday - Friday)

---

### Monday: The FreeRTOS Task State Machine

- **Conceptual Objective:** Master the lifecycle states of a task:
- **Running:** The CPU is actively executing the task's instructions.
- **Ready:** Task is eligible for execution but a higher or equal priority task is running.
- **Blocked:** Task is waiting for a temporal delay ( `vTaskDelay` ) or an event trigger (semaphore, queue, notification).
- **Suspended:** Task is explicitly removed from scheduling.
- **Practical Activity:**
  - Sketch the complete State Transition Diagram of FreeRTOS tasks.
  - Write a task that transitions dynamically between Ready, Blocked, and Suspended states using `vTaskSuspend` and `vTaskResume` .
- **Code/Circuit Reference:**

#### FreeRTOS Task State Transitions:



## Tuesday: Preemptive vs Co-operative Scheduling Algorithms

- **Conceptual Objective:** Study the scheduling engines:
- **Preemptive Time-Slicing:** The scheduler forces equal-priority tasks to swap execution every system tick (round-robin), and instantly preempts lower-priority tasks when a high-priority task becomes ready.
- **Co-operative:** Running tasks execute continuously until they explicitly yield control ( `taskYIELD()` ) or block.
- **Practical Activity:**
  - Configure two identical priority tasks.
  - Observe scheduling execution traces under preemptive mode versus co-operative mode.
- **Code/Circuit Reference:**

```

// Force current task to yield CPU to another ready task of equal priority
void vCooperativeWorker(void *pvParameters) {
 while (1) {
 // Perform processing chunk...
 taskYIELD(); // Yield control dynamically
 }
}

```

## Wednesday: Context Switch Mechanics I (The SysTick Trigger)

- **Conceptual Objective:** Understand how a context switch is initiated. A context switch is triggered either by a task blocking (e.g., calling `vTaskDelay` ) or by the **SysTick Timer interrupt** firing every 1ms. The SysTick handler checks if a higher priority task is ready or if time-slicing is needed, and requests a **PendSV Exception** by writing to the Interrupt Control State Register (ICSR).
- **Practical Activity:**
  - Deep-dive into PendSV exception routing.

- Trace SysTick interrupt triggers in assembly.
- **Code/Circuit Reference:**

```
// Triggering a PendSV exception programmatically via ICSR register
#define SCB_ICSR ((volatile uint32_t*)0xE000ED04U)
#define SCB_ICSR_PENDSVSET (1U << 28)

void trigger_pendsv_yield(void) {
 *SCB_ICSR = SCB_ICSR_PENDSVSET; // Request PendSV execution
}
```

## Thursday: Context Switch Mechanics II (Assembly Register Stacking)

- **Conceptual Objective:** Master the assembly-level mechanics of the PendSV Handler during a context swap:
- Hardware automatically stacks **R0-R3, R12, LR, PC, xPSR** registers onto the active stack pointer.
- Assembly code blocks interrupts and manually stacks remaining registers **R4-R11** using `push` or `stmdb` instructions.
- Load the address of the current TCB structure. Save the current Stack Pointer to `pxTopOfStack`.
- Call the C scheduler function `vTaskSwitchContext` to update `pxCurrentTCB` to point to the new highest-priority Ready task.
- Load the new Stack Pointer from the new TCB's `pxTopOfStack`.
- Manually pop registers **R4-R11** from the new stack.
- Exit PendSV. Hardware automatically restores **R0-R3, R12, LR, PC, xPSR** from the new stack.
- **Practical Activity:**
- Study the physical ARM Cortex-M4 assembly code inside FreeRTOS port file `port.c`.
- Draw stack structures showing values before and after context swaps.
- **Code/Circuit Reference:**

```

/* Conceptual PendSV assembly implementation (Cortex-M) */
.thumb
.align 2
.global xPortPendSVHandler
xPortPendSVHandler:
 mrs r0, psp /* Get active Process Stack Pointer (PSP) */

 ldr r3, =pxCurrentTCB /* Get current TCB address pointer */
 ldr r2, [r3] /* Load current TCB memory block */

 stmdb r0!, {r4-r11} /* Manually push R4-R11 to PSP stack */
 str r0, [r2] /* Save new top of stack pointer inside TCB */

 bl vTaskSwitchContext /* Call C function to select next task */

 ldr r3, =pxCurrentTCB
 ldr r2, [r3]
 ldr r0, [r2] /* Load new stack pointer address from TCB */

 ldmia r0!, {r4-r11} /* Manually pop R4-R11 from new stack */
 msr psp, r0 /* Update PSP to point to new stack */
 bx lr /* Exit. Hardware automatically pops R0-R3 etc. */

```

## Friday: Task Yielding & List Operations

- **Conceptual Objective:** Understand how the kernel manages task list transitions. When a task delays, it is removed from the `pxReadyTasksLists` array and linked to `pxDelayedTaskList`. Learn how list heads are traversed dynamically during scheduling context switches.
- **Practical Activity:**
  - Trace task list linkings inside `list.c`.
  - Document list traversal speeds.

---

## Hands-On Assignment: Assembly Context Switch Visualizer

---

### Functional Requirements

Create a mock context switch visualizer application in C that simulates the register state transformations of two tasks during a context switch. 1. The application must define a structure `mock_stack_frame_t` representing the register layout of an ARM stack frame after a context swap (containing R0-R12, LR, PC, xPSR). 2. Allocate separate `uint32_t stack_task_a[256]` and

`stack_task_b[256]` arrays. 3. Write an initialization function `void init_mock_stack(uint32_t *stack, void (*task_func)(void))` that programmatically writes initial values into the stack arrays: - Sets the initial PC to the address of the task function. - Sets the initial xPSR to `0x01000000` (thumb mode active). - Set general purpose registers to unique dummy signatures (e.g., R0 = `0x00000000`, R4 = `0x44444444`). 4. Write a simulated context switch function `void simulate_context_switch(uint32_t **current_sp, uint32_t *new_sp)` in C/Assembly. This function must simulate saving the "registers" of Task A, switching stack pointers, and loading the "registers" of Task B. 5. Print out the step-by-step register states to verify successful stack frame initialization and execution transitions.

## Structural Requirements & Code Skeleton

`switch_visualizer.c` (C-Skeleton):



```

#include <stdio.h>
#include <stdint.h>
#include <stdbool.h>

typedef struct {
 // Manually stacked registers (R4 - R11)
 uint32_t r4; uint32_t r5; uint32_t r6; uint32_t r7;
 uint32_t r8; uint32_t r9; uint32_t r10; uint32_t r11;
 // Automatically stacked registers (R0 - R3, R12, LR, PC, xPSR)
 uint32_t r0; uint32_t r1; uint32_t r2; uint32_t r3;
 uint32_t r12; uint32_t lr; uint32_t pc; uint32_t xpsr;
} mock_stack_frame_t;

void task_a(void) { printf("Inside Virtual Task A!\n"); }
void task_b(void) { printf("Inside Virtual Task B!\n"); }

uint32_t* init_mock_stack(uint32_t *stack_limit, void (*task_func)(void), uint32_t
task_id) {
 // Align pointer to the top of stack
 mock_stack_frame_t *frame = (mock_stack_frame_t*)(stack_limit -
sizeof(mock_stack_frame_t)/sizeof(uint32_t));

 // Initialize automatic registers
 frame->xpsr = 0x01000000; // Thumb mode
 frame->pc = (uint32_t)task_func;
 frame->lr = 0xDEADBEEF; // Dummy return address
 frame->r12 = 0x12121212;
 frame->r3 = 0x33333333;
 frame->r2 = 0x22222222;
 frame->r1 = 0x11111111;
 frame->r0 = task_id; // Pass Task ID in R0!

 // Initialize manual registers
 frame->r11 = 0x11111111;
 frame->r10 = 0x10101010;
 frame->r9 = 0x09090909;
 frame->r8 = 0x08080808;
 frame->r7 = 0x07070707;
 frame->r6 = 0x06060606;
 frame->r5 = 0x05050505;
 frame->r4 = 0x04040404;

```

```

 return (uint32_t*)frame; // Return new Top of Stack pointer
}

void print_stack_frame(const uint32_t *sp) {
 const mock_stack_frame_t *frame = (const mock_stack_frame_t*)sp;
 printf("--- Stack Frame Dump (SP = %p) ---\n", (void*)sp);
 printf(" PC : 0x%08X | xPSR: 0x%08X\n", frame->pc, frame->xpsr);
 printf(" R0 : 0x%08X | LR : 0x%08X\n", frame->r0, frame->lr);
 printf(" R4 : 0x%08X | R5 : 0x%08X\n", frame->r4, frame->r5);
 printf(" R11 : 0x%08X | R12 : 0x%08X\n", frame->r11, frame->r12);
 printf("-----\n");
}

```



## Deliverables

- [ ] Visualizer C source file `switch_visualizer.c`.
- [ ] Compilation Makefile compiling under standard warnings.
- [ ] Output analysis report `context_swap_flow.txt` documenting the step-by-step register changes during stack transitions.

## ? Self-Check Questions

1. Why must a task function block inside FreeRTOS (e.g., using `vTaskDelay`) instead of running simple empty loops? What happens physically to the ready task lists if a task runs a CPU-bound loop?
2. What are the four core task states in FreeRTOS? Which state does a task enter when it is waiting for an I2C data packet to arrive?
3. Why are context switches executed inside the **PendSV exception handler** instead of directly inside the SysTick timer handler? Differentiate execution priority contexts.
4. Explain step-by-step what CPU registers are stacked by hardware and what are stacked by software during Cortex-M interrupt entries.
5. What does the variable `pxCurrentTCB` represent inside the FreeRTOS kernel? How does `vTaskSwitchContext()` modify it?



## Stretch Task (Optional)

---

Implement a **Run-Time Performance Monitor** script in C that reads the stack pointers dynamically during task executions and prints active stack usage percentages.



## Motivation Checkpoint

---

Context switching is the magic that allows a single CPU core to run dozens of threads concurrently, creating the illusion of parallel processing. But context switching is highly timing-critical: every register push/pop adds execution latency. Mastering the low-level assembly dynamics of PendSV is standard requirement for real-time systems designers.



## Trainer Review Rule

---

- **Pass Criteria:**

- Virtual stack frames are correctly initialized with Thumb mode and program addresses.
- Context switch simulation runs successfully without address faults or memory leaks.
- Logic analyzer/assembly traces verify correct register assignments.

- **Fail Criteria:**

- Unaligned stack pointer bounds.
- System lock-up due to incorrect context calculations.

# Weekly Execution Plan: Week 21

---

## Weekly Goal

---

Master resource synchronization in Real-Time Operating Systems, configure Binary and Counting Semaphores, implement Mutexes for exclusive resource locking, and analyze/resolve the critical issue of **Priority Inversion** utilizing **Priority Inheritance**.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Secure Shared Hardware Resource Manager** using FreeRTOS. They will understand the mathematical dynamics of priority scheduling, how to prevent race conditions when tasks share hardware interfaces, and how to verify that priority inheritance operates on physical tasks.

---

## Session Breakdown (Monday - Friday)

---

### Monday: Binary Semaphores (Signaling & Task Sync)

- **Conceptual Objective:** Master the Binary Semaphore—a synchronization block that can hold a value of `0` (unavailable) or `1` (available). Differentiate between a semaphore and a lock: semaphores are designed for **task signaling** (Task A does work, then posts to the semaphore to signal Task B to execute) and do not have ownership (any task or ISR can post).
- **Practical Activity:**
  - Learn how to initialize binary semaphores using `xSemaphoreCreateBinary()`.
  - Implement a two-task signaling loop where Task 1 waits on `xSemaphoreTake` until Task 2 executes `xSemaphoreGive`.
- **Code/Circuit Reference:**

```

#include "semphr.h"

SemaphoreHandle_t xSignalSem = NULL;

void vSenderTask(void *pvParameters) {
 while (1) {
 // Perform work ...
 xSemaphoreGive(xSignalSem); // Signal receiver
 vTaskDelay(pdMS_TO_TICKS(1000));
 }
}

void vReceiverTask(void *pvParameters) {
 while (1) {
 if (xSemaphoreTake(xSignalSem, portMAX_DELAY) == pdTRUE) {
 // Wake up instantly on signal!
 }
 }
}

```

## Tuesday: Counting Semaphores (Resource Counting Pools)

- **Conceptual Objective:** Study Counting Semaphores—synchronization structures initialized with a maximum count value greater than 1. Understand how counting semaphores monitor a pool of identical resources (e.g., managing access to a buffer array containing 3 empty memory slots).
- **Practical Activity:**
  - Initialize a counting semaphore using `xSemaphoreCreateCounting()`.
  - Write a simulated printer-pool allocator managing 3 virtual print nodes across 5 requestor tasks.
- **Code/Circuit Reference:**

```
// Create counting semaphore: Max Count = 3, Initial Count = 3
SemaphoreHandle_t xPoolSem = xSemaphoreCreateCounting(3, 3);

void vUserTask(void *pvParameters) {
 while (1) {
 if (xSemaphoreTake(xPoolSem, pdMS_TO_TICKS(100)) == pdTRUE) {
 // Acquired 1 resource from the pool of 3
 // Perform processing...
 xSemaphoreGive(xPoolSem); // Release back to pool
 }
 }
}
```

## Wednesday: Mutexes (Mutual Exclusion & Ownership)

- **Conceptual Objective:** Master the Mutual Exclusion (Mutex) primitive. Differentiate a Mutex from a Binary Semaphore:
- **Ownership:** A mutex has a strict owner concept: the task that takes the mutex must be the task that releases it.
- **Usage:** Mutexes are designed strictly for **mutual exclusion** (protecting a shared memory block or physical peripheral from concurrent access corruptions).
- **Practical Activity:**
  - Initialize a mutex using `xSemaphoreCreateMutex()`.
  - Write a driver where multiple tasks attempt to transmit data over a single UART port concurrently, protecting the UART peripheral from data interleavings using a Mutex.
- **Code/Circuit Reference:**

```
SemaphoreHandle_t xUartMutex = NULL;

void print_uart_safe(const char *str) {
 if (xSemaphoreTake(xUartMutex, portMAX_DELAY) == pdTRUE) {
 // Crucial: Only one task can execute this block at a time
 while (*str) {
 write_char(*str++);
 }
 xSemaphoreGive(xUartMutex); // Release ownership
 }
}
```

## Thursday: The Priority Inversion Disaster

- **Conceptual Objective:** Deep-dive into **Priority Inversion**—a catastrophic real-time scheduling failure that occurs when a low-priority task holds a shared resource (via a simple semaphore) needed by a high-priority task, and a medium-priority task preempts the low-priority task, indefinitely blocking the high-priority task from executing. (Recall the classic Mars Pathfinder lock-up!).
- **Practical Activity:**
  - Sketch a detailed timeline diagram showing three tasks (High, Med, Low) entering a priority inversion loop.
  - Define the architectural sequence of the scheduling breakdown.
- **Code/Circuit Reference:**

Priority Inversion Timeline (No Priority Inheritance):

```
Low Task (P1) ---[Locks Sem]----->[Preempted by
Med]----->
Med Task (P2) -----[Starts Running]----->
High Task (P3) -----[Blocks on Sem held by Low (Starved)]----->
* Medium Task (P2) is running, preventing Low Task (P1) from releasing the Semaphore.
* Consequently, High Task (P3) is indefinitely blocked (Inverted priority!).
```

## Friday: Resolving Priority Inversion via Priority Inheritance

- **Conceptual Objective:** Study how **Priority Inheritance** resolves Priority Inversion. When a high-priority task blocks on a Mutex held by a low-priority task, the kernel temporarily boosts the priority of the low-priority task to match that of the high-priority task. This allows the low-priority task to run, finish its critical section, and release the Mutex, at which point its priority is restored, and the high-priority task instantly preempts it and runs.
- **Practical Activity:**
  - Configure FreeRTOS with Mutexes (priority inheritance is enabled by default in FreeRTOS mutexes).
  - Write a test case spawning three tasks (High, Med, Low) sharing a Mutex.
  - Observe priority inheritance preventing task starvation.
- **Code/Circuit Reference:**

```
// FreeRTOS Mutex automatically applies Priority Inheritance!
SemaphoreHandle_t xInheritMutex = xSemaphoreCreateMutex();
```

# Hands-On Assignment: Secure Multi-Tasking UART Resource Manager

---

## Functional Requirements

Create a robust FreeRTOS application that implements a secure, synchronized UART printer driver, demonstrating and verifying the mechanics of Priority Inversion and Priority Inheritance. 1. The application must initialize three tasks: - `vLowPriorityTask` (**Priority 1**): Periodically takes a shared lock, writes a long message (e.g., `"Low Task processing shared UART resource ..."`) character-by-character to the UART, and holds the lock. - `vMediumPriorityTask` (**Priority 2**): Periodically triggers a CPU-bound busy loop simulating calculations, running continuously without blocking. - `vHighPriorityTask` (**Priority 3**): Spawns after a delay, attempts to take the shared lock, and writes a short message. 2. **Scenario 1 (Failure)**: Implement the shared lock using a standard **Binary Semaphore**. Run the system and observe Priority Inversion: the Medium Task preempts the Low Task, indefinitely starving the High Task. Show this starvation on your serial terminal. 3. **Scenario 2 (Success)**: Replace the Binary Semaphore with a **Mutex** (which possesses Priority Inheritance). Run the system and demonstrate that the Low Task's priority is boosted, allowing it to complete and release the lock, preventing the Medium Task from starving the High Task. 4. Capture timing differences and document your findings.

## Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):



```

#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <stdio.h>

// Shared lock handles
SemaphoreHandle_t xSharedLock = NULL;

void vLowPriorityTask(void *pvParameters) {
 while (1) {
 if (xSemaphoreTake(xSharedLock, portMAX_DELAY) == pdTRUE) {
 printf("[LOW] Acquired Lock. Starting slow UART write ... \n");

 // Simulate slow write
 for (int i = 0; i < 50; i++) {
 printf(".");
 fflush(stdout);
 vTaskDelay(pdMS_TO_TICKS(10)); // Yield slightly
 }
 printf("\n[LOW] Releasing Lock.\n");
 xSemaphoreGive(xSharedLock);
 }
 vTaskDelay(pdMS_TO_TICKS(1000));
 }
}

void vMediumPriorityTask(void *pvParameters) {
 vTaskDelay(pdMS_TO_TICKS(100)); // Start slightly after Low
 while (1) {
 printf("[MED] Running CPU Busy loop ... \n");
 // Simulate busy loop
 for (volatile uint32_t i = 0; i < 5000000; i++);
 vTaskDelay(pdMS_TO_TICKS(500));
 }
}

void vHighPriorityTask(void *pvParameters) {
 vTaskDelay(pdMS_TO_TICKS(200)); // Start last
 while (1) {
 printf("[HIGH] Attempting to acquire Lock ... \n");
 if (xSemaphoreTake(xSharedLock, portMAX_DELAY) == pdTRUE) {
 printf("[HIGH] SUCCESS: Acquired Lock instantly!\n");
 }
 }
}

```

```
 xSemaphoreGive(xSharedLock);
 }
 vTaskDelay(pdMS_TO_TICKS(2000));
}
}
```

---

## Deliverables

- [ ] Source file `main.c` containing task definitions.
- [ ] Compilation Makefile.
- [ ] Output analysis log `inversion_proof.log` showing stdout prints of Scenario 1 (High task starved) vs Scenario 2 (High task runs successfully).

---

## ? Self-Check Questions

1. Why are Binary Semaphores considered unsafe for mutual exclusion? Explain how they differ from Mutexes regarding ownership and priority inheritance.
2. Walk through the Mars Pathfinder Priority Inversion bug. What were the task priorities, what was the shared resource, and how did engineers patch it remotely?
3. How does Priority Inheritance physically boost a task's priority? Trace what happens inside the TCB of the Low Task when the High Task attempts to take the Mutex.
4. Can you take or give a Mutex inside an Interrupt Service Routine (ISR)? Why does FreeRTOS restrict Mutex operations inside interrupt contexts?
5. What is a **Counting Semaphore**? Give a real-world example of when a counting semaphore should be used instead of a binary semaphore.

---

## Stretch Task (Optional)

Implement a **Recursive Mutex** ( `xSemaphoreCreateRecursiveMutex` ) that allows a single task to take the lock multiple times nested-style without deadlocking itself, and verify stack releases.

---

## Motivation Checkpoint

Concurrency is the baseline of modern computing. But concurrent systems are highly vulnerable to race conditions and synchronization deadlocks. In automotive-grade firmwares, a single priority

inversion on a braking or steering communication bus can cause physical accidents. Mastering synchronization primitives and priority inheritance is the primary rule of reliable firmware.



## Trainer Review Rule

---

- **Pass Criteria:**

- Code compiles without warnings with `-Wall -Wextra -Werror`.
- Inversion demonstration shows successful isolation of tasks.
- Priority Inheritance successfully bypasses Medium task preemptions.

- **Fail Criteria:**

- Missing return status validations.
- System lock-ups during execution due to unreleased locks.

# Weekly Execution Plan: Week 22

---

## Weekly Goal

---

Master Inter-Task Communication (ITC) within Real-Time Operating Systems, configure **Message Queues** for structured data routing, utilize **Event Groups** for multi-event synchronization, and harness light-weight **Task Notifications** for high-speed, minimal-overhead signaling.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Multi-Sensor Telemetry Concentrator** using FreeRTOS. They will understand how tasks pass structures safely without global memory corruption, how to synchronize multiple asynchronous tasks concurrently, and how to optimize communication latency using Task Notifications.

---

## 17 July Session Breakdown (Monday - Friday)

---

### Monday: FreeRTOS Message Queues (Structured FIFO Data Routing)

- **Conceptual Objective:** Master Message Queues—thread-safe FIFO buffers managed by the kernel. Understand that queues copy data by value (meaning the data structure is copied directly into queue memory, allowing the sender to overwrite its local copy safely). Learn about blocking on queues (tasks sleep when queue is full/empty).
- **Practical Activity:**
  - Learn how to initialize message queues using `xQueueCreate()`.
  - Implement a producer-consumer system where Task A packs sensor data into a C struct and sends it via `xQueueSend()`, and Task B unpacks it using `xQueueReceive()`.
- **Code/Circuit Reference:**

```

#include "queue.h"

typedef struct {
 uint32_t id;
 uint32_t val;
} packet_t;

QueueHandle_t xPacketQueue = NULL;

void vProducer(void *pvParameters) {
 packet_t pkt = {1, 100};
 while (1) {
 // Send by value (copies structure into queue memory)
 xQueueSend(xPacketQueue, &pkt, portMAX_DELAY);
 vTaskDelay(pdMS_TO_TICKS(500));
 }
}

```

## Tuesday: Queue Sets (Multiplexing Multiple Queues)

- **Conceptual Objective:** Understand that in complex systems, a single receiver task may need to monitor multiple independent queues (e.g., a CAN queue, a UART queue, and an ADC queue) without locking. Study **Queue Sets**—a mechanism allowing a task to block on multiple queues simultaneously.
- **Practical Activity:**
  - Initialize a queue set using `xQueueCreateSet()`.
  - Map multiple queue handles to the set and build a multiplexed polling loop.
- **Code/Circuit Reference:**

```

QueueSetHandle_t xQueueSet = xQueueCreateSet(20);
xQueueAddToSet(xQueueA, xQueueSet);
xQueueAddToSet(xQueueB, xQueueSet);

void vReceiverTask(void *pvParameters) {
 while (1) {
 // Blocks on any queue in the set having data!
 QueueSetMemberHandle_t active = xQueueSelectFromSet(xQueueSet, portMAX_DELAY);
 if (active == xQueueA) {
 // Process queue A
 }
 }
}

```

### Wednesday: Event Groups (Bitmask Multi-Event Coordination)

- **Conceptual Objective:** Study Event Groups—kernel synchronization blocks containing a bitmask of flags. Unlike semaphores (which trigger one task at a time), Event Groups allow tasks to block waiting for multiple asynchronous events to occur simultaneously (AND condition) or any single event to occur (OR condition).
- **Practical Activity:**
  - Initialize an event group using `xEventGroupCreate()`.
  - Write a system where a controller task blocks until three independent initialization tasks (Timer initialized, UART initialized, Flash initialized) set their respective bits.
- **Code/Circuit Reference:**

```

#include "event_groups.h"

#define BIT_TIMER_INIT (1 << 0)
#define BIT_UART_INIT (1 << 1)

EventGroupHandle_t xInitEvents = NULL;

void wait_for_init(void) {
 // Wait for BOTH bits to be set
 xEventGroupWaitBits(
 xInitEvents,
 BIT_TIMER_INIT | BIT_UART_INIT,
 pdTRUE, // Clear on exit
 pdTRUE, // Wait for ALL bits (AND)
 portMAX_DELAY
);
}

```

## Thursday: Lightweight Task Notifications (High-Speed Signaling)

- **Conceptual Objective:** Master Task Notifications. Understand that every task has a 32-bit notification value built directly into its TCB. Because there is no separate queue or semaphore structure in memory, using Task Notifications to unblock a task has up to **45% lower latency** and uses **0 bytes of extra SRAM** compared to semaphores.
- **Practical Activity:**
  - Study task notifications APIs: `xTaskNotifyGive()`, `ulTaskNotifyTake()`, `xTaskNotify()`.
  - Rewrite a binary semaphore signaling system using Task Notifications and document footprint improvements.
- **Code/Circuit Reference:**

```

TaskHandle_t xReceiverTaskHandle = NULL;

void vSender(void *pvParameters) {
 while (1) {
 // Send notification directly to target task
 xTaskNotifyGive(xReceiverTaskHandle);
 vTaskDelay(pdMS_TO_TICKS(100));
 }
}

void vReceiver(void *pvParameters) {
 while (1) {
 // Wait for notification atomically
 uint32_t count = ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
 if (count > 0) {
 // Process event ...
 }
 }
}

```

## Friday: Designing Communication Topologies

- **Conceptual Objective:** Master architectural design configurations. Differentiate:
- **Message Queues:** Best for data transfer of structured frames.
- **Event Groups:** Best for state coordination and task barriers.
- **Notifications:** Best for ultra-low latency, one-to-one task signaling.
- **Practical Activity:**
  - Map out an industrial IoT gateway communication topology.
  - Present it in a block diagram.



## Hands-On Assignment: Multi-Sensor Telemetry Concentrator

### Functional Requirements

Create a fully modular FreeRTOS application that implements a Multi-Sensor Telemetry Concentrator routing structured data through message queues and coordinating startups via event groups. 1. Define a packed structure representing a Telemetry Frame containing a Sensor ID, Type, value, and timestamp. 2. Implement three tasks: - **vTempSensorTask (Priority 2):** Reads simulated temperature data every 150ms and writes Telemetry Frames to a shared Message Queue. - **vPressureSensorTask (Priority 2):** Reads simulated pressure data every 250ms and writes to the



same Queue. - `vConcentratorTask` (**Priority 3**): Blocks on the Message Queue, extracts packets, converts types, and prints raw details to the console. 3. Coordinate startup: before any sensor task starts writing data, they must wait for an **Event Group** signal. A dedicated `vSystemInitializer` task must run, perform virtual checks (delays), set the "System Ready" bits in the Event Group, and then delete itself. 4. Integrate a hardware tick interrupt simulator. Every 1 second, the interrupt must send a **Task Notification** directly to the `vConcentratorTask` to trigger an output of total packets processed statistics. 5. Provide a Makefile that builds the program cleanly.

## Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):

```

#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "event_groups.h"
#include <stdio.h>

#define BIT_SYS_READY (1 << 0)
#define BIT_CONC_READY (1 << 1)

typedef struct {
 uint32_t sensor_id;
 float value;
 uint32_t timestamp;
} telemetry_t;

QueueHandle_t xTelemetryQueue = NULL;
EventGroupHandle_t xStartEvents = NULL;
TaskHandle_t xConcentratorHandle = NULL;

void vSystemInitializer(void *pvParameters) {
 printf("[INIT] Starting virtual system checks ... \n");
 vTaskDelay(pdMS_TO_TICKS(1000)); // Simulate hardware startup
 printf("[INIT] Hardware OK. Signaling Sensor Tasks.\n");

 // Set bits in Event Group
 xEventGroupSetBits(xStartEvents, BIT_SYS_READY | BIT_CONC_READY);
 vTaskDelete(NULL); // Self terminate
}

void vTempSensorTask(void *pvParameters) {
 // Wait for initialization
 xEventGroupWaitBits(xStartEvents, BIT_SYS_READY | BIT_CONC_READY, pdFALSE, pdTRUE,
portMAX_DELAY);

 telemetry_t frame = { 101, 25.0f, 0 };
 while (1) {
 frame.value = 20.0f + ((float)(rand() % 100) / 10.0f);
 frame.timestamp = xTaskGetTickCount();
 xQueueSend(xTelemetryQueue, &frame, portMAX_DELAY);
 vTaskDelay(pdMS_TO_TICKS(150));
 }
}

```

```

void vConcentratorTask(void *pvParameters) {
 xEventGroupWaitBits(xStartEvents, BIT_SYS_READY | BIT_CONC_READY, pdFALSE, pdTRUE,
portMAX_DELAY);

 telemetry_t rx_frame;
 uint32_t packets_count = 0;

 while (1) {
 // Check for direct interrupt notifications with a 10ms timeout
 uint32_t notify_val = ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(10));
 if (notify_val > 0) {
 printf("\n--- DIAGNOSTICS: Total Packets Processed: %u ---\n\n",
packets_count);
 }

 // Read queue
 if (xQueueReceive(xTelemetryQueue, &rx_frame, pdMS_TO_TICKS(100)) == pdTRUE) {
 packets_count++;
 printf("[CONCENTRATOR] Received from Sensor %u | Value: %.2f | Time:
%u\n",
 rx_frame.sensor_id, rx_frame.value, rx_frame.timestamp);
 }
 }
}

```

## Deliverables

- [ ] Source file `main.c` containing tasks.
- [ ] Compilation Makefile.
- [ ] Console output report `telemetry_run.log` showing startup synchronization and continuous queue processing prints.

## Self-Check Questions

1. Why does a Message Queue copy data **by value**? What are the advantages and disadvantages of this compared to passing data **by reference** (pointers)?
2. How do **Queue Sets** enable a task to monitor multiple independent queues? Why is this preferred over polling queues sequentially?

3. Explain the difference between `xQueueSend()` and `xQueueSendFromISR()`. Why can you not use standard FreeRTOS APIs inside an ISR?
  4. How do **Task Notifications** achieve lower execution latency and lower SRAM footprint compared to standard binary semaphores?
  5. Walk through the parameter configurations of `xEventGroupWaitBits()`. What does setting `xClearOnExit` to `pdTRUE` accomplish?
- 

## Stretch Task (Optional)

---

Modify the Concentrator to use a **Queue of Pointers** to pass large data structures dynamically, and write a thread-safe memory manager that handles memory allocations and stack balances cleanly.

## Motivation Checkpoint

---

Inside real-world embedded architectures, peripherals run asynchronously: an accelerometer sends data on SPI, a button triggers an interrupt, and a GPS module streams strings. Operating systems sync these streams using Message Queues and low-overhead Task Notifications. Mastering Inter-Task Communications is the baseline key to system integration.

## Trainer Review Rule

---

### • Pass Criteria:

- Telemetry concentrator starts only after the Event Group initialization bits are set.
- Telemetry structs pass cleanly without data corruption.
- Direct Task Notifications are processed correctly.

### • Fail Criteria:

- Hard-coded busy loops inside receiving queues.
- Dynamic malloc allocations inside task loops without safety guards.

# Weekly Execution Plan: Week 23

---

## Weekly Goal

---

Master low-level dynamic memory allocation inside Real-Time Operating Systems, analyze the exact differences between the five FreeRTOS allocation models ( `heap_1` to `heap_5` ), configure stack overflow detection guards, and profile heap limits.

## Weekly Outcome

---

By Friday, the student will have built a **Memory Sentinel Logger Utility** under FreeRTOS. They will have configured dynamic heap boundaries, resolved stack overflow hook triggers, and structured diagnostic routines that monitor and print available heap bytes and stack high-water marks in real-time.

---

## 17 Session Breakdown (Monday - Friday)

---

### Monday: The FreeRTOS Dynamic Heap Schemes ( `heap_1` & `heap_2` )

- **Conceptual Objective:** Master the fundamental mechanics of FreeRTOS memory schemes:
- `heap_1` : The simplest model. Only allows allocation ( `pvPortMalloc` ); once memory is allocated, it **cannot be freed**. This is the most secure and deterministic model, universally used in high-reliability firmwares where dynamic freeing is banned.
- `heap_2` : Allows allocation and freeing, but does not merge (coalesce) adjacent free memory blocks. This is susceptible to severe fragmentation.
- **Practical Activity:**
  - Compile a project using `heap_1` and verify that any attempt to free memory has no effect.
  - Inspect memory layout registers in a debug map.
- **Code/Circuit Reference:**

```
// heap_1.c core allocation concept:
// Moves a simple index offset along a static array.
// No free mechanism exists: vPortFree is an empty stub.
void vPortFree(void *pv) {
 (void) pv;
 // Banned operation!
}
```

## Tuesday: Advanced Heap Allocators ( `heap_3` & `heap_4` )

- **Conceptual Objective:** Master the standard dynamic allocators:
- `heap_3` : A simple wrapper around standard C library `malloc` and `free` . It is non-deterministic and requires the compiler to set up linker heap spaces.
- `heap_4` : The most widely used allocator. It manages a static byte array, allocates memory chunks, and automatically **coalesces (merges)** adjacent free blocks to prevent fragmentation. It is highly robust.
- **Practical Activity:**
- Compile a project using `heap_4` .
- Call `xPortGetFreeHeapSize()` and `xPortGetMinimumEverFreeHeapSize()` to trace memory metrics.
- **Code/Circuit Reference:**

```
// Trace active heap metrics in heap_4
size_t free_bytes = xPortGetFreeHeapSize();
size_t min_free = xPortGetMinimumEverFreeHeapSize(); // High water mark of memory allocation
```

## Wednesday: Multi-Segment Heap Allocations via `heap_5`

- **Conceptual Objective:** Study `heap_5` . Differentiate it from `heap_4` : while `heap_4` only allows allocating from a single static array in SRAM, `heap_5` can span **multiple independent memory segments** across different physical RAM blocks (e.g., SRAM1, external SDRAM, CCM RAM) using `vPortDefineHeapRegions()` .
- **Practical Activity:**
- Construct a C region map defining two independent RAM segments.
- Initialize `heap_5` using the region array.
- **Code/Circuit Reference:**

```
#include "FreeRTOS.h"

// Define independent memory regions
const HeapRegion_t xHeapRegions[] = {
 { (uint8_t*)0x20000000UL, 0x10000 }, // Region 1: 64KB in SRAM1
 { (uint8_t*)0x10000000UL, 0x08000 }, // Region 2: 32KB in CCM RAM
 { NULL, 0 } // Terminator
};

void init_heap_5(void) {
 // Must be called before any pvPortMalloc!
 vPortDefineHeapRegions(xHeapRegions);
}
```

## Thursday: Stack Overflow Detection Hooks

- **Conceptual Objective:** Master Stack Overflow mechanics. Every task has its own private stack in SRAM. If a task nests deeply or allocates large local variables, it grows past its stack limits, corrupting adjacent TCB or task memory. Study the two FreeRTOS stack overflow check methods:
- **Method 1:** Checks if the Stack Pointer grew past limits during context switches (fast, misses overflows occurring in the middle of task runs).
- **Method 2:** Writes a dummy pattern at the end of the stack at task creation, and checks if the pattern is corrupted during context switches (slower, highly reliable).
- **Practical Activity:**
  - Enable Method 2 in your configurations.
  - Write the stack overflow hook handler.
- **Code/Circuit Reference:**

```
// Inside FreeRTOSConfig.h:
#define configCHECK_FOR_STACK_OVERFLOW 2

// The application hook function executed automatically on overflow detection
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
 // Trap execution! Safe diagnostic shutdown.
 printf("CRITICAL ERROR: Stack Overflow in task: %s\n", pcTaskName);
 while (1);
}
```

## Friday: Stack High-Water Mark Profiling

- **Conceptual Objective:** Learn how to profile task stack usage dynamically. The API `uxTaskGetStackHighWaterMark()` returns the minimum number of free stack bytes ever remaining in a task since it started running.
- **Practical Activity:**
  - Write a periodic diagnostics task.
  - Poll and print stack high-water marks for all running tasks.
- **Code/Circuit Reference:**

```
void vDiagnosticsTask(void *pvParameters) {
 while (1) {
 // High-water mark return: closer to 0 means closer to overflow!
 UBaseType_t high_water = uxTaskGetStackHighWaterMark(NULL);
 printf("[DIAG] Stack High Water Mark: %u words\n", (unsigned int)high_water);
 vTaskDelay(pdMS_TO_TICKS(5000));
 }
}
```

---

## Hands-On Assignment: RTOS Memory Sentinel & Leak Auditor

---

### Functional Requirements

Create a highly robust FreeRTOS application that implements a Memory Sentinel Logger, validating stack bounds and heap metrics under dynamic memory stresses.

1. Configure `FreeRTOSConfig.h` to enable:
  - `configCHECK_FOR_STACK_OVERFLOW` set to `2`.
  - `configSUPPORT_DYNAMIC_ALLOCATION` set to `1`.
2. Compile your application using the `heap_4.c` memory allocator.
3. Spawn three tasks:
  - **`vDynamicWorkerTask` (Priority 2):** Dynamically allocates byte chunks of varying sizes (\$100\dots 500\$ bytes) using `pvPortMalloc()`, writes dummy values, and frees them using `vPortFree()`.
  - **`vRecursiveStackTask` (Priority 1):** Executes a recursive calculation function. On each loop, increment local stack usage. This is designed to intentionally trigger a stack overflow for validation.
  - **`vSentinelTask` (Priority 3):** Periodically (every 1 second) reads and prints: current free heap bytes, the minimum ever free heap bytes since boot, and the stack high-water marks of both worker tasks.
4. Implement the `vApplicationStackOverflowHook` handler.
5. Demonstrate the sentinel logging heap parameters cleanly on the console. Then, trigger the stack overflow task and verify that the system is safely trapped and logs the exact name of the overflowing task before halting.

### Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):



```

#include "FreeRTOS.h"
#include "task.h"
#include <stdio.h>

TaskHandle_t xDynamicWorkerHandle = NULL;
TaskHandle_t xStackWorkerHandle = NULL;

// Recursive function to force stack overflow
uint32_t recursive_sum(uint32_t n) {
 uint32_t large_array[16] = {0}; // Allocates 64 bytes on stack per call
 large_array[0] = n;
 if (n == 0) return 0;
 return large_array[0] + recursive_sum(n - 1);
}

void vRecursiveStackTask(void *pvParameters) {
 vTaskDelay(pdMS_TO_TICKS(3000)); // Let sentinel run first
 printf("[STACK WORKER] Starting recursive stack stress test...\n");
 recursive_sum(100); // Will trigger overflow hook!
 vTaskDelete(NULL);
}

void vDynamicWorkerTask(void *pvParameters) {
 while (1) {
 size_t size = 128 + (rand() % 256);
 void *ptr = pvPortMalloc(size);
 if (ptr != NULL) {
 // Simulate processing
 vTaskDelay(pdMS_TO_TICKS(50));
 vPortFree(ptr);
 }
 vTaskDelay(pdMS_TO_TICKS(100));
 }
}

void vSentinelTask(void *pvParameters) {
 while (1) {
 size_t free_heap = xPortGetFreeHeapSize();
 size_t min_heap = xPortGetMinimumEverFreeHeapSize();

 UBaseType_t dyn_stack = uxTaskGetStackHighWaterMark(xDynamicWorkerHandle);
 }
}

```

```

 printf("[SENTINEL] Free Heap: %zu bytes | Min Ever Free: %zu bytes\n",
free_heap, min_heap);
 printf("[SENTINEL] Dynamic Task Stack High-Water: %u words\n", (unsigned
int)dyn_stack);

 vTaskDelay(pdMS_TO_TICKS(1000));
 }
}

void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
 // Trapped execution safely
 printf("\nCRITICAL EMERGENCY: STACK OVERFLOW DETECTED!\n");
 printf("Offending Task: %s\n", pcTaskName);
 while (1); // Safe halt
}

```

## Deliverables

- [ ] Source file `main.c` containing tasks.
- [ ] Compilation Makefile.
- [ ] Console output log `memory_sentinel.log` showing:
  - Stable sentinel metrics of heap usage.
  - The exact crash traceback sequence showing the Stack Overflow Hook catching and trapping the `vRecursiveStackTask` overflow event.

## ? Self-Check Questions

1. Differentiate the memory behaviors of `heap_1` vs `heap_4` . Under what security parameters is `heap_1` mandatory inside flight controller firmwares?
2. What is the cause of **Heap Fragmentation**? How does `heap_4` resolve this issue compared to `heap_2` ?
3. How does `heap_5` initialize multi-segment memory? Write a region structure mapping 32KB RAM at `0x20000000` and 64KB RAM at `0x10000000` .
4. Walk through the physical operations of the **Stack Overflow Check Method 2** inside FreeRTOS. What bytes are written at stack boundaries and when are they verified?
5. What is the unit of size returned by `uxTaskGetStackHighWaterMark()` ? (Hint: is it in bytes or `StackType_t` words?).



## Stretch Task (Optional)

Implement an **Out-Of-Memory Recovery Hook** (`vApplicationMallocFailedHook`) that automatically checks for dynamic allocation failures, frees cached telemetry buffers, and re-allocates memory safely without crashing the system.



## Motivation Checkpoint

A major cause of microcontroller failures inside autonomous devices is stack overflows. Because microcontrollers do not have physical MMU hardware to isolate memory pages, a stack overflow on one task silently overwrites global configurations, causing random crashes that are impossible to debug without proper diagnostic tracking hooks.



## Trainer Review Rule

- **Pass Criteria:**

- Memory sentinel log runs cleanly with heap tracking.
- Stack overflow hook successfully catches the recursive overflow and prints the exact task name before halting.
- Zero compiler warnings.

- **Fail Criteria:**

- Use of standard library `malloc()` or `free()` inside FreeRTOS tasks.
- Missing stack overflow hook registrations.

# Weekly Execution Plan: Week 24

---

## Weekly Goal

---

Master FreeRTOS Software Timers, understand the exact mechanics of ISR-to-task communication utilizing safe `FromISR` API variants, implement deferred interrupt processing, and execute real-time execution profiling.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Deferred Interrupt Debouncer & Timer Controller** using FreeRTOS. They will understand how to write safe ISR handlers that communicate with tasks without locking the kernel scheduler, how to configure software timers (one-shot vs auto-reload), and how to complete the Phase 4 Milestone Challenge.

---

## 17 Session Breakdown (Monday - Friday)

---

### Monday: FreeRTOS Software Timers (One-Shot vs Auto-Reload)

- **Conceptual Objective:** Master Software Timers—timers managed by the RTOS kernel daemon task ( `prvTimerTask` ). Differentiate between:
  - **One-Shot Timers:** Execute their callback function exactly once when they expire, and stop.
  - **Auto-Reload Timers:** Automatically restart their timing period every time they expire, generating periodic callbacks.
- **Practical Activity:**
  - Learn how to initialize software timers using `xTimerCreate()` and start/stop them using `xTimerStart()`.
  - Differentiate timer callback constraints (callbacks execute inside the timer daemon task context and must never block or call blocking APIs).
- **Code/Circuit Reference:**

```

#include "timers.h"

TimerHandle_t xAutoTimer = NULL;

void vTimerCallback(TimerHandle_t xTimer) {
 // Executes in the daemon task context.
 // Banned: vTaskDelay or any blocking queue take!
 toggle_pin();
}

void init_timers(void) {
 xAutoTimer = xTimerCreate(
 "Periodic_Timer",
 pdMS_TO_TICKS(1000), // 1 second period
 pdTRUE, // Auto-Reload
 (void*)0, // Timer ID
 vTimerCallback
);
 xTimerStart(xAutoTimer, 0);
}

```

## Tuesday: The Timer Daemon Task Configuration

- **Conceptual Objective:** Study how software timers are scheduled. Software timers are not hardware interrupts; they are managed by the kernel's **Timer Service/Daemon Task** (`prvTimerTask`). Master configurations inside `FreeRTOSConfig.h`: `configUSE_TIMERS`, `configTIMER_TASK_PRIORITY`, and `configTIMER_QUEUE_LENGTH`.
- **Practical Activity:**
  - Deep-dive into `timers.c` to locate and analyze the timer command queue mechanism.
  - Differentiate why the timer task priority must normally be set high to prevent timing delays.
- **Code/Circuit Reference:**

```

/* FreeRTOSConfig.h Timer settings */
#define configUSE_TIMERS 1
#define configTIMER_TASK_PRIORITY (configMAX_PRIORITIES - 1) // Set High
#define configTIMER_QUEUE_LENGTH 10
#define configTIMER_TASK_STACK_DEPTH (configMINIMAL_STACK_SIZE * 2)

```

## Wednesday: ISR-to-Task Communication & FromISR APIs

- **Conceptual Objective:** Master the strict segregation between Task execution context and Interrupt execution context. Standard FreeRTOS APIs (like `xQueueSend` or `xSemaphoreGive`) are banned inside an ISR because they can cause the scheduler to switch contexts in the middle of an interrupt, leading to immediate system crashes. You must exclusively use **FromISR API variants** (e.g., `xQueueSendFromISR`, `xSemaphoreGiveFromISR`).
- **Practical Activity:**
  - Study `FromISR` signatures.
  - Differentiate the role of the `pxHigherPriorityTaskWoken` parameter.
- **Code/Circuit Reference:**

```
// Sending to a queue safely inside an ISR
void EXTI0_IRQHandler(void) {
 BaseType_t xHigherPriorityTaskWoken = pdFALSE;

 if (*EXTI_PR & (1U << 0)) {
 uint32_t val = 42;
 // FromISR call with wake tracking
 xQueueSendFromISR(xTelemetryQueue, &val, &xHigherPriorityTaskWoken);

 // Clear pending interrupt flag
 *EXTI_PR = (1U << 0);

 // Force a context switch if a higher priority task was unblocked
 portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
 }
}
```

## Thursday: Deferred Interrupt Processing (Bottom-Half Design)

- **Conceptual Objective:** Understand that to maintain a responsive real-time system, code inside ISRs must be minimal (top-half: save state, clear flag, signal task). The actual, slow processing of the interrupt data must be deferred (bottom-half) to a high-priority task unblocked dynamically by the ISR using a Task Notification or Semaphore.
- **Practical Activity:**
  - Write a button interrupt handler that defers processing.
  - The ISR sends a task notification, and a high-priority task unblocks instantly to execute the debounce logic.
- **Code/Circuit Reference:**

```
// ISR (Top-half): Simple, ultra-fast signaling
void EXTI15_10_IRQHandler(void) {
 BaseType_t xHigherPriorityTaskWoken = pdFALSE;
 vTaskNotifyGiveFromISR(xDeferredTaskHandle, &xHigherPriorityTaskWoken);
 portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

## Friday: Real-Time Execution Profiling & Milestone 4 Setup

- **Conceptual Objective:** Study real-time execution analysis. Learn how to configure runtime statistics ( `configGENERATE_RUN_TIME_STATS` ) to measure exactly how much CPU time in percent each task consumed. Complete the Phase 4 Milestone Challenge.
- **Practical Activity:**
  - Configure a high-speed hardware timer to act as the runtime stats clock.
  - Print the execution time table to the UART console.
- **Code/Circuit Reference:**

```
// Print human-readable runtime table
char stats_buffer[512];
vTaskGetRunTimeStats(stats_buffer);
printf("%s\n", stats_buffer);
```

## Hands-On Assignment: Deferred Interrupt Debouncer & Software Timer Controller

### Functional Requirements

Create a highly robust, bare-metal and FreeRTOS application that implements a Deferred Interrupt Button Debouncer and coordinates LED indicators using Software Timers.

1. Configure **EXTIO** linked to Button PA0 to generate a hardware rising-edge interrupt.
2. Implement `EXTI0_IRQHandler`. The ISR must be minimal: send a direct **Task Notification** to `vDeferredDebounceTask` and yield using `portYIELD_FROM_ISR`.
3. Implement `vDeferredDebounceTask` (**Priority 3 - High**):
  - Blocks on `ulTaskNotifyTake(pdTRUE, portMAX_DELAY)`.
  - Upon unblocking, start a 50ms **One-Shot Software Timer** (`xDebounceTimer`) to simulate debounce wait time.
4. Implement the `xDebounceTimer` **Callback**:
  - Executes 50ms after the physical button press.
  - Verifies the physical PA0 pin state. If the pin is still High, toggle LED PD12 and toggle a periodic **Auto-Reload Software Timer** (`xLedAlertTimer`) that blinks PD13 continuously.
5. In `main.c`:
  - Initialize FreeRTOS software timers.
  - Enable EXTIO interrupts in the NVIC.
  - Run the scheduler.
6. Verify deferred response timings: capture the signal delay on your logic analyzer to prove the 50ms debounce gap.

## Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):



```

#include "FreeRTOS.h"
#include "task.h"
#include "timers.h"
#include <stdio.h>

TaskHandle_t xDeferredDebounceHandle = NULL;
TimerHandle_t xDebounceTimer = NULL;
TimerHandle_t xLedAlertTimer = NULL;

void vDebounceTimerCallback(TimerHandle_t xTimer) {
 // 50ms has elapsed. Verify pin PA0 state.
 volatile uint32_t *idr = (volatile uint32_t*)0x40020010U;
 if (*idr & (1U << 0)) { // Button still pressed
 printf("[TIMER] Debounced Button Press Confirmed! Toggling Alert Timer.\n");
 // Start or Stop the periodic alert timer
 if (xTimerIsTimerActive(xLedAlertTimer)) {
 xTimerStop(xLedAlertTimer, 0);
 ((volatile uint32_t)0x40020C18U) = (1U << (13 + 16)); // Turn off PD13
 } else {
 xTimerStart(xLedAlertTimer, 0);
 }
 }
}

void vLedAlertCallback(TimerHandle_t xTimer) {
 // Toggles PD13 every 500ms
 volatile uint32_t *odr = (volatile uint32_t*)0x40020C14U;
 *odr ^= (1U << 13);
}

void vDeferredDebounceTask(void *pvParameters) {
 while (1) {
 // Block until ISR signals button press
 ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
 printf("[DEFERRED TASK] ISR received. Launching 50ms software debounce timer...\n");
 xTimerStart(xDebounceTimer, 0);
 }
}

void EXTI0_IRQHandler(void) {
 BaseType_t xHigherPriorityTaskWoken = pdFALSE;

```

```
volatile uint32_t *pr = (volatile uint32_t*)0x40013C14U;

if (*pr & (1U << 0)) {
 // Signal the deferred task instantly
 vTaskNotifyGiveFromISR(xDeferredDebounceHandle, &xHigherPriorityTaskWoken);

 *pr = (1U << 0); // Clear pending bit
 portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
}
```

---

## Deliverables

- [ ] Updated `startup.c` with the `EXTI0_IRQHandler` vector hook.
  - [ ] Application source code `main.c`.
  - [ ] Compilation Makefile.
  - [ ] Console output report `deferred_timers.log` showing:
    - Startup logs.
    - Software debounce confirmation logs.
    - Periodic alert timer starts and stops.
- 

## ? Self-Check Questions

1. Why are standard blocking APIs (like `vTaskDelay`) strictly banned inside Software Timer Callback functions? What happens to the system if a callback blocks?
  2. Trace the parameter `pxHigherPriorityTaskWoken` inside `FromISR` API calls. What does it physically do, and why is `portYIELD_FROM_ISR()` required?
  3. Differentiate between a **Top-Half** and a **Bottom-Half** interrupt design. Why is this model mandatory for high-performance, real-time operating systems?
  4. What is the role of the Timer Service/Daemon Task (`prvTimerTask`)? Which priority should it hold compared to other tasks?
  5. How does a one-shot software timer physically differ from an auto-reload software timer?
-



## Stretch Task (Optional)

---

Configure FreeRTOS **Runtime Stats** using a custom timer. Implement a terminal command parsing interface that dynamically prints task execution runtime percentages on a UART console when requested.



## Motivation Checkpoint

---

High-reliability real-time firmwares require absolute timing deterministic parameters. When an critical interrupt fires, the CPU must exit the hardware handler as fast as possible to avoid blocking other hardware lines. Deferred interrupt processing using `FromISR` Task Notifications is the universal design pattern of elite firmware engineers.



## Trainer Review Rule

---

- **Pass Criteria:**

- `EXTI0_IRQHandler` only triggers notifications, containing no loops or prints.
- Software debounce timer accurately waits 50ms before evaluating button state.
- Periodic alert timers trigger stable blinks.

- **Fail Criteria:**

- Use of raw delay functions inside ISRs or Callback functions.
- Missing interrupt pending cleans.

# Weekly Execution Plan: Week 25

---

## Weekly Goal

---

Master the high-level architecture of Embedded Linux systems, understand the complete, multi-stage hardware boot sequence from power-on reset to user-space main application launch, and analyze U-Boot bootloader configurations.

## Weekly Outcome

---

By Friday, the student will have dissected a physical **Linux Boot Log** captured from a serial console. They will be able to explain the roles of ROM Boot, SPL (Secondary Program Loader), U-Boot, Kernel unpacking, Device Tree loading, and the transitioning of control to the `init` process in rootfs.

---



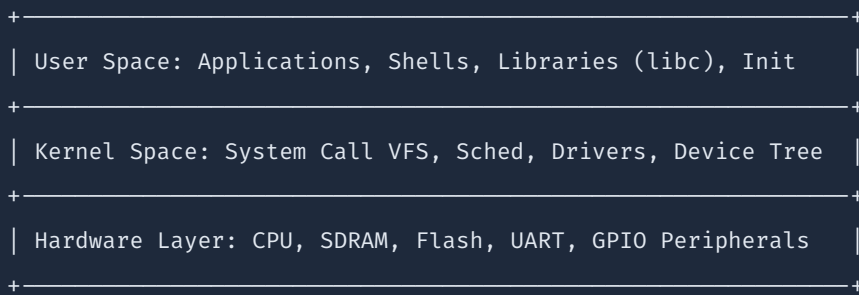
## Session Breakdown (Monday – Friday)

---

### Monday: The Four Pillars of Embedded Linux

- **Conceptual Objective:** Understand that unlike microcontrollers (where code is compiled into a single binary running on bare-metal), an Embedded Linux system is composed of four distinct, independent software blocks:
- **Toolchain:** The compiler suite targeting the specific hardware architecture.
- **Bootloader:** Initializes memory and boots the OS.
- **Linux Kernel:** Manages processes, virtual memory, and hardware drivers.
- **Root Filesystem (Rootfs):** Contains user-space libraries, configurations, and applications.
- **Practical Activity:**
  - Sketch a block diagram showing how these four components interact.
  - Define the architectural boundaries of each component.
- **Code/Circuit Reference:**

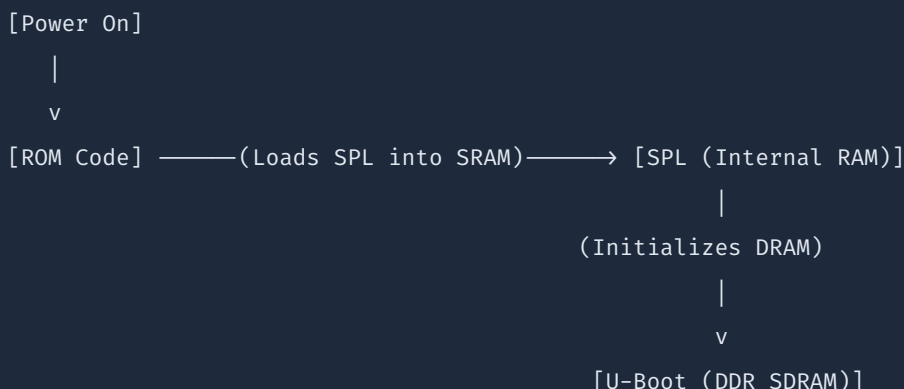
Embedded Linux System Stack:



## Tuesday: The Multi-Stage Boot Flow (ROM to U-Boot)

- **Conceptual Objective:** Master the exact boot stages of an embedded processor (e.g., ARM Cortex-A):
- **Stage 1 (ROM Code):** Hardcoded in silicon. Powers on, initializes basic clocks, searches for boot media (SD card, eMMC, NAND), loads the next stage into internal SRAM, and runs it.
- **Stage 2 (SPL / Secondary Program Loader):** A minimal bootloader (like U-Boot SPL). Initializes DRAM memory, loads the full bootloader (U-Boot) from flash/SD into DRAM, and jumps to it.
- **Stage 3 (Full Bootloader - U-Boot):** Fully initializes buses, USB, Ethernet. Provides an interactive command line shell. Loads the Linux Kernel ( `uImage` or `zImage` ) and the Device Tree Blob ( `.dtb` ) into RAM.
- **Practical Activity:**
  - Study U-Boot configuration setups.
  - Draw the step-by-step memory relocation maps of U-Boot.
- **Code/Circuit Reference:**

Multi-Stage Boot Relocation:



## Wednesday: U-Boot Shell Operations & Environment Control

- **Conceptual Objective:** Learn how to operate the interactive U-Boot serial console. Master core U-Boot commands: listing files ( `ext4ls` , `fatls` ), reading memory ( `md` , `mw` ), loading binaries into

RAM ( `fatload` , `tftp` ), and configuring environment parameters ( `printenv` , `setenv` , `saveenv` ). Understand the `bootcmd` environment variable.

- **Practical Activity:**

- Connect a serial console to your target board (Raspberry Pi/BBB).
- Interrupt the boot sequence to enter the U-Boot shell.
- Manually modify boot parameters to alter configurations.

- **Code/Circuit Reference:**

```
U-Boot Shell commands
List files on fat partition of MMC 0
fatls mmc 0:1

Load kernel zImage into DRAM offset 0x82000000
fatload mmc 0:1 0x82000000 zImage

Set custom boot command and save permanently
setenv bootcmd "fatload mmc 0:1 0x82000000 zImage; fatload mmc 0:1 0x88000000
board.dtb; bootz 0x82000000 - 0x88000000"
saveenv
```

## Thursday: The Kernel Initialization Phase & Device Trees

- **Conceptual Objective:** Study what the Linux Kernel does when U-Boot branches to its execution entry point ( `bootz` or `bootm` ). The kernel:
  - Unpacks itself in RAM.
  - Initializes virtual memory, page tables, and scheduler ticks.
  - Parses the **Device Tree Binary (DTB)** to locate and initialize hardware peripherals.
  - Mounts the **Root Filesystem (Rootfs)** specified by the `bootargs` variable.

- **Practical Activity:**

- Trace kernel init logs.
- Differentiate how `bootargs` directs the mount target (e.g., `root=/dev/mmcblk0p2 rootwait rw` ).

- **Code/Circuit Reference:**

```
Example bootargs configuration passed by U-Boot to Kernel
setenv bootargs "console=ttyS0,115200 root=/dev/mmcblk0p2 rootwait rw earlyprintk"
```

## Friday: Entering User Space (The Init Process)

- **Conceptual Objective:** Understand the exact transition from Kernel space to User space. Once the kernel mounts the root filesystem, it spawns the first user-space process: `/sbin/init` (PID 1). Init reads system configuration tables (Systemd, SystemV init scripts, or busybox `inittab`), spawns system services, and launches login shell terminals.
- **Practical Activity:**
  - Analyze a complete, raw Linux boot log.
  - Isolate and highlight the exact transitions between ROM, U-Boot, Kernel init, and User-space Init launch.
- **Code/Circuit Reference:**

```
Trace output in serial boot log showing user space entry:
[3.456789] VFS: Mounted root (ext4 filesystem) on device 179:2.
[3.468200] devtmpfs: mounted
[3.475412] Freeing unused kernel memory: 1024K
[3.489020] Run /sbin/init as init process
Starting syslogd: OK
Starting klogd: OK
```

---

## Hands-On Assignment: Linux Boot Log Dissector & Analyzer

---

### Functional Requirements

Create a robust automated text-parsing pipeline using standard Linux scripting utilities ( `grep` , `sed` , `awk` ) to analyze a raw, 1000-line Linux Boot Log and generate a detailed system health diagnostic report. 1. The analysis tool must be a Bash script `dissect_boot.sh` parsing a raw boot log text file `boot.log` as a parameter. 2. The script must parse, isolate, and output: - **Bootloader Details:** Extract the U-Boot compile date, version number, and detected DRAM size. - **Kernel Details:** Extract the Linux Kernel version and the boot arguments ( `bootargs` command line). - **Peripheral Enumeration:** Scan and list all detected hardware peripherals (extract lines mapping I2C, SPI, UART, and MMC controllers initialization). - **Timing Latency Calculations:** Locate the timestamp of the first kernel log line and the timestamp of the `/sbin/init` user-space launch. Calculate the exact elapsed time (in seconds) the kernel took to boot. - **Failure & Warnings Audit:** Scan the log for any line containing `"fail"` , `"error"` , `"warning"` , or `"corrupt"` . Isolate them cleanly and list them under a warning block. 3. Output the parsed metrics cleanly to a report file `boot_analysis_report.txt` .

### Structural Requirements & Code Skeleton

`dissect_boot.sh` (Shell Skeleton):

```
#!/usr/bin/env bash
set -euo pipefail

if ["$#" -ne 1]; then
 echo "Usage: $0 <boot_log_file>"
 exit 1
fi

BOOT_LOG="$1"
REPORT="boot_analysis_report.txt"

echo "=== Embedded Linux Boot Log Analysis ===" > "$REPORT"
echo "Log File: $BOOT_LOG" >> "$REPORT"
echo "Timestamp: $(date)" >> "$REPORT"
echo "===== " >> "$REPORT"

1. Extract U-Boot Metadata
echo "[1] Bootloader Info:" >> "$REPORT"
grep -i "U-Boot" "$BOOT_LOG" | head -n 2 >> "$REPORT" || echo "No U-Boot signature" >> "$REPORT"
grep -iE "dram:|ram:|memory:" "$BOOT_LOG" | head -n 1 >> "$REPORT" || true
echo "-----" >> "$REPORT"

2. Extract Kernel Version & Bootargs
echo "[2] Linux Kernel Info:" >> "$REPORT"
grep -i "Linux version" "$BOOT_LOG" | head -n 1 >> "$REPORT" || echo "No kernel version found" >> "$REPORT"
grep -i "Kernel command line" "$BOOT_LOG" | head -n 1 >> "$REPORT" || true
echo "-----" >> "$REPORT"

3. Calculate Boot Latency using awk
echo "[3] Kernel Boot Latency:" >> "$REPORT"
awk '
BEGIN { first = -1; last = -1; }
/^[[]*[0-9]+\.[0-9]+/ {
 split($1, parts, "[.]");
 val = parts[2];
 if (first == -1) first = val;
 last = val;
}
END {
 if (first != -1 && last != -1) {
```



```

 printf "First Kernel Log: %.6f seconds\n", first;
 printf "Last Kernel Log (User space entry): %.6f seconds\n", last;
 printf "Total Kernel Execution Duration: %.6f seconds\n", (last - first);
}
}
' "$BOOT_LOG" >> "$REPORT"
echo "_____ " >> "$REPORT"

4. Extract Peripherals
echo "[4] Detected Hardware Peripherals:" >> "$REPORT"
grep -iE 'i2c|spi|uart|mmc' "$BOOT_LOG" | grep -vE 'command line' | sort -u | head -n
15 >> "$REPORT" || true
echo "_____ " >> "$REPORT"

5. Warnings and Errors
echo "[5] Warnings & Errors Detected during Boot:" >> "$REPORT"
grep -inE 'fail|error|warning|corrupt' "$BOOT_LOG" | head -n 20 >> "$REPORT" || echo
"No warnings detected" >> "$REPORT"

cat "$REPORT"

```

---

## Deliverables

- [ ] Automated script `dissect_boot.sh`.
  - [ ] Sample boot log file `boot.log` containing raw console logs.
  - [ ] Generated diagnostics output report `boot_analysis_report.txt`.
- 

## ? Self-Check Questions

---

1. Why does an embedded processor require a multi-stage boot process (ROM -> SPL -> U-Boot)? Why can ROM code not load the kernel directly?
2. What is the role of the U-Boot environment variable `bootcmd`? How does it differ from the `bootargs` variable?
3. How does the Linux Kernel locate hardware peripherals during boot? What is the function of the Device Tree Blob ( `.dtb` ) in this process?
4. What is the first process launched in user-space by the kernel? What is its Process ID (PID)?
5. How can you capture serial boot logs physically from an embedded target? What hardware tools and configurations are required?



## Stretch Task (Optional)

Extend the boot log dissector to parse standard **systemd boot-up performance** logs ( `systemd-analyze plot` ) and generate a graphical timing chart isolating slow system startup scripts.



## Motivation Checkpoint

When an embedded product locks up, crashes, or fails to boot in the field, there is no monitor or debugger attached. You must diagnose failures strictly by connecting a serial debug line, dumping the raw console logs, and analyzing where the boot sequence broke down. Mastering bootlog diagnostics is a core skill for any systems developer.



## Trainer Review Rule

- **Pass Criteria:**

- Script successfully parses boot timelines.
- Latency math is calculated accurately using awk.
- Zero unhandled exit failures inside script logic.

- **Fail Criteria:**

- Direct shell crashes when parsing empty search fields.

# Weekly Execution Plan: Week 26

---

## Weekly Goal

---

Master cross-compilation mechanics, build a dedicated cross-compilation workspace, fetch mainline Linux kernel source code, compile a custom kernel and device trees from source, and boot it on your target hardware.

## Weekly Outcome

---

By Friday, the student will have compiled a custom **Linux Kernel Image** ( `zImage` / `Image` ) and matching **Device Tree Blob** ( `.dtb` ) for their target board (e.g., Raspberry Pi or BeagleBone) from source. They will have loaded it onto an SD card, connected a serial debug cable, and successfully booted their custom kernel to a command-line init prompt.

---

## Session Breakdown (Monday - Friday)

---

### Monday: The Cross-Compilation Paradigm

- **Conceptual Objective:** Master the differences between **Native Compilation** (compiling code on the host, for execution on the same host, e.g., x86 compiler on x86 PC) and **Cross-Compilation** (compiling code on the host, for execution on a completely different target architecture, e.g., compiling ARM64 binaries on an Intel x86 host). Study target triplets (e.g., `aarch64-linux-gnu-gcc` ).
- **Practical Activity:**
  - Set up cross-compilers on host:

```
sudo apt update && sudo apt install -y gcc-aarch64-linux-gnu gcc-arm-linux-gnueabi
bc u-boot-tools libncurses-dev libssl-dev bison flex.
```
  - Differentiate dynamic linkages inside cross-compiled objects using `file` and `readelf` .
- **Code/Circuit Reference:**

```
Verify cross-compiler toolchain
aarch64-linux-gnu-gcc --version

Differentiate dynamic libraries dependencies
aarch64-linux-gnu-readelf -d build/my_arm_app
```

## Tuesday: Fetching Kernel Sources & Defconfig Configurations

- **Conceptual Objective:** Understand that the Linux kernel is configurable: you can compile specific features (network protocols, filesystems, device drivers) directly into the kernel, or compile them as loadable modules ( `.ko` ), or strip them out to minimize binary footprint. Learn how configurations are managed via `.config` files and target presets ( `defconfig` ).

- **Practical Activity:**

- Clone the mainline stable Linux kernel source repository.
- Locate the default configurations directory ( `arch/arm/configs/` or `arch/arm64/configs/` ).
- Load the target default config (e.g., `bcm2711_defconfig` for Raspberry Pi 4).

- **Code/Circuit Reference:**

```
Clone Linux Kernel (shallow clone to save space)
git clone --depth 1 --branch rpi-6.1.y https://github.com/raspberrypi/linux.git
cd linux

Configure for Target Board (Raspberry Pi 4)
ARCH specifies target architecture; CROSS_COMPILE defines prefix
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2711_defconfig
```

## Wednesday: Modifying Kernels via `menuconfig`

- **Conceptual Objective:** Master interactive kernel configuration tools ( `menuconfig` ). Learn how to navigate the textual menu system to enable/disable kernel parameters, toggle drivers between built-in ( `[*]` ) and module ( `[M]` ) formats, and set custom local kernel version tags.

- **Practical Activity:**

- Open the interactive config screen using `make menuconfig`.
- Locate and enable support for custom USB serial devices.
- Append a personalized string to `CONFIG_LOCALVERSION`.

- **Code/Circuit Reference:**

```
Launch interactive configuration menu
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig

Inside menuconfig:
General setup → Local version - append to kernel release: "-siva-custom-v1"
Device Drivers → USB support → USB Serial Converter support → Enable FTDI
```

## Thursday: Compiling Kernel, Modules, and DTBs

- **Conceptual Objective:** Master the compilation commands of the Linux kernel. Understand what files are produced:
- **Image / zImage** : The compiled, uncompressed or compressed executable kernel binary.
- **Device Tree Blobs ( .dtb )**: The compiled hardware descriptions.
- **Kernel Modules ( .ko )**: Dynamic driver files compiled out-of-tree.
- **Practical Activity:**
  - Run multi-threaded kernel compilation.
  - Compile all device tree blobs.
- **Code/Circuit Reference:**

```
Compile Kernel, Device Trees, and Modules (using 8 parallel CPU cores)
make -j8 ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs

Locate compiled outputs
Kernel: arch/arm64/boot/Image
DTB: arch/arm64/boot/dts/broadcom/bcm2711-rpi-4-b.dtb
```

## Friday: Formatting Boot Media & First Target Boot

- **Conceptual Objective:** Learn how to partition and format flash media (SD Card) to boot your custom system. Partition into a FAT32 boot partition (holds U-Boot/SPL, kernel **Image** , and **.dtb** ) and an ext4 rootfs partition. Configure standard config boot parameters.
- **Practical Activity:**
  - Insert an SD card and create partitions using **fdisk** .
  - Copy compiled kernel **Image** and matching **bcm2711-rpi-4-b.dtb** to the boot partition.
  - Connect serial debug converter wires to the target board TX/RX pins, open serial minicom console on host, power on target, and watch your custom kernel boot cleanly.
- **Code/Circuit Reference:**

```
Mount SD card boot partition
sudo mount /dev/sdX1 /mnt/boot

Copy compiled outputs
sudo cp arch/arm64/boot/Image /mnt/boot/kernel8.img
sudo cp arch/arm64/boot/dts/broadcom/bcm2711-rpi-4-b.dtb /mnt/boot/bcm2711-rpi-4-b.dtb

Unmount cleanly to sync caches
sudo umount /mnt/boot
```

---

## Hands-On Assignment: Custom Compiled Kernel Boot & Log Verification

---

### Functional Requirements

Compile a custom Linux kernel from source, boot it on your target board, and verify your configurations inside the running terminal. 1. Download, configure, and compile a stable Linux kernel for your target board from source. 2. Customize `CONFIG_LOCALVERSION` to contain a custom string (e.g., `-custom-mentor-v1`). 3. Compile all required Device Tree Blobs. 4. Partition an SD card, copy the compiled kernel and DTB, and boot the target board. 5. Connect via Serial Debugger. Once booted, enter the terminal (or check boot parameters) and execute `uname -r` to verify your custom kernel version string is active. 6. Retrieve the active kernel config from `/proc/config.gz` (or `/boot/config-*`) and verify that your custom-enabled drivers are active.

### Structural Requirements & Code Skeleton

#### Kernel Verification Pipeline:

```
1. Check kernel release version in booted system terminal
uname -r
Output must match: e.g., 6.1.21-custom-mentor-v1

2. Check active boot command parameters
cat /proc/cmdline

3. Verify driver module presence
lsmod | grep -i ftdi || echo "FTDI compiled as built-in or inactive"
```

## Deliverables

---

- [ ] Detailed compilation steps text file `compile_steps.txt`.
  - [ ] Compiled target kernel image `Image` (or `zImage`).
  - [ ] Output verification screenshots showing the `uname -r` command execution on the physical target terminal.
- 

## ? Self-Check Questions

---

1. Why can you not execute a standard GCC compiled binary (e.g., `gcc main.c -o main`) compiled on your host PC inside your ARM-based Raspberry Pi?
  2. What are the roles of `-j8` or `-j$(nproc)` flags during compilation runs? What is the impact on compile speed?
  3. Differentiate between compiling a device driver as **Built-in ( \* )** versus compiling it as a **Module ( M )**. How does the kernel load modules dynamically?
  4. What is the role of `/proc/config.gz`? How do you enable this feature inside kernel configurations?
  5. Explain what target triplets (e.g., `arm-linux-gnueabi`) represent. Identify the meaning of each word.
- 

## Stretch Task (Optional)

---

Modify the kernel configurations to **disable** all unnecessary network and multimedia drivers, reducing the compiled kernel `Image` footprint size by at least 40%. Measure and compare the boot timings of the minimal kernel versus the default build.

## Motivation Checkpoint

---

Standard Linux distributions (like Raspberry Pi OS or Debian) are bloated, packed with drivers for devices you will never connect, and slow to boot. When designing professional medical devices, smart vehicle gateways, or industrial controllers, you must compile minimal, secured, and highly optimized custom kernels tailored strictly to your specific target hardware.

## Trainer Review Rule

---

- **Pass Criteria:**
- Custom kernel compiles successfully without errors.

- Target system boots successfully over serial line.
- `uname -r` matches the customized local version tag.
- **Fail Criteria:**
  - Kernel panic on boot due to missing device tree configurations.
  - Bypassed compilation stages.



# Weekly Execution Plan: Week 27

---

## Weekly Goal

---

Master the Linux Device Tree specification, understand the syntax of Device Tree Source ( `.dts` ) and Include ( `.dtsi` ) files, compile device trees using the Device Tree Compiler ( `dtc` ), and write custom peripheral node definitions.

## Weekly Outcome

---

By Friday, the student will have designed and compiled a custom **Device Tree Overlay** ( `.dtbo` ) to map and enable a physical hardware peripheral (such as an external LED and button) on their embedded board. They will understand address cells, size cells, device compatibility strings, and interrupt mappings.

---



## Session Breakdown (Monday - Friday)

---

### Monday: Why Device Trees? & The Hardware Representation Model

- **Conceptual Objective:** Understand that unlike x86 PCs (which use self-describing buses like PCI/USB to auto-detect hardware at boot), embedded systems use simple memory-mapped buses that the kernel cannot auto-detect. Learn how **Device Trees** act as static hardware databases describing the exact physical addresses, clocks, and interrupts of all non-discoverable peripherals to the kernel.
- **Practical Activity:**
  - Sketch a block diagram showing how the kernel reads the `.dtb` at boot to initialize platform devices dynamically.
  - Differentiate between DTS (source), DTSI (include), and DTB (compiled binary).
- **Code/Circuit Reference:**

Hardware Description Evolution:

Old Linux Method: Hardcoded C struct definitions inside "boardfiles" (bloated kernel!).

Modern Linux Method: Hardware described inside text `.dts`` files, compiled into binary `.dtb``, and loaded into RAM by the bootloader at boot.

### Tuesday: Device Tree Syntax (Nodes, Properties, and Compatible Strings)

- **Conceptual Objective:** Master the tree syntax of DTS files. Understand:

- **Nodes:** Represent physical blocks (e.g., `gpio@40020000` ).
- **Properties:** Key-value pairs (e.g., `reg = <0x40020000 0x400>` ).
- **Compatible Strings:** The critical link connecting the node to the matching kernel driver (e.g., `compatible = "brcm,bcm2835-gpio"` ).
- **Practical Activity:**
  - Read a section of the mainline SoC DTSI file.
  - Identify the compatible strings and address mapping parameters.
- **Code/Circuit Reference:**

```
/* Conceptual Device Tree Node */
/ {
 soc {
 #address-cells = <1>;
 #size-cells = <1>;

 gpio@40020000 {
 compatible = "vendor,custom-gpio";
 reg = <0x40020000 0x1000>; // Base Address, Size
 gpio-controller;
 #gpio-cells = <2>;
 };
 };
};
```

## Wednesday: Address Cells, Size Cells, and Register Properties

- **Conceptual Objective:** Master memory cell parameters:
  - `#address-cells` : Specifies how many 32-bit words are used to represent physical base addresses.
  - `#size-cells` : Specifies how many words are used to represent memory sizes.
  - `reg` : The register address and length mapping arrays.
- **Practical Activity:**
  - Calculate cell counts for 64-bit addresses (e.g., `#address-cells = <2>` ).
  - Construct a memory mapping block for a dual-port memory controller.
- **Code/Circuit Reference:**

```

/* Mapping a 64-bit address space */
soc {
 #address-cells = <2>; // 64-bit address requires two 32-bit cells
 #size-cells = <1>; // 32-bit size

 timer@4,40000000 {
 compatible = "vendor,timer";
 reg = <0x4 0x40000000 0x1000>; // Address = 0x44000000, Size = 0x1000
 };
};

```

## Thursday: Interrupt Mappings & Pin Multiplexing (Pinctrl)

- **Conceptual Objective:** Study how interrupts and pin routing are represented inside Device Trees. Understand:
  - `interrupt-parent` : Identifies the core interrupt controller.
  - `interrupts` : Specifies the interrupt line number and trigger properties (rising/falling edge).
  - `pinctrl` : Configures pin alternative functions and pull-up/down behaviors.
- **Practical Activity:**
  - Locate pinctrl definitions inside your board's DTS files.
  - Map out how physical GPIO pins are bound to peripheral controllers.
- **Code/Circuit Reference:**

```

/* Interrupt routing node */
custom_sensor: sensor@40030000 {
 compatible = "vendor,custom-sensor";
 reg = <0x40030000 0x100>;
 interrupt-parent = <8gic>;
 interrupts = <0 17 4>; // IRQ Type (SPI/PPI), IRQ Number, Trigger (Active High)
};

```

## Friday: Compiling Overlays via DTC

- **Conceptual Objective:** Master the **Device Tree Compiler ( `dtc` )**. Learn how to compile source DTS files into target DTB files. Study **Device Tree Overlays ( `.dto` / `.dts` )**—a modular compilation mechanism that allows modifying the main system device tree dynamically at runtime without recompiling the entire SoC OS tree.
- **Practical Activity:**
  - Write a custom Device Tree Overlay defining a physical LED pin.
  - Compile it using `dtc` into a `.dtbo` and load it into your system configuration.

### • Code/Circuit Reference:

```
Compile DTS to DTB
dtc -I dts -O dtb -o board.dtb board.dts

Compile Overlay to DTBO
dtc -@ -I dts -O dtb -o overlay.dtbo overlay.dts
```

## Hands-On Assignment: Custom GPIO & Button Device Tree Overlay

### Functional Requirements

Write, compile, and configure a custom Device Tree Overlay that maps and enables a physical LED output and a physical Pushbutton input on your target hardware. 1. The overlay source file must be named `hardware_overlay.dts`. 2. Define a custom compatible string `compatible = "custom-gpio-led"`. 3. The overlay must: - Map a physical GPIO pin to act as an output LED. - Map a physical GPIO pin to act as an input button, setting its default pull-down resistor and configuring it as an active-low trigger source. 4. Compile the overlay using the `dtc` compiler utility to generate `hardware_overlay.dtbo`. 5. Copy the `.dtbo` file to the target boot partition overlays directory (e.g., `/boot/overlays/`). 6. Enable the overlay in the target boot configuration file (e.g., `config.txt` or `uEnv.txt`). 7. Boot the system and verify the overlay node is active by navigating the virtual filesystem `/proc/device-tree/` to locate your new compatible device node.

### Structural Requirements & Code Skeleton

`hardware_overlay.dts` :

```

/dts-v1/;
/plugin/;

/ {
 compatible = "brcm,bcm2711"; // Target SoC compatible

 fragment@0 {
 target = <&gpio>;
 __overlay__ {
 custom_led_pins: custom_led_pins {
 brcm,pins = <18>; // GPIO18
 brcm,function = <1>; // Output
 brcm,pull = <0>; // No pull
 };

 custom_btn_pins: custom_btn_pins {
 brcm,pins = <23>; // GPIO23
 brcm,function = <0>; // Input
 brcm,pull = <2>; // Pull-down
 };
 };
 };

 fragment@1 {
 target-path = "/";
 __overlay__ {
 custom_hardware {
 compatible = "custom-gpio-peripheral";
 pinctrl-names = "default";
 pinctrl-0 = <&custom_led_pins &custom_btn_pins>;
 led-gpios = <&gpio 18 0>; // GPIO18, Active High
 btn-gpios = <&gpio 23 1>; // GPIO23, Active Low
 status = "okay";
 };
 };
 };
};
};

```

## Deliverables

---

- [ ] Device Tree Overlay source file `hardware_overlay.dts` .
  - [ ] Compiled binary overlay `hardware_overlay.dtbo` .
  - [ ] Verification report `dt_verification.txt` listing the active nodes found under `/sys/firmware/devicetree/base/` or `/proc/device-tree/` .
- 

## ? Self-Check Questions

---

1. Why does an Intel x86 PC not require Device Tree files to boot and initialize its peripherals, whereas an ARM-based SoC requires them?
  2. What are **Compatible Strings**? Explain how the kernel matches a device tree node to a compiled driver.
  3. How do `#address-cells` and `#size-cells` control the formatting of the `reg` property array?
  4. What are Device Tree Overlays? Why are they highly useful when prototyping custom shield boards?
  5. How do you check if a device tree node was successfully loaded by the kernel at runtime?
- 

## Stretch Task (Optional)

---

Extend the overlay to define an I2C sensor peripheral (e.g., MPU6050) linked to the primary I2C bus node. Set its I2C slave address property to `0x68` and configure its interrupt line.

## Motivation Checkpoint

---

In modern Linux systems, drivers are generic: you do not write a separate driver for every board. The driver only handles the general chip control logic. The specific physical pin mappings and base addresses are fed to the driver dynamically at boot via the Device Tree. Understanding Device Trees is the primary step to hardware systems integration.

## Trainer Review Rule

---

- **Pass Criteria:**
- Device Tree overlay compiles without syntax errors.
- Overlay successfully updates the kernel hardware list.
- Custom nodes are visible under `/proc/device-tree/` .

- **Fail Criteria:**

- Direct hardware conflicts due to incorrect pin selections.
- DTC compiler warnings.

# Weekly Execution Plan: Week 28

---

## Weekly Goal

---

Master the mechanics of writing, compiling, loading, and debugging Linux Loadable Kernel Modules (LKMs), understand the difference between user space and kernel space memory contexts, and write dynamic parameter interfaces.

## Weekly Outcome

---

By Friday, the student will have written and compiled a custom **Linux Kernel Module ( .ko )** from scratch. They will understand kernel module signatures, how to load and unload them dynamically via shell utilities ( `insmod` , `rmmod` ), pass input parameters dynamically at loading, and trace print outputs inside the kernel ring buffer ( `dmesg` ).

---

## 17 Session Breakdown (Monday - Friday)

---

### Monday: Kernel Space vs User Space Memory Contexts

- **Conceptual Objective:** Master the fundamental separation between User Space (unprivileged, virtual memory isolated per process, cannot access hardware directly) and Kernel Space (privileged, shared flat memory, direct hardware control). Understand that kernel code must be written with extreme discipline: a single NULL pointer dereference inside a kernel module will not crash a simple process—it triggers a **Kernel Panic** or **Oops**, halting the entire operating system.
- **Practical Activity:**
  - Sketch a virtual memory diagram showing the user/kernel space division.
  - Study kernel programming constraints (no standard library `<stdio.h>` or `<stdlib.h>` , must use `<linux/init.h>` and `<linux/kernel.h>` ).
- **Code/Circuit Reference:**



Operating System Execution Contexts:

+-----+	
User Space: Processes (Virtual Memory page mapped)	
- Banned direct I/O or raw register accesses.	
+-----+	
[System Call Interface Gateway]	
+-----+	
Kernel Space: Shared Flat Memory (Direct Execution)	
- Custom Modules, Memory Page Allocator, Core Scheduler	
+-----+	

## Tuesday: The Anatomy of a Kernel Module ( `init` & `exit` hooks)

- **Conceptual Objective:** Study the entry and exit lifecycles of a Loadable Kernel Module (LKM). Understand:
  - `module_init()` : Specifies the initialization function executed when the module is loaded.
  - `module_exit()` : Specifies the cleanup function executed when the module is unloaded.
- **License declarations:** `MODULE_LICENSE("GPL")` is mandatory to prevent tainting the kernel.
- **Practical Activity:**
  - Write a minimal "Hello World" kernel module.
  - Map out the kernel header requirements.
- **Code/Circuit Reference:**

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Senior Mentor");
MODULE_DESCRIPTION("A minimal LKM skeleton");
MODULE_VERSION("0.1");

static int __init my_module_init(void) {
 printk(KERN_INFO "LKM: Initialized successfully.\n");
 return 0; // Success
}

static void __exit my_module_exit(void) {
 printk(KERN_INFO "LKM: Unloaded cleanly.\n");
}

module_init(my_module_init);
module_exit(my_module_exit);

```

## Wednesday: Compiling Kernel Modules (Kbuild & Makefiles)

- **Conceptual Objective:** Learn how kernel modules compile. Because kernel modules execute inside the kernel context, they must compile using the **Kbuild** build system of the target kernel source tree. You cannot compile a kernel module using standard host GCC pipelines.
- **Practical Activity:**
  - Write a custom out-of-tree `Makefile` designed to target active kernel source trees.
  - Compile your module and verify the generation of the `.ko` (Kernel Object) binary.
- **Code/Circuit Reference:**

```
Professional Kernel Module Makefile
obj-m += my_module.o

KDIR points to host active kernel headers or custom cross-compiled kernel directory
KDIR ?= /lib/modules/$(shell uname -r)/build

all:
 make -C $(KDIR) M=$(PWD) modules

clean:
 make -C $(KDIR) M=$(PWD) clean
```

## Thursday: Kernel Logging via `printk` & Shell Commands

- **Conceptual Objective:** Understand that kernel modules cannot call `printf()` (which relies on standard C library user-space stdout). You must use `printk` —the kernel logging system. Study log levels ( `KERN_EMERG` , `KERN_ERR` , `KERN_WARNING` , `KERN_INFO` , `KERN_DEBUG` ). Master terminal shell utilities: `insmod` (insert module), `rmmod` (remove module), `lsmod` (list loaded modules), `modinfo` (inspect module details), and `dmesg` (read kernel circular log buffer).
- **Practical Activity:**
  - Load your compiled `.ko` module.
  - Monitor the `dmesg` output in real-time inside a separate shell terminal.
- **Code/Circuit Reference:**

```
Monitor kernel logs continuously
sudo dmesg -w

Insert module
sudo insmod my_module.ko

Check loaded modules status
lsmod | grep my_module

Remove module
sudo rmmod my_module
```

## Friday: Passing Parameters dynamically ( `module_param` )

- **Conceptual Objective:** Learn how to pass input parameters dynamically to a kernel module at runtime (e.g., configuring hardware IRQ lines or debug settings during `insmod` ). Study `module_param()` and permissions masks.

- **Practical Activity:**

- Declare module parameters inside your code.
- Insert the module with custom arguments and verify that parameter values are captured.

- **Code/Circuit Reference:**

```
static int debug_level = 0;
// Parameter: Variable Name, Type (int, charp), Permission (Sysfs read/write)
module_param(debug_level, int, 0660);
MODULE_PARM_DESC(debug_level, "Debug logging level (0-3)");
```

---

## Hands-On Assignment: Custom Telemetry Parameter LKM

---

### Functional Requirements

Create a fully functional, out-of-tree Linux Loadable Kernel Module that manages dynamic input parameters, checks system limits, and logs footprints safely inside the kernel space. 1. The module must be named `telemetry_lkm.c`. 2. Define three dynamic parameters: - `sensor_id` (int) - Default value `100`. - `log_interval_ms` (int) - Default `1000`. - `device_name` (charp) - Default `"dummy-sensor"`. 3. In `module_init`: - Print a clean welcome log including author details. - Print the values of all three parameters. - If `log_interval_ms` is negative or exceeds `10000`, print an error log using `KERN_ERR` and return `EINVAL` (Error Invalid Argument) to **prevent the module from loading**. 4. In `module_exit`: - Print a clean shutdown log indicating successful unmapping. 5. Provide a Makefile that supports out-of-tree compilation targeting the host system kernel. 6. Test loading the module with different parameter configurations and verify logs using `dmesg`.

### Structural Requirements & Code Skeleton

`telemetry_lkm.c` (C-Skeleton):

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/moduleparam.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded Mentor");
MODULE_DESCRIPTION("LKM Telemetry Parameter Driver");
MODULE_VERSION("1.0");

static int sensor_id = 100;
static int log_interval_ms = 1000;
static char* device_name = "dummy-sensor";

module_param(sensor_id, int, 0660);
MODULE_PARM_DESC(sensor_id, "Unique ID of the sensor");

module_param(log_interval_ms, int, 0660);
MODULE_PARM_DESC(log_interval_ms, "Sampling log interval in milliseconds");

module_param(device_name, charp, 0660);
MODULE_PARM_DESC(device_name, "Name string of the device");

static int __init telemetry_init(void) {
 printk(KERN_INFO "TELEMETRY: Loading module ... \n");

 // Bounds Check
 if (log_interval_ms < 10 || log_interval_ms > 10000) {
 printk(KERN_ERR "TELEMETRY: ERROR: Invalid log_interval_ms %d (Range:
10-10000)\n", log_interval_ms);
 return -EINVAL; // Prevents loading!
 }

 printk(KERN_INFO "TELEMETRY: Device Name : %s\n", device_name);
 printk(KERN_INFO "TELEMETRY: Sensor ID : %d\n", sensor_id);
 printk(KERN_INFO "TELEMETRY: Interval : %d ms\n", log_interval_ms);
 return 0;
}

static void __exit telemetry_exit(void) {
 printk(KERN_INFO "TELEMETRY: Module unloaded cleanly. Goodbye!\n");
}

```

```
module_init(telemetry_init);
module_exit(telemetry_exit);
```

---

## Deliverables

- [ ] Kernel module source file `telemetry_lkm.c`.
  - [ ] Compilation `Makefile`.
  - [ ] Capture trace log file `dmesg_run.log` showing:
    - Welcome prints with custom parameters on success.
    - Rejection prints and shell outputs when attempting to load with invalid interval parameters ( `sudo insmod telemetry_lkm.ko log_interval_ms=99999` ).
- 

## ? Self-Check Questions

1. Why does code running inside Kernel Space have the power to destroy the system upon simple pointer corruption, whereas User-Space application crashes are isolated?
  2. Why can you not use standard header libraries like `<stdio.h>` or call functions like `printf()` inside a Linux kernel module?
  3. Walk through the role of the Kbuild Makefile structure. What does the variable flag `obj-m += my_module.o` tell the compilation engine?
  4. How do you return error states in `module_init` to programmatically reject module loading? Give a standard error macro example.
  5. What is the role of `lsmod`? Where does the utility query this information dynamically in the filesystem?
- 

## Stretch Task (Optional)

Extend the telemetry module to expose the parameters dynamically inside `/sys/module/telemetry_lkm/parameters/`. Demonstrate that you can read and write the values of the parameters from the user space shell using `cat` and `echo`.



## Motivation Checkpoint

---

Standard application developers compile packages that run on top of operating systems. But embedded systems engineers write the operating system interfaces. When a new custom board comes from the manufacturing line, you must write the kernel integration code to route peripherals dynamically. Mastering Loadable Kernel Modules is the baseline of kernel development.



## Trainer Review Rule

---

- **Pass Criteria:**

- Module compiles cleanly using target Kbuild systems.
- Parameter checks successfully block loading on invalid ranges, returning `EINVAL` to `insmod`.
- Kernel circular buffer traces confirm prints.

- **Fail Criteria:**

- Memory leaks or pointer corruptions that trigger kernel Oops.
- Bypassing standard Kbuild compile rules.

# Weekly Execution Plan: Week 29

---

## Weekly Goal

---

Master the architecture of Linux Character Device Drivers, understand Major and Minor numbers, configure device classes, implement the `file_operations` structure, and safely transfer memory buffers between user-space and kernel-space address blocks.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Linux Character Device Driver** from scratch. They will understand how physical virtual device nodes (e.g., `/dev/my_sensor`) are registered, how user-space systems calls (`open`, `read`, `write`, `close`) map directly to driver functions, and how to verify driver data flows safely.

---



## Session Breakdown (Monday - Friday)

---

### Monday: The Character Device Paradigm & Major/Minor Numbers

- **Conceptual Objective:** Master character device driver designs. Understand that a character device handles data streams sequentially byte-by-byte (like serial ports, keypads, or virtual devices). Learn about **Major Numbers** (identifies the specific driver linked to the device) and **Minor Numbers** (identifies individual physical device instances managed by that driver).
- **Practical Activity:**
  - Study dynamic allocation APIs: `alloc_chrdev_region()` (allocates major/minor numbers dynamically) and `unregister_chrdev_region()`.
  - Inspect `/proc/devices` to locate loaded character drivers major offsets.
- **Code/Circuit Reference:**



```
#include <linux/fs.h>

dev_t dev_num; // Holds 32-bit major/minor number combined

static int __init chrdev_init(void) {
 // Dynamically allocate 1 device number starting at minor 0
 int ret = alloc_chrdev_region(&dev_num, 0, 1, "my_char_device");
 if (ret < 0) {
 printk(KERN_ERR "CHDEV: Failed to allocate device numbers.\n");
 return ret;
 }
 printk(KERN_INFO "CHDEV: Major = %d, Minor = %d\n", MAJOR(dev_num),
 MINOR(dev_num));
 return 0;
}
```

## Tuesday: File Operations ( `file_operations` structure)

- **Conceptual Objective:** Master the `file_operations` (**fops**) structure—the core translation matrix of driver development. Understand how the kernel routes user-space system calls ( `open()` , `read()` , `write()` , `close()` ) executing on a device file to your matching driver implementation function pointers.
- **Practical Activity:**
  - Define individual driver implementations for open, read, write, and release.
  - Construct the `file_operations` structure, mapping it to your custom functions.
- **Code/Circuit Reference:**

```
static int dev_open(struct inode *inodep, struct file *filep);
static int dev_release(struct inode *inodep, struct file *filep);
static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t
*offset);
static ssize_t dev_write(struct file *filep, const char __user *buffer, size_t len,
loff_t *offset);

static struct file_operations fops = {
 .owner = THIS_MODULE,
 .open = dev_open,
 .read = dev_read,
 .write = dev_write,
 .release = dev_release,
};
```

## Wednesday: Character Device Registration ( `cdev` )

- **Conceptual Objective:** Study the `cdev` structure—the kernel's internal representation of a character device. Learn how to initialize `cdev` with your `file_operations` and register it inside kernel tables using `cdev_add()`.
- **Practical Activity:**
  - Integrate `cdev` variables inside your module.
  - Implement proper cleanup structures inside `module_exit` to remove the device cleanly.
- **Code/Circuit Reference:**

```
#include <linux/cdev.h>

struct cdev my_cdev;

static int register_cdev(void) {
 cdev_init(&my_cdev, &fops);
 my_cdev.owner = THIS_MODULE;
 // Add 1 device starting at dev_num
 int ret = cdev_add(&my_cdev, dev_num, 1);
 return ret;
}
```

## Thursday: Memory Segregation & Data Copy Utilities

- **Conceptual Objective:** Master the strict memory boundaries of operating systems. User-space programs use virtual pointers that cannot be accessed directly by the kernel driver without security risks. To transfer data safely, you must use kernel memory copy routines:
- `copy_to_user()` : Copies data from kernel-space buffer to user-space buffer.
- `copy_from_user()` : Copies data from user-space buffer to kernel-space buffer.
- **Practical Activity:**
  - Deep-dive into buffer bounds checks.
  - Write safe data copy blocks verifying return checks.
- **Code/Circuit Reference:**

```
static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t
*offset) {
 char message[] = "Hello User Space!";
 size_t msg_len = sizeof(message);

 if (*offset ≥ msg_len) return 0; // EOF reached

 // copy_to_user returns 0 on complete success, or number of uncopied bytes
 if (copy_to_user(buffer, message + *offset, msg_len - *offset) ≠ 0) {
 return -EFAULT; // Bad address error
 }

 *offset += msg_len;
 return msg_len;
}
```

## Friday: Dynamic Device Nodes Creation ( `sysfs` & `class` )

- **Conceptual Objective:** Understand that in modern Linux systems, you do not create physical device nodes (e.g., `/dev/my_device`) manually using `mknod` commands. The kernel manages **Classes** ( `sysfs` ). Creating a device class ( `class_create()` ) and device parent ( `device_create()` ) triggers the system daemon `udev` to automatically generate the physical `/dev/` file node upon module insertion.
- **Practical Activity:**
  - Implement device class creation and device registers inside your module.
  - Load module and verify the dynamic appearance of your `/dev/` node.
- **Code/Circuit Reference:**

```
#include <linux/device.h>

static struct class* my_class = NULL;
static struct device* my_device = NULL;

static int init_sysfs(void) {
 // 1. Create class
 my_class = class_create("my_sensor_class");

 // 2. Create device (triggers udev to create /dev/my_sensor)
 my_device = device_create(my_class, NULL, dev_num, NULL, "my_sensor");
 return 0;
}

static void cleanup_sysfs(void) {
 device_destroy(my_class, dev_num);
 class_destroy(my_class);
}
```

## Hands-On Assignment: Virtual Ring Buffer Character Driver

### Functional Requirements

Create a fully operational, robust Linux Character Device Driver that implements a virtual character buffer, allowing user-space applications to read and write bytes safely. 1. The driver must compile as a loadable module `vbuf_driver.ko`. 2. Upon initialization: - Dynamically allocate Major/Minor numbers. - Create a device class `vbuf_class` and a device node `/dev/vbuf`. - Allocate a static internal kernel memory buffer of 1024 bytes. 3. Implement file operations: - `open`: Print a diagnostic log inside the kernel buffer. - `write`: Receive data from a user-space string using `copy_from_user()`, write it into the internal kernel buffer up to limits, and update the write pointer offset. - `read`: Copy data from the internal kernel buffer to the user-space buffer using `copy_to_user()`, update the read offset, and return the exact byte length. - `release`: Print diagnostic exit messages. 4. Implement clean error checks. If any step in module initialization fails, rollback all previous registrations (e.g., destroy device, destroy class, unregister region) in reverse order to prevent memory leak corruptions. 5. Write a user-space test program `test_driver.c` that writes a message string into `/dev/vbuf` and reads it back to confirm functional loopback operation.

### Structural Requirements & Code Skeleton

`vbuf_driver.c` (C-Skeleton):

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded System Architect");
MODULE_DESCRIPTION("Virtual Buffer Character Driver");
MODULE_VERSION("1.0");

#define DEVICE_NAME "vbuf"
#define BUF_SIZE 1024

static dev_t dev_num;
static struct cdev char_cdev;
static struct class *dev_class = NULL;
static struct device *dev_device = NULL;

static char kernel_buffer[BUF_SIZE];
static size_t data_len = 0;

static int dev_open(struct inode *inodep, struct file *filep) {
 printk(KERN_INFO "VBUF: Device opened.\n");
 return 0;
}

static int dev_release(struct inode *inodep, struct file *filep) {
 printk(KERN_INFO "VBUF: Device closed.\n");
 return 0;
}

static ssize_t dev_read(struct file *filep, char __user *buffer, size_t len, loff_t
*offset) {
 size_t to_copy = (len < (data_len - *offset)) ? len : (data_len - *offset);
 if (to_copy == 0) return 0; // EOF

 if (copy_to_user(buffer, kernel_buffer + *offset, to_copy) != 0) {
 return -EFAULT;
 }
}

```

```

 *offset += to_copy;
 return to_copy;
 }

static ssize_t dev_write(struct file *filep, const char __user *buffer, size_t len,
loff_t *offset) {
 size_t to_copy = (len < (BUF_SIZE - *offset)) ? len : (BUF_SIZE - *offset);
 if (to_copy == 0) return -ENOMEM; // Out of memory

 if (copy_from_user(kernel_buffer + *offset, buffer, to_copy) != 0) {
 return -EFAULT;
 }
 *offset += to_copy;
 data_len = *offset;
 return to_copy;
}

static struct file_operations fops = {
 .owner = THIS_MODULE,
 .open = dev_open,
 .release = dev_release,
 .read = dev_read,
 .write = dev_write,
};

static int __init vbuf_init(void) {
 // 1. Allocate device numbers
 if (alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME) < 0) return -1;

 // 2. Initialize cdev
 cdev_init(&char_cdev, &fops);
 if (cdev_add(&char_cdev, dev_num, 1) < 0) goto r_class;

 // 3. Create class
 dev_class = class_create("vbuf_class");
 if (IS_ERR(dev_class)) goto r_cdev;

 // 4. Create device node
 dev_device = device_create(dev_class, NULL, dev_num, NULL, DEVICE_NAME);
 if (IS_ERR(dev_device)) goto r_device;

 printk(KERN_INFO "VBUF: Driver loaded. /dev/%s active.\n", DEVICE_NAME);
 return 0;
}

```

```

r_device:
 class_destroy(dev_class);
r_cdev:
 cdev_del(&char_cdev);
r_class:
 unregister_chrdev_region(dev_num, 1);
 return -1;
}

static void __exit vbuf_cleanup(void) {
 device_destroy(dev_class, dev_num);
 class_destroy(dev_class);
 cdev_del(&char_cdev);
 unregister_chrdev_region(dev_num, 1);
 printk(KERN_INFO "VBUF: Driver unloaded.\n");
}

module_init(vbuf_init);
module_exit(vbuf_cleanup);

```

---

## Deliverables

- [ ] Character driver source file `vbuf_driver.c`.
  - [ ] User test application `test_driver.c`.
  - [ ] Compilation `Makefile`.
  - [ ] Terminal capture verification log demonstrating successful execution of writing a string from the user test into `/dev/vbuf` and reading it back cleanly.
- 

## Self-Check Questions

1. Why can a user-space application not access the memory address of a kernel driver directly using a standard C pointer? Explain the memory isolation boundaries of the operating system.
2. What are the roles of Major and Minor numbers? How do you map major allocations dynamically?
3. Walk through the parameter variables of the `file_operations` write function. What is the role of `loff_t *offset`?
4. What is the consequence if the kernel module initialization fails and you forget to execute dynamic unregistrations (rollback) of previously allocated memory region pools?

5. How does the `udev` system daemon automatically capture `device_create()` outputs to generate dynamic physical nodes in `/dev/` ?

---

## Stretch Task (Optional)

---

Extend the virtual character driver to support the `ioctl` **system call interface**: implement custom control commands that allow the user-space test application to wipe the internal kernel buffer or query memory usage parameters.

## Motivation Checkpoint

---

Inside any operating system, character device drivers are the bridge to the physical world. Whether you are interfacing an industrial UART controller, a custom FPGA data register, or reading sensor buses, you must write character drivers to route the raw bytes to the user space cleanly. Mastering character drivers is standard requirement for kernel engineers.

## Trainer Review Rule

---

### • Pass Criteria:

- Driver successfully registers Major/Minor numbers dynamically.
- Dynamically creates `/dev/vbuf` node upon insertion.
- Memory copy operations check all bounds and prevent heap overflows.
- Zero warnings compiled using Kbuild.

### • Fail Criteria:

- Direct pointer dereferences crossing user-kernel space boundaries without `copy_to/from_user`.
- System lock-up due to unreleased dynamic allocations on failure rollbacks.



# Weekly Execution Plan: Week 30

---

## Weekly Goal

---

Master physical hardware access inside Linux Kernel Drivers, understand memory-mapped I/O mapping using `ioremap()`, read and write device control registers safely inside kernel space, configure GPIO pins directly from drivers, and register hardware interrupts in the kernel.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Physical Hardware Linux Kernel Driver** on their embedded target. They will have mapped physical hardware base registers, configured a physical GPIO output pin to toggle an LED, registered a physical button GPIO as an interrupt source inside the kernel, and successfully completed the Phase 5 Milestone Challenge.

---

## 17 Session Breakdown (Monday - Friday)

---

### Monday: Kernel Memory-Mapped I/O ( `ioremap` )

- **Conceptual Objective:** Master memory-mapped I/O inside kernel space. Understand that to access physical hardware registers, the driver must explicitly map the physical base addresses of the SoC peripherals into the kernel's virtual memory map using `ioremap()`.
- **Practical Activity:** remap target base register addresses and set up clean unmappings.
- **Code/Circuit Reference:**

```
#include <asm/io.h>

void __iomem *gpio_base = NULL;

static int map_hardware(void) {
 gpio_base = ioremap(0xFE200000, 256);
 if (gpio_base == NULL) {
 printk(KERN_ERR "HWDRV: ioremap failed.\n");
 return -ENOMEM;
 }
 return 0;
}
```

## Tuesday: Kernel Register Reads & Writes (readl/writel)

- **Conceptual Objective:** Study register access functions. You must never read/write remapped memory-mapped pointers using direct C pointer assignments. You must use standardized kernel register access macros ( `readl` , `writel` ).

- **Code/Circuit Reference:**

```
#define GPSET0_OFFSET 0x1C

void set_gpio_pin(void) {
 uint32_t reg_val = readl(gpio_base + GPSET0_OFFSET);
 reg_val |= (1U << 18);
 writel(reg_val, gpio_base + GPSET0_OFFSET);
}
```

## Wednesday: The Kernel GPIO Subsystem (Legacy vs Modern gpiod)

- **Conceptual Objective:** Study the standardized kernel GPIO subsystem framework designed to abstract SoC-specific register maps. Request pins ( `gpio_request()` ), configure directions ( `gpio_direction_output()` ), and write pin states.

- **Code/Circuit Reference:**

```
#include <linux/gpio.h>
#define LED_GPIO 18

static int init_gpio(void) {
 if (gpio_request(LED_GPIO, "sys_led") < 0) return -1;
 gpio_direction_output(LED_GPIO, 0);
 return 0;
}
```

## Thursday: Hardware Interrupts in Kernel Space

- **Conceptual Objective:** Master the registration of hardware interrupts inside kernel space. Convert a physical GPIO pin into a kernel-compatible Interrupt Request (IRQ) number using `gpio_to_irq()` and attach an ISR callback using `request_irq()` .

- **Code/Circuit Reference:**

```
#include <linux/interrupt.h>
#define BTN_GPIO 23

static irqreturn_t button_isr(int irq, void *dev_id) {
 printk(KERN_INFO "HWDVR: Interrupt Triggered inside Kernel!\n");
 return IRQ_HANDLED;
}
```

## Friday: Interrupt Bottom-Halves (Tasklets vs Workqueues)

- **Conceptual Objective:** Differentiate execution contexts inside kernel interrupts. A kernel ISR must run extremely fast and must never sleep. Slow tasks must be deferred to the bottom-half: Tasklets (softirq context) or Workqueues (kernel thread context). Launch the Phase 5 Milestone Challenge.



## Hands-On Assignment: Physical LED and Button Interrupt Kernel Driver

### Functional Requirements

Create a fully functional, out-of-tree Linux Character Device Driver that maps physical hardware pins, controls a physical LED, and registers a physical button as a hardware interrupt source inside the kernel. 1. The driver must compile as a loadable module `hardware_driver.ko`. 2. Upon initialization: - Dynamically allocate character major/minor regions and register `/dev/hw_ctrl` node class. - Request physical LED GPIO pin (e.g., GPIO18) as output. - Request physical Button GPIO pin (e.g., GPIO23) as input. - Convert Button GPIO to an IRQ number, and register a kernel ISR triggered on **Falling Edges** (active-low transition). 3. The kernel ISR must: - Print notifications to kernel log circular buffers (`printk`). - Toggle the state of the physical LED pin instantly. 4. Implement clean error rollbacks. If interrupt registration fails, request releases of all acquired GPIO pins and dynamically free major/minor character spaces. 5. In the module exit cleanup function, release the IRQ channel (`free_irq`) and release the GPIO pins (`gpio_free`) to prevent hardware pin lockups. 6. Test on target hardware, press the button, and verify instant LED toggle response, tracking log outputs inside `dmesg`.

### Structural Requirements & Code Skeleton

`hardware_driver.c`:

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/gpio.h>
#include <linux/interrupt.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded Kernels Dev");
MODULE_DESCRIPTION("LED and Button Interrupt Driver");
MODULE_VERSION("1.0");

#define LED_PIN 18
#define BTN_PIN 23

static dev_t dev_num;
static struct cdev char_cdev;
static struct class *dev_class = NULL;
static struct device *dev_device = NULL;

static int button_irq = -1;
static bool led_state = false;

static irqreturn_t button_isr(int irq, void *dev_id) {
 led_state = !led_state;
 gpio_set_value(LED_PIN, led_state ? 1 : 0);
 printk(KERN_INFO "HW_CTRL: Interrupt! Toggled LED state to: %s\n", led_state ?
"ON" : "OFF");
 return IRQ_HANDLED;
}

static int dev_open(struct inode *inodep, struct file *filep) { return 0; }
static int dev_release(struct inode *inodep, struct file *filep) { return 0; }

static struct file_operations fops = {
 .owner = THIS_MODULE,
 .open = dev_open,
 .release = dev_release,
};

```

```

static int __init hw_driver_init(void) {
 if (alloc_chrdev_region(&dev_num, 0, 1, "hw_ctrl") < 0) return -1;

 cdev_init(&char_cdev, &fops);
 if (cdev_add(&char_cdev, dev_num, 1) < 0) goto r_unreg;

 dev_class = class_create("hw_ctrl_class");
 if (IS_ERR(dev_class)) goto r_cdev;

 dev_device = device_create(dev_class, NULL, dev_num, NULL, "hw_ctrl");
 if (IS_ERR(dev_device)) goto r_class;

 if (gpio_request(LED_PIN, "sys_led") < 0) goto r_device;
 gpio_direction_output(LED_PIN, 0);

 if (gpio_request(BTN_PIN, "sys_btn") < 0) goto r_led;
 gpio_direction_input(BTN_PIN);
 gpio_set_debounce(BTN_PIN, 200);

 button_irq = gpio_to_irq(BTN_PIN);
 if (request_irq(button_irq, button_isr, IRQF_TRIGGER_FALLING, "hw_ctrl_btn", NULL)
 < 0) goto r_btn;

 printk(KERN_INFO "HW_CTRL: Custom Hardware Driver Loaded successfully.\n");
 return 0;

r_btn:
 gpio_free(BTN_PIN);
r_led:
 gpio_free(LED_PIN);
r_device:
 device_destroy(dev_class, dev_num);
r_class:
 class_destroy(dev_class);
r_cdev:
 cdev_del(&char_cdev);
r_unreg:
 unregister_chrdev_region(dev_num, 1);
 return -1;
}

static void __exit hw_driver_exit(void) {
 free_irq(button_irq, NULL);
}

```

```
 gpio_free(BTN_PIN);
 gpio_set_value(LED_PIN, 0);
 gpio_free(LED_PIN);

 device_destroy(dev_class, dev_num);
 class_destroy(dev_class);
 cdev_del(&char_cdev);
 unregister_chrdev_region(dev_num, 1);
 printk(KERN_INFO "HW_CTRL: Driver Unloaded cleanly.\n");
}

module_init(hw_driver_init);
module_exit(hw_driver_exit);
```

# Weekly Execution Plan: Week 31

---

## Weekly Goal

---

Master automated Root Filesystem (Rootfs) generation using the **Buildroot** build system, configure custom system overlays, add custom C applications as packages, and generate lightweight, minimal flashable operating system images.

## Weekly Outcome

---

By Friday, the student will have built a fully custom, lightweight **Embedded Linux Image** (total size under 30MB) using Buildroot. They will have configured system initialization scripts, integrated custom user-space packages, and flashed and booted their custom system successfully on target hardware.

---



## Session Breakdown (Monday - Friday)

---

### Monday: The Buildroot Paradigm & Workspace Setup

- **Conceptual Objective:** Differentiate manual rootfs construction (copying library files and compiling busybox by hand) from automated build engines. Understand that Buildroot is a simple, efficient, and easy-to-use tool that utilizes standard **Kconfig (menuconfig)** structures to automate the cross-compilation of cross-toolchains, bootloaders, kernels, and root filesystems.
- **Practical Activity:**
  - Clone the mainline stable Buildroot repository.
  - Explore directory trees ( `configs/` , `package/` , `system/` , `fs/` ).
  - Load the target default SoC configuration.
- **Code/Circuit Reference:**

```
Clone Buildroot stable branch
git clone git://git.buildroot.net/buildroot --depth 1 --branch 2023.02.x
cd buildroot

Load target board configuration (e.g., Raspberry Pi 4)
make raspberrypi4_64_defconfig
```

## Tuesday: Buildroot Menuconfig & Custom Packages Selection

- **Conceptual Objective:** Master the interactive Buildroot configuration utility. Learn how to configure target system parameters (architecture, toolchains), select pre-built third-party user-space libraries and packages (e.g., OpenSSH, python, sqlite), and configure system logins/passwords.

- **Practical Activity:**

- Open the interactive config menu.
- Enable toolchain options (compiler parameters).
- Enable required third-party system debug utilities (such as `strace`, `htop`, `gdb`).

- **Code/Circuit Reference:**

```
Launch interactive configuration menu
make menuconfig

Inside menuconfig:
Target options → Target Architecture (AArch64)
Toolchain → Toolchain type (Buildroot toolchain) → C library (glibc)
Target packages → Debugging, profiling and benchmark → Enable strace, gdb, htop
System configuration → System password (set to "root")
```

## Wednesday: System Directory Overlays & Init Configurations

- **Conceptual Objective:** Understand that compiled filesystems are generic. To add custom configuration files (e.g., custom network configs inside `/etc/network/interfaces` or startup scripts in `/etc/init.d/`), you must use **System Directory Overlays**. An overlay is a host directory structure copied directly on top of the generated rootfs before the final flash image is packed.

- **Practical Activity:**

- Create a custom overlay folder mapping target directories.
- Configure a startup shell script that automatically runs at boot.

- **Code/Circuit Reference:**



```
Create custom overlay inside buildroot root directory
mkdir -p my_overlay/etc/init.d/
mkdir -p my_overlay/usr/bin/

Write startup service script
cat << 'EOF' > my_overlay/etc/init.d/S99telemetry
#!/bin/sh
case "$1" in
 start)
 echo "Starting custom telemetry daemon..."
 /usr/bin/telemetry_daemon &
 ;;
 stop)
 killall telemetry_daemon
 ;;
esac
exit 0
EOF
chmod +x my_overlay/etc/init.d/S99telemetry
```

## Thursday: Compiling Custom Packages in Buildroot

- **Conceptual Objective:** Learn how to add a custom C application as a compiled package inside Buildroot. Understand that Buildroot requires a package directory inside `package/` containing:
  - `Config.in` : Defines the package selection option in menuconfig.
  - `package.mk` : The Makefile script describing where to fetch source code and how to compile and install binaries.
- **Practical Activity:**
  - Create a custom package directory `package/telemetry/`.
  - Write the Kconfig and Makefile scripts, enable it in menuconfig, and compile.
- **Code/Circuit Reference:**

```
package/telemetry/telemetry.mk
TELEMETRY_VERSION = 1.0
TELEMETRY_SITE = $(subst " ,,$(BR2_EXTERNAL_TELEMETRY_PATH))/src
TELEMETRY_SITE_METHOD = local

define TELEMETRY_BUILD_CMDS
 $(MAKE) CC="$(TARGET_CC)" LD="$(TARGET_LD)" -C $(@D)
endef

define TELEMETRY_INSTALL_TARGET_CMDS
 $(INSTALL) -D -m 0755 $(@D)/telemetry_daemon $(TARGET_DIR)/usr/bin/telemetry_daemon
endef

$(eval $(generic-package))
```

## Friday: Target Flashing and Booting

- **Conceptual Objective:** Master image compilation and SD media formatting. When compilation completes, Buildroot generates flashable files inside `output/images/` :
- `rootfs.tar` : The uncompressed rootfs.
- `sdcard.img` : A pre-partitioned, single-file flash image containing both boot and rootfs partitions.
- **Practical Activity:**
  - Trigger full system compilation.
  - Flash the compiled `sdcard.img` directly to an SD card using the `dd` command, boot your target, and verify.
- **Code/Circuit Reference:**

```
Compile everything from scratch (takes 1-2 hours on first run!)
make

Flash final image to SD card cleanly
sudo dd if=output/images/sdcard.img of=/dev/sdX bs=4M conv=fsync status=progress
```

# Hands-On Assignment: Minimal Custom OS with Boot Daemon

---

## Functional Requirements

Create a fully custom, lightweight Linux operating system using Buildroot that automatically boots, logs in as root, and launches a custom compiled C telemetry daemon. 1. Download Buildroot and configure it for your target board. 2. Integrate a custom **System Directory Overlay** that: - Configures a static IP address in `/etc/network/interfaces` . - Places a custom startup init script `S99telemetry` inside `/etc/init.d/` . 3. Create a custom user-space package `telemetry` containing a simple C program `telemetry_daemon.c` that loops writing system status logs to a local file `/var/log/telemetry.log` every 5 seconds. 4. Enable the custom package inside `make menuconfig` . 5. Compile the system to generate a final `sdcard.img` under 50MB. 6. Flash the image, boot the board, and verify that the C daemon is running as a background service and successfully logging data.

## Structural Requirements & Code Skeleton

`package/telemetry/Config.in` :

```
config BR2_PACKAGE_TELEMETRY
 bool "telemetry"
 help
 Installs a custom background telemetry daemon.
```

`package/telemetry/src/telemetry_daemon.c` :

```
#include <stdio.h>
#include <unistd.h>
#include <time.h>

int main(void) {
 FILE *log = fopen("/var/log/telemetry.log", "a");
 if (!log) return 1;

 while (1) {
 time_t now = time(NULL);
 fprintf(log, "[TELEMETRY LOG] Time: %s", ctime(&now));
 fflush(log);
 sleep(5);
 }
 fclose(log);
 return 0;
}
```

---

## Deliverables

- [ ] Custom Buildroot package configuration files ( `Config.in` , `telemetry.mk` ).
  - [ ] Telemetry daemon C source code `telemetry_daemon.c` .
  - [ ] Verification log `boot_run.log` from target serial console proving startup.
- 

## ? Self-Check Questions

1. Differentiate the roles of **Buildroot** versus standard **Yocto Project**. Under what architectural specifications is Buildroot preferred?
  2. What are **System Directory Overlays**? Why are they highly efficient for placing local configuration files into target filesystems?
  3. Walk through the role of the variables `TARGET_CC` and `TARGET_DIR` inside Buildroot package Makefile scripts.
  4. Why is a tar archive of rootfs ( `rootfs.tar` ) produced? How do you manually unpack it onto a target SD card partition?
  5. How does busybox integrate multiple standard system tools (like `ls` , `grep` , `cd` ) inside a single dynamic executable binary?
-



## Stretch Task (Optional)

---

Extend the Buildroot configuration to compile and bundle a custom **Linux Out-of-tree Kernel Module** dynamically during the system build run, loading it automatically at boot.



## Motivation Checkpoint

---

Commercial IoT gateways, routers, and automotive ECUs do not run standard desktop OS images. They run minimal, secured, and lightning-fast custom operating systems compiled directly from source. Mastering automated build engines like Buildroot is a core requirement for production-level embedded Linux developers.



## Trainer Review Rule

---

- **Pass Criteria:**

- Buildroot compiles cleanly, producing a flashable `sdcard.img`.
- Core image boots cleanly to login shell.
- Custom C daemon starts automatically as a background systemd/init service.

- **Fail Criteria:**

- Image size exceeds 100MB (indicates unnecessary bloated packages).
- Unhandled script errors during compilation.

# Weekly Execution Plan: Week 32

---

## Weekly Goal

---

Master the industry-standard **Yocto Project** build system, understand the layers metadata architecture, write custom recipes ( `.bb` ) to compile source packages, create dynamic system configuration layers, and build secure production OS images using BitBake.

## Weekly Outcome

---

By Friday, the student will have designed and compiled a custom **Yocto BSP (Board Support Package) Layer** from scratch. They will understand BitBake task execution lifecycles, how to write recipes containing build dependencies, and how to verify compiled custom images on physical hardware.

---



## Session Breakdown (Monday - Friday)

---

### Monday: The Yocto Project & Metadata Layers Architecture

- **Conceptual Objective:** Master the architecture of the Yocto Project—the universal standard for enterprise-grade Embedded Linux OS creation. Understand that Yocto is not a Linux distribution, but a collaborative project providing tools and templates. Learn the **Metadata Layers** design pattern: everything is organized in layers prefixed with `meta-` (e.g., `meta-raspberrypi` ), allowing developers to extend configurations cleanly without modifying core systems code.
- **Practical Activity:**
  - Set up host system build dependencies.
  - Clone the core Yocto Project repository (**Poky**) and locate the base layers structure.
- **Code/Circuit Reference:**

```
Poky Workspace Layout:
poky/
├── bitbake/ # The engine: parses metadata and executes tasks
├── meta/ # Core metadata recipes
├── meta-poky/ # Poky distribution configurations
└── meta-yocto-bsp/ # Default board configurations
```

## Tuesday: BitBake Task Lifecycles & Build Variables

- **Conceptual Objective:** Study **BitBake**—the execution engine of Yocto. Learn how BitBake parses recipes to execute specific task steps: `do_fetch` (gets source code), `do_unpack`, `do_patch`, `do_configure`, `do_compile`, `do_install` (copies outputs into target directory), and `do_package`. Master Yocto assignment variables: `=` (static), `?=` (conditional default), `??=` (lazy default), `+=` (append with space), and `_append` (overrides suffix).
- **Practical Activity:**
  - Initialize the Poky build environment.
  - Run simple target compilation commands.
- **Code/Circuit Reference:**

```
Initialize build environment (creates build/ directory)
source poky/oe-init-build-env build/

Check environment parameters inside build/conf/local.conf
Set target machine (e.g., Raspberry Pi 4)
MACHINE = "raspberrypi4-64"
```

## Wednesday: Creating a Custom Metadata Layer

- **Conceptual Objective:** Learn how to create a custom metadata layer to house your company's configurations and hardware descriptions. Understand the roles of `layer.conf` and `bbayers.conf` files.
- **Practical Activity:**
  - Create a custom layer directory `meta-custom-gateway` using the Yocto layer tool.
  - Integrate the layer into your active build configurations.
- **Code/Circuit Reference:**

```
Create custom layer
bitbake-layers create-layer ../meta-custom-gateway

Add layer to active build configuration
bitbake-layers add-layer ../meta-custom-gateway
```

## Thursday: Writing Custom Yocto Recipes ( `.bb` )

- **Conceptual Objective:** Master Yocto **Recipes**—text files containing variables and tasks detailing how to fetch, compile, and package applications. Study the required fields: `SUMMARY`, `LICENSE`, `LIC_FILES_CHKSUM` (enforces exact licensing audits), `SRC_URI` (source code repository link), and system installation path macros ( `${D}` destination, `${bindir}` binary directory).

- **Practical Activity:**

- Write a custom recipe compiling a C application.
- Test compilation tasks individually.

- **Code/Circuit Reference:**

```
meta-custom-gateway/recipes-apps/telemetry/telemetry_1.0.bb
SUMMARY = "Custom Telemetry Service Application"
LICENSE = "MIT"
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/
MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://telemetry.c"

S = "${WORKDIR}"

do_compile() {
 ${CC} ${CFLAGS} ${LDFLAGS} telemetry.c -o telemetry_bin
}

do_install() {
 install -d ${D}${bindir}
 install -m 0755 telemetry_bin ${D}${bindir}/telemetry_bin
}
```

## Friday: Compiling and Customizing the OS Image

- **Conceptual Objective:** Understand how to customize the final operating system image. Learn how to inherit classes (e.g., `image.bbclass`), enable specific package features directly via recipes, and trigger target BitBake builds.

- **Practical Activity:**

- Write a custom image recipe that inherits Poky's base image and automatically appends your custom telemetry package.
- Build and verify the final image.

- **Code/Circuit Reference:**

```
Compile the minimal target image containing your custom layer
bitbake core-image-minimal

Locate compiled flashable outputs in deploy folder
build/tmp/deploy/images/raspberrypi4-64/core-image-minimal-raspberrypi4-64.wic
```



---

## Hands-On Assignment: Custom Gateway Metadata Layer and Recipe

---

### Functional Requirements

Create a fully custom, production-grade Yocto Metadata Layer and write a recipe compiling a C application dynamically. 1. Download Poky and configure a Yocto workspace. 2. Create a custom layer named `meta-industrial-gateway` and integrate it into `bblayers.conf`. 3. Inside your custom layer, write a recipe named `telemetry-app_1.0.bb` that: - Fetches a local C source code file `telemetry_app.c` (or a remote Git repository link). - Audits MIT licensing cleanly. - Compiles the code during `do_compile`. - Installs the compiled binary into the target `/usr/bin/` folder. 4. Modify `build/conf/local.conf` to automatically append your custom package to the final system image build: `IMAGE_INSTALL_append = " telemetry-app"`. 5. Trigger BitBake to build `core-image-minimal`. 6. Verify the successful compilation, locate the generated `.wic` or `.sdcard` flashable image, and check that your package exists in the target deploy directory.

### Structural Requirements & Code Skeleton

`meta-industrial-gateway/conf/layer.conf` :

```
BBPATH += ":${LAYERDIR}"
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
 ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "meta-industrial-gateway"
BBFILE_PATTERN_meta-industrial-gateway = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-industrial-gateway = "6"

LAYERSERIES_COMPAT_meta-industrial-gateway = "kirkstone"
```

---

## Deliverables

---

- [ ] Custom layer folder `meta-industrial-gateway` containing conf and recipe structures.
  - [ ] Recipe file `telemetry-app_1.0.bb` and matching source `telemetry_app.c`.
  - [ ] Output build log `bitbake_run.log` demonstrating successful image compilation.
-

## ? Self-Check Questions

---

1. Differentiate **poky**, **BitBake**, and **OpenEmbedded**. How do they interact to construct a Yocto image?
  2. What are **Metadata Layers**? Why are they highly superior to hardcoded changes inside system directories when customizing large projects?
  3. Walk through the BitBake task execution order. What does `${D}` represent, and why must files be placed inside `${D}${bindir}` instead of direct root directories?
  4. What is the role of `LIC_FILES_CHKSUM`? What happens if you modify the license file string without updating the checksum?
  5. How do you append custom packages to a final image using `local.conf` variables?
- 

## Stretch Task (Optional)

---

Extend the layer to support a **systemd service recipe**: write a custom Yocto recipe that automatically installs a systemd configuration file, enabling your telemetry application to start automatically as a systemd service at boot.

## Motivation Checkpoint

---

Inside enterprise organizations (such as Tesla, Garmin, or Siemens), Yocto is the gold standard. Every Board Support Package (BSP) and application layer is modularized: if the silicon vendor releases a new hardware revision, you only swap the base BSP layer, leaving all proprietary application recipes untouched. Mastering Yocto is the hallmark of senior systems developers.

## Trainer Review Rule

---

### • Pass Criteria:

- Custom Yocto layer compiles cleanly using BitBake.
- Core image builds successfully with the custom package included.
- License checksums are audited correctly.

### • Fail Criteria:

- Hardcoded paths in recipes that fail on other hosts.
- Bypassing license validation directives.

# Weekly Execution Plan: Week 33

---

## Weekly Goal

---

Master real-world analog and actuator hardware interfaces, configure Analog-to-Digital Converters (ADC) and Pulse-Width Modulation (PWM) channels strictly via register writes, and build multi-tasking, RTOS-based sensor data processors.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Real-Time Actuator & Sensor Control Node** using FreeRTOS on the STM32F4. They will have configured the ADC peripheral to read analog inputs, mapped a general-purpose timer to generate physical PWM outputs, and integrated safe closed-loop task control feedback.

---



## Session Breakdown (Monday - Friday)

---

### Monday: Analog-to-Digital Conversion (ADC) Architecture & Registers

- **Conceptual Objective:** Master the principles of Successive Approximation Register (SAR) ADCs. Understand sample times, resolution (12-bit on STM32), input voltage scales ( $V_{REF}$ ), and critical control registers (SR, CR1, CR2, SMPR, SQR, DR).
- **Practical Activity:**
  - Study sampling rate calculations and prescaler configurations.
  - Write a C macro to initialize ADC1 on Pin PA1 (ADC Channel 1).
- **Code/Circuit Reference:**

```

#define ADC1_BASE 0x40012000U
#define ADC1_CR1 ((volatile uint32_t*)(ADC1_BASE + 0x04U))
#define ADC1_CR2 ((volatile uint32_t*)(ADC1_BASE + 0x08U))
#define ADC1_DR ((volatile uint32_t*)(ADC1_BASE + 0x4CU)) // Data Register
#define ADC1_SQR3 ((volatile uint32_t*)(ADC1_BASE + 0x34U)) // Regular
Sequence 3

void adc1_init_pa1(void) {
 // 1. Enable GPIOA and ADC1 clock gates
 ((volatile uint32_t)0x40023830U) |= (1U << 0); // GPIOAEN
 ((volatile uint32_t)0x40023844U) |= (1U << 8); // ADC1EN

 // 2. Configure PA1 as Analog mode (MODER '11')
 ((volatile uint32_t)0x40020000U) |= (3U << (1 * 2));

 // 3. Configure ADC regular sequence: Channel 1 first (SQR3 = 1)
 *ADC1_SQR3 = 1U;

 // 4. Enable ADC (ADON bit 0 in CR2)
 *ADC1_CR2 |= (1U << 0);
}

```

## Tuesday: Polling & Interrupt-Driven ADC Reads

- **Conceptual Objective:** Learn how to trigger conversions and read values. Study polling conversions using status registers (EOC: End of Conversion flag) compared to interrupt-driven conversions unblocking tasks.
- **Practical Activity:**
  - Implement a polling read API `uint16_t adc_read_value(void)`.
  - Implement an interrupt-driven ADC task signaling completion.
- **Code/Circuit Reference:**

```
uint16_t adc_read_value(void) {
 // Start conversion (SWSTART bit 30 in CR2)
 *ADC1_CR2 |= (1U << 30);

 // Wait until conversion complete (EOC bit 1 in SR)
 volatile uint32_t *sr = (volatile uint32_t*)0x40012000U; // ADC_SR
 while (!(*sr & (1U << 1)));

 return (uint16_t)(*ADC1_DR & 0xFFF); // Return 12-bit reading
}
```

## Wednesday: Pulse-Width Modulation (PWM) Actuator Control

- **Conceptual Objective:** Master Pulse-Width Modulation (PWM)—a technique to generate simulated analog output voltages by toggling digital pins at high frequencies, controlling the **Duty Cycle** (ratio of High time to total period). Study Timer Capture/Compare Mode Registers (CCMR) and Capture/Compare Enable Register (CCER) configuration.
- **Practical Activity:**
  - Configure TIM3 Channel 1 to generate a 1kHz PWM signal.
  - Route the output to Pin PA6 (Timer 3 Channel 1 alternate function AF2).
- **Code/Circuit Reference:**

```

#define TIM3_BASE 0x40000400U
#define TIM3_CR1 ((volatile uint32_t*)(TIM3_BASE + 0x00U))
#define TIM3_PSC ((volatile uint32_t*)(TIM3_BASE + 0x28U))
#define TIM3_ARR ((volatile uint32_t*)(TIM3_BASE + 0x2CU))
#define TIM3_CCR1 ((volatile uint32_t*)(TIM3_BASE + 0x34U)) // Duty cycle
control
#define TIM3_CCMR1 ((volatile uint32_t*)(TIM3_BASE + 0x18U))
#define TIM3_CCER ((volatile uint32_t*)(TIM3_BASE + 0x20U))

void pwm_tim3_ch1_init(void) {
 ((volatile uint32_t)0x40023840U) |= (1U << 1); // Enable TIM3 Clock
 *TIM3_PSC = 15; // Clock ticks at 1MHz
 *TIM3_ARR = 999; // Period = 1ms = 1kHz PWM

 // Configure PWM Mode 1 (bits 6:4 = 110) and enable output preload (bit 3)
 *TIM3_CCMR1 = (6U << 4) | (1U << 3);

 // Enable Channel 1 Output Compare (bit 0 in CCER)
 *TIM3_CCER |= (1U << 0);

 *TIM3_CCR1 = 500; // Default 50% duty cycle (500/1000)
 *TIM3_CR1 |= (1U << 0); // Enable Timer
}

```

## Thursday: ADC to PWM Closed-Loop RTOS Design

- **Conceptual Objective:** Study closed-loop feedback designs. Design an RTOS task pipeline where an analog sensor reading dynamically scales the speed or brightness of an actuator (LED/Motor) using the PWM duty cycle.
- **Practical Activity:**
  - Create a sensor polling task (Priority High) and an actuator controller task (Priority Low).
  - Route values safely via a message queue.
- **Code/Circuit Reference:**

```

void vControlTask(void *pvParameters) {
 uint16_t adc_val;
 while (1) {
 if (xQueueReceive(xAdcQueue, &adc_val, portMAX_DELAY) == pdTRUE) {
 // Map 12-bit ADC (0-4095) to 10-bit PWM Period (0-999)
 uint32_t duty = (adc_val * 999U) / 4095U;
 *TIM3_CCR1 = duty; // Dynamically update PWM output compare register
 }
 }
}

```

## Friday: Hardware Diagnostics and Verification

- **Conceptual Objective:** Learn how to verify analog inputs and PWM waveforms using debugging gear.
- **Practical Activity:**
  - Capture the PWM output on a physical logic analyzer.
  - Verify that changing the analog input voltage (via a potentiometer) dynamically and linearly modifies the duty cycle Bit Width timing on the analyzer trace.



## Hands-On Assignment: RTOS Analog Dimmer Controller

### Functional Requirements

Create a fully modular FreeRTOS application that implements an Analog LED Dimmer: an analog input dynamically controls the brightness of an LED using register-level PWM adjustments.

1. Initialize **ADC1 Channel 1** on Pin **PA1** (potentiometer analog input source).
2. Configure **TIM3 Channel 1** to output a **2kHz PWM signal** on Pin **PA6** (routes to PD12 LED or external LED).
3. Spawn two tasks:
  - **vAnalogReaderTask (Priority 2):** Polls the ADC every 50ms, reads the 12-bit value, and writes it to `xSharedAdcQueue`.
  - **vDimmerControllerTask (Priority 3):** Blocks on `xSharedAdcQueue`, reads values, calculates duty scale, and updates `TIM3_CCR1`.
4. Implement a watchdog safety task: if no ADC value is received for 1 second (timeout), force the PWM duty cycle to `0` (turn off actuator) and print an error alert.
5. Provide a Makefile that builds the program cleanly.

### Structural Requirements & Code Skeleton

`main.c` (C-Skeleton):

```

#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include <stdint.h>
#include <stdio.h>

QueueHandle_t xSharedAdcQueue = NULL;

void vAnalogReaderTask(void *pvParameters) {
 adc1_init_pa1();
 while (1) {
 uint16_t val = adc_read_value();
 xQueueSend(xSharedAdcQueue, &val, 0);
 vTaskDelay(pdMS_TO_TICKS(50));
 }
}

void vDimmerControllerTask(void *pvParameters) {
 pwm_tim3_ch1_init();
 uint16_t adc_val;

 while (1) {
 // Block with a 1-second timeout (watchdog safety check)
 if (xQueueReceive(xSharedAdcQueue, &adc_val, pdMS_TO_TICKS(1000)) == pdTRUE) {
 // Scale: 4095 → 999 (MAX ARR)
 uint32_t duty = (adc_val * 999U) / 4095U;
 *TIM3_CCR1 = duty; // Update PWM duty cycle
 } else {
 // Watchdog Timeout: Turn off LED
 printf("[WARNING] Sensor inactive. Actuator safety shutoff activated.\n");
 *TIM3_CCR1 = 0;
 }
 }
}

```



## Deliverables

- [ ] Driver source files `main.c` and compilation `Makefile` .
- [ ] Captured Logic Analyzer trace report `pwm_duty.log` proving varying duty cycles (e.g., captures at 25%, 50%, 75% duty states).



---

## ? Self-Check Questions

---

1. Why is a Sample Time parameter needed inside an Analog-to-Digital Converter? What happens if sample times are set too short for high-impedance analog sources?
  2. Walk through the mathematics required to configure a 2kHz PWM wave using a Timer clocked at 16MHz. Calculate PSC and ARR values.
  3. Differentiate **PWM Mode 1** from **PWM Mode 2** inside the Timer CCMR registers configurations.
  4. What is a Watchdog task? Why is it critical inside industrial environments to have automatic fallback timeouts that disable actuator outputs?
  5. How does DMA-driven ADC reading prevent task latency over regular software polling loops?
- 

## Stretch Task (Optional)

---

Extend the system to support a **Closed-Loop PI Controller**: write a software proportional-integral control loop that reads motor speed feedback and dynamically adjusts the PWM duty cycle to maintain a target speed.

## Motivation Checkpoint

---

Inside chemical plants, robotics lines, and automotive systems, microcontrollers do not just read parameters—they execute changes. Whether you are driving a fuel injector solenoid, a steering motor actuator, or reading temperature scales, you must coordinate analog sensors to duty-cycle actuators. Direct register programming guarantees footprint safety and real-time reliability.

## Trainer Review Rule

---

### • Pass Criteria:

- ADC and PWM configured cleanly strictly via register registers.
- Dimmer functions correctly without timing lockups.
- Watchdog safety shutdown triggers correctly on sensor disconnects.

### • Fail Criteria:

- Polling timers inside task loops without RTOS delay yields.
- Missing bounds validations.

# Weekly Execution Plan: Week 34

---

## Weekly Goal

---

Master low-level network programming in C using standard POSIX **Socket APIs**, understand TCP/UDP protocol stacks, build highly reliable Client-Server communication systems, and manage concurrent socket connections.

## Weekly Outcome

---

By Friday, the student will have designed and implemented a **Multi-Client TCP Telemetry Server** in C. They will understand socket states (bind, listen, accept), read/write network buffers safely, handle client disconnections gracefully, and parse custom application protocol frames.

---



## Session Breakdown (Monday – Friday)

---

### Monday: The TCP/IP Layer Stack & POSIX Socket Concepts

- **Conceptual Objective:** Master the TCP/IP networking model (Physical, Network - IP, Transport - TCP/UDP, Application). Understand POSIX **Sockets**—virtual file descriptors acting as endpoints for network data streams. Differentiate between:
  - **Connection-Oriented (TCP):** High reliability, sliding windows, flow control, guarantees packet order.
  - **Connectionless (UDP):** Unreliable, low latency, best for high-frequency streaming.
- **Practical Activity:**
  - Sketch a flowchart showing the step-by-step Socket call sequence for a TCP Server and TCP Client.
  - Differentiate IP address formats (IPv4 vs IPv6) and byte-ordering macros.
- **Code/Circuit Reference:**

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

// Byte order helper: Host-to-Network-Short (converts port to big-endian network byte
order)
uint16_t net_port = htons(8080);
```

## Tuesday: Low-Level TCP Server Construction (socket, bind, listen, accept)

- **Conceptual Objective:** Master the server-side socket setup sequence:
- `socket()` : Allocates the socket descriptor.
- `bind()` : Binds the socket to a local IP address and Port.
- `listen()` : Configures the socket to act as a passive listener, setting the connection backlog.
- `accept()` : Blocks until a client connects, returning a **new socket descriptor** dedicated to communicating with that client.
- **Practical Activity:**
  - Write a C program initializing a passive TCP listener.
  - Add robust error checks for every system call.
- **Code/Circuit Reference:**

```

#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <unistd.h>

void init_server(void) {
 int server_fd = socket(AF_INET, SOCK_STREAM, 0);
 struct sockaddr_in address;
 address.sin_family = AF_INET;
 address.sin_addr.s_addr = INADDR_ANY; // Bind to any interface
 address.sin_port = htons(8080);

 bind(server_fd, (struct sockaddr*)&address, sizeof(address));
 listen(server_fd, 5); // Backlog of 5 connections

 printf("Server listening on port 8080 ... \n");
 int client_fd = accept(server_fd, NULL, NULL); // Blocks here
 printf("Client connected!\n");
 close(client_fd);
 close(server_fd);
}

```

## Wednesday: Read/Write Transactions & Client Setup

- **Conceptual Objective:** Master network buffer reads and writes. Differentiate standard file read/writes from socket I/O: network packets are fragmented, meaning a single `read()` call may return only a partial segment of a sent string. You must implement robust buffer aggregation loops to guarantee complete frame parsing.
- **Practical Activity:**
  - Write a C TCP Client application that connects ( `connect()` ) to the server, sends a message, and reads the reply.
  - Verify buffer data flows.
- **Code/Circuit Reference:**

```
// Robust function to read exactly 'n' bytes from a socket
ssize_t readn(int fd, void *vptr, size_t n) {
 size_t nleft = n;
 ssize_t nread;
 char *ptr = vptr;

 while (nleft > 0) {
 if ((nread = read(fd, ptr, nleft)) < 0) {
 if (errno == EINTR) nread = 0; // Handle interrupts
 else return -1;
 } else if (nread == 0) {
 break; // EOF (connection closed by peer)
 }
 nleft -= nread;
 ptr += nread;
 }
 return (n - nleft); // Return bytes read
}
```

## Thursday: Concurrency and Multi-Client Support

- **Conceptual Objective:** Differentiate between Single-client blocking servers and Concurrent servers. A single-client server cannot accept new connections while communicating with an active client. Study multi-threaded designs: spawning a dedicated worker thread via `pthread_create()` for every accepted client connection.
- **Practical Activity:**
  - Write a multi-threaded TCP server.
  - Verify that multiple client terminals (e.g., Telnet/NC instances) can connect and communicate simultaneously.
- **Code/Circuit Reference:**

```
// Inside multi-threaded server loop
while (1) {
 int *client_fd = malloc(sizeof(int));
 *client_fd = accept(server_fd, NULL, NULL);

 pthread_t thread;
 // Spawn worker thread to handle communication concurrently
 pthread_create(&thread, NULL, client_handler, (void*)client_fd);
 pthread_detach(thread); // Detach to release thread resources automatically
}
```

## Friday: Protocol Framing and Network Debugging

- **Conceptual Objective:** Understand that data sent over TCP has no boundaries (stream-based). To parse messages cleanly, you must define **Application Protocol Frames** (e.g., prefixing packets with a header containing a start byte and payload length). Learn to debug sockets using `netstat` and `telnet`.
- **Practical Activity:**
  - Connect a client and monitor transactions.
  - Use `netstat` to trace socket states (LISTEN, ESTABLISHED, TIME\_WAIT).
- **Code/Circuit Reference:**

```
List active TCP connections and ports
netstat -tlnp

Connect to target server using Netcat
nc localhost 8080
```

## Hands-On Assignment: Concurrent Industrial Telemetry Server

### Functional Requirements

Create a robust, multi-threaded C Telemetry Server that parses custom protocol data frames concurrently from multiple remote clients. 1. Define a packed Telemetry Frame packet containing: - Start Flag ( `0x55` ) - 1 byte. - Device ID - 4 bytes. - Temperature Float value - 4 bytes. - CRC-8 Checksum - 1 byte. 2. Build a multi-threaded TCP Server `telemetry_server` listening on port `9090`. 3. For every accepted connection, spawn a thread to parse incoming frames. 4. The worker thread must: - Implement a buffer reading loop to collect exactly 10 bytes (frame size). - Verify the Start Flag and calculate the CRC-8 checksum. - If valid, print the parsed values cleanly to standard output and log them to `network_telemetry.log`. - If invalid or connection closed, exit the thread cleanly and close the socket descriptor. 5. Write a python or C client simulator `sensor_client` that connects to the server, transmits 5 valid frames, 1 corrupted frame (invalid CRC), and disconnects. 6. Provide a Makefile that builds the server cleanly.

### Structural Requirements & Code Skeleton

`telemetry_server.c` (C-Skeleton):

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <pthread.h>
#include <errno.h>

#define PORT 9090
#define START_FLAG 0x55

typedef struct __attribute__((packed)) {
 uint8_t start_flag;
 uint32_t device_id;
 float temperature;
 uint8_t checksum;
} frame_t;

uint8_t calculate_crc8(const uint8_t *data, size_t len) {
 uint8_t crc = 0x00;
 for (size_t i = 0; i < len; i++) {
 crc ^= data[i];
 for (int j = 0; j < 8; j++) {
 if (crc & 0x80) crc = (crc << 1) ^ 0x07;
 else crc <<= 1;
 }
 }
 return crc;
}

void* handle_client(void *arg) {
 int client_fd = *(int*)arg;
 free(arg);
 frame_t frame;

 printf("[SERVER THREAD] Handling client socket %d\n", client_fd);

 while (1) {
 // Read complete 10-byte frame
 ssize_t bytes = recv(client_fd, &frame, sizeof(frame_t), MSG_WAITALL);
 if (bytes <= 0) {

```

```

 printf("[SERVER THREAD] Client disconnected or read error.\n");
 break;
 }

 // Verify frame
 if (frame.start_flag != START_FLAG) {
 printf("[SERVER THREAD] ERROR: Invalid Start Flag 0x%02X\n",
frame.start_flag);
 break;
 }

 uint8_t calc_crc = calculate_crc8((uint8_t*)&frame, sizeof(frame_t) - 1);
 if (calc_crc != frame.checksum) {
 printf("[SERVER THREAD] ERROR: Checksum mismatch. Calc: 0x%02X, Recv:
0x%02X\n",
 calc_crc, frame.checksum);
 continue; // Skip corrupted frame
 }

 printf("[DATA RECEIVED] Dev: %u | Temp: %.2f C | Checksum: OK\n",
 frame.device_id, frame.temperature);
}

close(client_fd);
return NULL;
}

```

---

## Deliverables

- [ ] Server source file `telemetry_server.c`.
  - [ ] Client simulator file `sensor_client.py` (or C).
  - [ ] Compilation Makefile.
  - [ ] Execution log `network_telemetry.log` showing concurrent connections and frame parsing.
- 

## Self-Check Questions

1. Why does TCP require a **Three-Way Handshake** to establish a connection? How do the SYN and ACK packets coordinate states?



2. Why is raw TCP called a "byte stream" protocol? How do application headers prevent data fragmentation issues during parsing?
  3. What is the role of `htons()` and `ntohl()` macros? Why is network byte ordering strictly Big-Endian?
  4. What is the purpose of detaching a POSIX thread using `pthread_detach()`?
  5. How does the `select()` system call enable single-threaded servers to monitor multiple client sockets concurrently?
- 

## Stretch Task (Optional)

---

Extend the TCP Server to support **Non-Blocking I/O** and multiplexing using the modern Linux `epoll` system call suite ( `epoll_create1` , `epoll_ctl` , `epoll_wait` ) to handle up to 1000 concurrent client simulations in a single execution thread.

## Motivation Checkpoint

---

Industrial edge gateways, smart home hubs, and internet servers do not execute standalone calculations; they are networked. They process sensor streams from client nodes, manage REST APIs, and upload data to central servers. Mastering POSIX sockets, multi-threading, and protocol framing is a core skill for any systems engineer.

## Trainer Review Rule

---

### • Pass Criteria:

- TCP server compiles cleanly and handles multiple concurrent netcat connections.
- Sockets close cleanly on exits, leaving no orphan ports locked in TIME\_WAIT states.
- Custom protocol frames are verified correctly (corruptions are dropped).

### • Fail Criteria:

- Server halts execution completely if a single client disconnects.
- Absence of dynamic multi-threaded worker spawns.

# Weekly Execution Plan: Week 35

---

## Weekly Goal

---

Master advanced embedded systems debugging, memory leak profiling, system call tracking, and performance diagnostics using `gdbserver`, `valgrind`, `strace`, `ldd`, `nm`, and `perf`.

## Weekly Outcome

---

By Friday, the student will have conducted a comprehensive **Systems Diagnostics Audit** on a compiled application. They will be able to locate memory access errors, track dynamic libraries, profile kernel system calls in real-time, execute remote source-level debugging, and identify slow code loops using performance counters.

---



## Session Breakdown (Monday – Friday)

---

### Monday: System Call Tracing via `strace` & Library Analysis ( `ldd` )

- **Conceptual Objective:** Master the diagnostic utilities for system execution:
- `strace` : Intercepts and logs all system calls triggered by a running process, providing immediate visibility into kernel interactions (e.g., file opens, network socket transactions, memory allocations).
- `ldd` : Lists all dynamic shared library ( `.so` ) dependencies required by an executable.
- **Practical Activity:**
  - Profile a compiled binary using `ldd` and track where libraries reside in the host FHS directory.
  - Run `strace` on a networking binary to trace the socket system calls.
- **Code/Circuit Reference:**

```
List dynamic shared libraries of an executable
ldd build/app

Trace specific system calls (e.g., only file and memory calls)
strace -e trace=open,read,mmap ./build/app
```

## Tuesday: Memory Auditing via `valgrind`

- **Conceptual Objective:** Master memory diagnostic suites. Learn how **Valgrind (Memcheck)** instruments compiled code to detect critical memory safety violations in real-time:
- Memory leaks (allocating memory via `malloc` and forgetting to `free`).
- Use of uninitialized values.
- Out-of-bounds array reads and writes.
- Double-frees.
- **Practical Activity:**
- Write a C program containing memory safety bugs.
- Run the application under Valgrind, analyze the stack trace logs, and resolve the errors.
- **Code/Circuit Reference:**

```
Compile with debug symbols active (-g) and optimization disabled (-O0)
gcc -g -O0 src/buggy_app.c -o build/buggy_app

Run Valgrind with complete leak tracking
valgrind --leak-check=full --show-leak-kinds=all ./build/buggy_app
```

## Wednesday: Remote Debugging via `gdbserver`

- **Conceptual Objective:** Understand that embedded targets have limited resources: you cannot run a giant graphic IDE or compile debugging structures directly on the target. Learn the **GDB Server** design pattern:
- Run `gdbserver` on the lightweight target embedded board, binding it to a target port.
- Run the full source-level `gdb` debugger on the powerful host development PC, connecting remotely to the target board over Ethernet.
- **Practical Activity:**
- Start `gdbserver` on target board.
- Connect from host PC using `gdb-multiarch`.
- Step through code, inspect registers, and set breakpoints remotely.
- **Code/Circuit Reference:**

```
Target Board terminal: start gdbserver on port 1234
gdbserver :1234 ./build/app

Host PC terminal: connect remotely
gdb-multiarch ./build/app
(gdb) target remote <target_ip>:1234
(gdb) break main
(gdb) continue
```

## Thursday: Symbol Inspections ( `nm` & `objdump` )

- **Conceptual Objective:** Study binary symbol tables and disassemblers. Understand how `nm` dumps symbol names, types, and addresses, and how `objdump` disassembles raw binary sections back to assembly instructions.
- **Practical Activity:**
  - Run `nm` to isolate global and static functions in a compiled binary.
  - Disassemble specific sections and locate the physical assembly instructions of C loops.
- **Code/Circuit Reference:**

```
List all symbols sorted by size
nm --size-sort -S build/app

Disassemble the text segment showing source code inline
objdump -S -d build/app > build/disassembly.txt
```

## Friday: System Performance Profiling ( `perf` )

- **Conceptual Objective:** Master the Linux `perf` utility—the official tool for performance auditing. Understand how `perf` uses CPU hardware performance counters to sample system executions, identifying **Hotspots** (the exact functions or loops where the CPU spends most of its execution cycles).
- **Practical Activity:**
  - Run `perf record` on a CPU-intensive calculations binary.
  - Generate a performance report using `perf report` and isolate the execution hotspots.
- **Code/Circuit Reference:**

```
Record system execution statistics
perf record -g ./build/app

Analyze performance report
perf report
```

---

## Hands-On Assignment: Custom Systems Diagnostics Audit

---

### Functional Requirements

Create a fully integrated, multi-file C database application containing intentional memory safety and efficiency bugs. Then, execute a complete systems diagnostics audit to resolve all errors. 1. The C application `audit_app.c` must: - Allocate memory dynamically inside a loop. - Intentionally contain a memory leak (forgetting to free a subset of allocations). - Intentionally read past an array boundary. - Execute a highly inefficient CPU-bound calculation loop. 2. Compile the application with debug symbols active ( `-g -O0` ). 3. Conduct the Systems Audit using host utilities, generating an audit report `diagnostics_audit.txt` containing: - **Shared Libraries:** Output of `ldd` listing dynamic dependencies. - **System Call Footprint:** Output of `strace` isolating total file opens and memory mappings. - **Memory Audit:** Output of `valgrind` identifying the exact line numbers of the memory leak and the out-of-bounds read, followed by code modifications resolving them. - **Performance Audit:** Output of `perf report` isolating the hotspot function. 4. Resolve all bugs, re-compile, and re-run Valgrind to prove a clean execution (zero memory leaks, zero errors).

---

### Deliverables

---

- [ ] Buggy source file `audit_app.c` and corrected file `safe_app.c` .
  - [ ] Compilation Makefile.
  - [ ] Systems diagnostics audit report `diagnostics_audit.txt` .
- 

## Self-Check Questions

---

1. Why does `valgrind` require compilation flag `-O0` and `-g` to print accurate debug reports? What happens if you run it on a stripped `-O3` binary?
2. Differentiate the roles of `strace` versus `valgrind` . Which tool is used to monitor kernel-space interactions and which is used for user-space memory safety?
3. How does remote debugging using `gdbserver` optimize resources? Explain the host-target connection model.

4. What is a **memory leak**? Trace how a memory leak degraded system performance over time.
  5. How does `perf` determine CPU execution hotspots? Differentiate sampling rate statistics from direct execution tracing.
- 

## Stretch Task (Optional)

---

Extend the remote debugging pipeline to configure **GDB inside VS Code**: map a custom launch configuration that automatically launches the remote target `gdbserver` connection over SSH, allowing graphic breakpoints mapping directly on host workspaces.

## Motivation Checkpoint

---

A junior developer spends hours inserting `printf` statements to debug a crash. A senior systems engineer runs `strace` and `valgrind`, isolates the exact system call failure or memory leak address in 2 minutes, and resolves the issue programmatically. Master the professional diagnostics suite to master systems engineering.

## Trainer Review Rule

---

### • Pass Criteria:

- Audit report cleanly isolates and documents all memory leaks, system calls, and hotspots.
- Final corrected application compiles without warnings and runs cleanly under Valgrind (0 leaks, 0 errors).

### • Fail Criteria:

- Unresolved memory leaks or out-of-bounds reads in the final target.
- Monolithic diagnostics reports lacking structural details.

# Weekly Execution Plan: Week 36

---

## Weekly Goal

---

Deliver the multi-tiered Capstone Production Project (Industrial IoT Gateway), execute complete system validations, customize your professional engineering resume, and complete system-design mock interviews.

## Weekly Outcome

---

By Friday, the student will have finalized their **Industrial IoT Gateway Capstone Project**, linking a downstream real-time sensor node (FreeRTOS) with an upstream Linux Gateway (Yocto-built) communicating over secure sockets. They will defend their portfolio in a mock technical interview and stand 100% prepared for market entry as a highly competitive Systems Engineer.

---

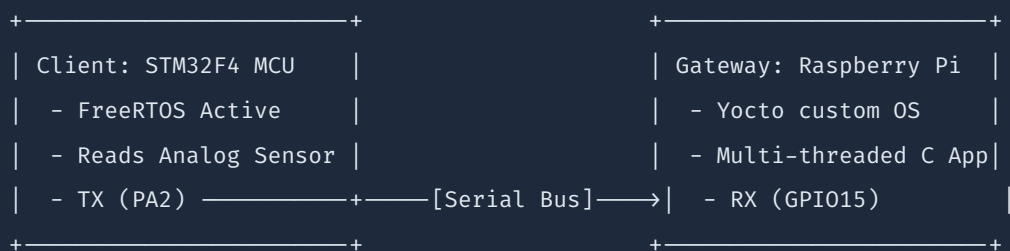
## Session Breakdown (Monday - Friday)

---

### Monday: Capstone Integration & Communication Setup

- **Conceptual Objective:** The project bridges all milestones built throughout the 36 weeks, combining a real-time bare-metal/RTOS microcontroller client with an embedded Linux Yocto-built gateway.
- **Practical Activity:**
  - Assemble hardware nodes: mount target microcontroller (STM32F4) and gateway board (Raspberry Pi/BBB) onto a shared breadboard.
  - Wire UART communication lines between the nodes, linking grounds safely.
- **Code/Circuit Reference:**

Capstone Hardware Topology:



## Tuesday: Multi-Threaded Gateway Processing & Socket Streaming

- **Conceptual Objective:** Master the gateway-side processing logic. The gateway runs a multi-threaded user-space daemon (compiled via a custom Yocto recipe) that reads raw sensor packets from the UART serial port, verifies integrity, and publishes statistics via secure TCP/IP sockets to a server dashboard.
- **Practical Activity:** Write the gateway processing daemon in C. Test concurrent client socket connections.
- **Code/Circuit Reference:**

```
void* serial_reader_thread(void *arg) {
 int uart_fd = open("/dev/ttyS0", O_RDONLY | O_NOCTTY);
 // Read loop, parse frames, stream to active socket clients
}
```

## Wednesday: System Optimization & Boot Audits

- **Conceptual Objective:** Study boot-time optimization patterns. A production embedded system must boot extremely fast. Learn how to audit systemd startup sequences using `systemd-analyze` and strip unnecessary initialization scripts to achieve a total boot-time to user-space main application under 8 seconds.
- **Practical Activity:** Profile system boot time using `systemd-analyze time` and `systemd-analyze blame`. Disable unneeded system services and verify improvements.
- **Code/Circuit Reference:**

```
Profile boot time
systemd-analyze time
systemd-analyze blame
```

## Thursday: Portfolio Defense & Resume Optimization

- **Conceptual Objective:** Understand how to present your physical engineering portfolio to hiring managers. The best resume is a clean GitHub repository containing well-structured, production-grade code, detailed READMEs, and logic analyzer captures of functional protocols.
- **Practical Activity:** Clean up your workspace directories. Ensure all files are styled, warning-free, and follow Conventional Commits. Update your professional resume to highlight direct hardware competencies.



## Friday: System Design Mock Interviews & Graduation

- **Conceptual Objective:** Stand ready to face technical interview pipelines. Embedded interviewers do not test simple algorithms; they grill candidates on physical registers, memory maps, concurrency, interrupt latency, and driver architectures.
  - **Practical Activity:** Conduct a mock interview covering the core topics of the curriculum. Execute the final Capstone graduation defense.
- 



## Hands-On Assignment: Industrial IoT Gateway Capstone Project

---

### Functional Requirements

Deliver, compile, and document the complete **Industrial IoT Gateway Capstone System**.

1. **Microcontroller Node (FreeRTOS):** - Polls an analog sensor (ADC) every 100ms. - Packages data into a packed frame with a Start Byte ( `0xAA` ) and a CRC-8 checksum. - Transmits frames via USART TX.
2. **Gateway Node (Yocto Custom OS):** - Run a custom Yocto-built Linux OS containing your proprietary layer. - Launch a background C systemd service `gateway_daemon` that reads raw packets from `/dev/ttyS0` (UART Rx), decodes and audits them. - Expose a multi-threaded TCP socket on Port `9090` . When a client connects (e.g., your host PC running a python plotter), stream parsed telemetry in real-time.
3. **Boot Optimization:** Audit the system boot flow. Reduce total boot time under 8 seconds, documenting systemd blame stats.
4. **Graduation Report:** Write a 5-page README.md for your portfolio repo detailing: block diagrams, register layouts, custom linker scripts, thread configurations, Yocto recipes, and logic analyzer timings captures.

---



### Deliverables

---

- [ ] Full Client source code (FreeRTOS STM32F4).
  - [ ] Full Gateway source code and custom Yocto recipes directory ( `meta-industrial-gateway` ).
  - [ ] Comprehensive Capstone Graduation Report `walkthrough.md` embedded with timing captures.
- 



### Self-Check Questions

---

1. Differentiate the execution scopes of your microcontroller client (RTOS) versus your gateway node (Embedded Linux). How do they cooperate in the system topology?
2. How did you optimize the Yocto boot sequence? Which services did you disable and what boot-time speedup was achieved?

3. Walk through the thread sync model of your gateway daemon: how are client sockets kept thread-safe during high-rate data streams?
  4. What is a **Watchdog Timer (WDT)**? How do you configure a hardware watchdog to auto-reset the gateway if the software daemon freezes?
  5. Outline the exact memory relocation sequence of a compiled C variable from Flash to SRAM at system boot.
- 



## Graduation Checkpoint

---

Congratulations! You have completed a highly rigorous, first-principles embedded systems and Linux curriculum. You did not use graphical configurators or vendor HAL frameworks; you configured clocks, built custom allocators, wrote POSIX multithreading pipelines, compiled custom kernels, and registered dynamic kernel character drivers. You possess the elite, first-principles systems engineering skills sought by world-class tech firms. Welcome to the industry!



## Trainer Review Rule

---

- **Pass Criteria:**

- Entire integrated system operates concurrently without dropouts.
- Boot time is optimized under 8 seconds.
- Code compiles cleanly with zero warnings under `-Wall -Wextra -Werror`.
- Capstone report is complete and professional.

- **Fail Criteria:**

- Missing any functional block.
- Monolithic single commits or warning compile outputs.